Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

volclava 管理员手册

Product Name : volclava

Product Version : 2.2.1

Release Date : 2026.10.30

Contributor： @李明泽（limingze.jiayou@bytedance.com）

@舒光波（shuguangbo@bytedance.com）

@亓楠（qinan.cn@bytedance.com）

@周灵汛（zhoulingxun@bytedance.com）

@李艳青（liyanqing.1987@bytedance.com）

@陆尹凯（ luyinkai@bytedance.com)

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

目录

1. 集群的管理 6

1.1 关于 VOLCLAVA 6

1.1.1 相关名词解释 6

1.1.2 volclava 服务进程 8

1.1.3 用户与管理员 11

1.1.4 负载阈值 11

1.1.5 资源需求 12

1.1.6 作业生命周期 13

1.1.7 作业的提交 14

1.1.8 作业的调度与派发 16

1.1.9 节点机选择 17

1.1.10 作业的执行环境 17

1.1.11 容错 18

1.2 管理集群 19

1.2.1 检查集群的信息 19

1.2.2 目录结构的示例 21

1.2.3 集群的管理员 21

1.2.4 控制 volclava 的后台进程 21

1.2.5 控制 mbatchd 进程 23

1.2.6 重新设置你的集群 25

1.3. 管理节点 27

1.3.1 节点状态 27

1.3.2 检查节点信息 29

1.3.3 控制节点 29

1.3.4 增加一个节点 31

1.3.5 移除一个节点 31

1.3.6 向 lsf.shared 中添加节点类型与型号 32

1.3.7 登记服务端口 33

1.3.8 节点命名 34

1.3.9 节点组 34

1.4. 管理队列 35

1.4.1 队列状态 35

1.4.2 检查队列信息 36

1.4.3 控制队列 36

1.4.4 增加与移除队列 38

1.4.5 管理队列 39

1.5. 管理作业 39

1.5.1 理解作业的状态 39

1.5.2 检查作业的信息 42

1.5.3 检查历史作业信息 47

1.5.4 在队列中改变作业顺序 50

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.5.5 把作业从一个队列转移到另一个队列 *51*
1.5.6 强制作业执行 *52*
1.5.7 挂起与恢复作业 *52*
1.5.8 取消作业 *53*
1.6. 管理用户 53
1.6.1 检查用户与用户组信息 *53*
1.6.2 用户组 *54*
1.6.3 取消 default 对用户组的影响 *56*
2. 资源的操作 **59**
2.1 理解资源 59
2.1.1 volclava 资源 *59*
2.1.2 资源如何分类 *60*
2.1.3 如何使用资源 *61*
2.1.4 负荷指数 *63*
2.1.5 静态资源 *66*
2.1.6 自动检测硬件配置 *68*
2.2 添加新资源 68
2.2.1 如何给集群添加新资源 *68*
2.2.2 如何配置 lsf.shared 的 Resource 段 *69*
2.2.3 如何配置 lsf.cluster.cluster_name 中的 Host 段 *70*
2.2.4 配置 lsf.cluster.cluster_name 中 ResourceMap 段 *71*
2.3 资源分配限制 71
2.3.1 关于资源分配限制 *71*
2.3.2 配置资源使用限制 *73*
2.3.3 通用资源配额限制 *74*
2.4 预留资源 81
2.4.1 关于资源预留 *81*
2.4.2 使用资源预留 *82*
2.4.3 定义资源预留方式 *83*
2.4.4 查看资源预留信息 *83*
3. 作业调度和派发 **84**
3.1 作业调度策略 84
3.1.1 先来先服务（FCFS） *84*
3.1.2 公平共享调度策略 *84*
3.1.3 用户指定的作业优先级 *99*
3.2 派发及运行窗口 100
3.2.1 派发及运行窗口 *100*
3.2.2 运行窗口 *100*
3.2.3 派发窗口 *101*
3.3 作业重队列和作业重运行 101
3.3.1 关于作业队列重排 *101*
3.4 作业组 103

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

3.4.1 创建作业组 *103*
3.4.2 处理输入、输入文件 *104*
3.4.3 查看作业组 *105*
3.4.4 控制作业组 *105*
3.4.5 重新排队作业组 *106*
3.4.6 作业组的作业槽限 *106*
3.5 运行并行作业 107
3.5.1 提交并行作业 *107*
3.5.2 并行作业的作业槽限制 *107*
3.5.3 指定处理器的最小、最大数 *108*
3.5.4 控制跨机器的处理器分配 *108*
3.5.5 限制分配的处理器数 *109*
3.6 作业包 112
3.6.1 功能描述 *112*
3.6.2 启用并配置作业包 *112*
3.7 定义最大排队任务数 113
3.7.1 功能描述 *113*
3.7.2 配置定义 *113*
3.7.3 功能效果 *115*
3.8 自定义资源限制的单位 118
3.8.1 功能描述 *118*
3.8.2 配置定义 *118*
3.8.3 功能效果 *120*

4. 控制作业的执行 126

4.1 运行时资源使用限制 126
4.1.1 关于资源使用限制 *126*
4.1.2 资源使用的单位设置 *128*
4.1.3 指定资源使用限制 *128*
4.1.4 实例 *130*
4.1.5 CPU 时间和运行时间标准化 *131*
4.2. 负载阈值 132
4.2.1 自动作业挂起 *132*
4.2.2 挂起条件 *133*
4.3. 前处理和后处理命令 136
4.3.1 关于前处理和后处理命令 *136*
4.3.2 配置前处理和后处理命令 *137*
4.4 作业启动器 140
4.4.1 关于作业启动器 *140*
4.4.2 命令级作业启动器 *141*
4.4.3 队列级作业启动器 *142*
4.4.4 使用作业启动器控制运行环境 *143*
4.5 外部作业提交和执行控制 144
4.5.1 理解外部执行体 *144*

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

4.5.2 使用 esub *145*
4.5.3 缺省的作业控制动作 *148*
4.6 自定义作业的工作目录 150
4.6.1 CWD 功能说明 *150*
4.6.2 lsb.params 参数配置说明 *151*
4.6.3 cwd 维护机制说明 *152*
5. 交互式作业 152
5.1 用 BSUB 来提交交互式作业 152
5.1.1 交互式作业 *152*
5.1.2 提交交互式作业 *153*
5.1.3 交互式批处理作业信息 *155*
5.1.4 用 bsub 运行 X 应用 *156*
5.1.5 写作业脚本 *156*
6. 集群的监控 158
6.1 错误与事件日志 158
6.1.1 系统目录和日志文件 *158*
6.1.2 错误日志管理 *160*
6.1.3 系统事件日志 *161*
6.1.4 复制事件日志记录 *162*
6.1.5 volclava 作业退出码 *164*
6.2. 故障排除和错误信息 164
6.2.1 访问共享文件 *164*
6.2.2 volclava 上的常见问题 *165*
附录： 170
附一、变更历史 170

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1. 集群的管理

1.1 关于 volclava

volclava 是 100%免费、开源、兼容 IBM LSF 的工作负载调度器，支持各种高性能计算和分析应用。

当前 volclava 基于 openlava 2.0 版本进行了二次开发，修复了一些问题，并增加了新的功能支持，可以覆盖 EDA 业务基本功能需求。推荐在小规模集群（小于百台节点），业务对调度性能不高的场景使用。

volclava 的命令行和文件格式与大多数 LSF 功能相兼容，因此用户和管理员都将非常熟悉 volclava 的操作。

1.1.1 相关名词解释

1. **集群**：一群或一组计算机，通过运行 volclava 来共同工作，从而组成一个整体，不但能把各个节点机的计算能力集合在一起，而且彼此间能够分担负载、分享资源。一个集群就是把分散的计算资源整合，向外提供一个统一的系统映像。

> 相关命令介绍：
>
> `lshosts` -- 检查集群中节点机的静态资源信息
>
> `bhosts` -- 检查集群中服务节点的资源信息与作业信息
>
> `lsid` -- 检查集群的名称
>
> 相关设置介绍： 在 `lsf.cluster.<cluster_name>`文件中，定义所有的节点机
>
> **注意**：在你的集群中，请保持集群名称的唯一性。它不能与任何节点机或队列重名。

2. **作业（job）**：作业是 volclava 系统中的工作单元。一个作业是提交到 volclava 系统中的一个执行命令。volclava 会根据所设定的策略来调度、控制和跟踪这个作业。一个作业可能是复杂问题、情景模拟、延展计算，以及任何需要计算资源的任务。

> 相关命令介绍：
>
> `bjobs` -- 检查系统中的作业信息
>
> `bsub` -- 提交作业

3. **作业槽（slots）**：作业槽就好比一个盛装作业的容器，在 volclava 系统中，一个作业槽可以盛装一个作业。依据你的设置而定，一个节点机可以含有一个或多个作业

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

槽，你可以通过队列不停地会向一组节点机分派作业，直到所有设定的作业槽全都被填满。

> 相关命令介绍：
>
> `bhosts` -- 检查一个或一组节点机上所设定的作业槽数目限制
>
> `bqueues` -- 检查队列上所设定的作业槽数目限制
>
> `busers` -- 检查用户或用户组上所设定的作业槽数目限制

4. **作业状态：**在 volclava 系统中，作业可能处于下列的状态中：

> `PEND` （待定）-- 作业在队列中等待调度与分派
>
> `RUN` （运行）-- 作业已经被分派到节点机上，正在运行
>
> `DONE` （完成）-- 作业已正常结束，正常退出值为 `0`
>
> `EXIT` （退出）-- 作业已经结束，但退出值不是 `0`
>
> `PSUSP`（待定挂起）-- 作业在待定时被挂起
>
> `USUSP`（用户挂起）-- 作业被用户指定挂起
>
> `SSUSP`（系统挂起）-- 作业被 `volclava` 系统挂起
>
> `POST_DONE`（后处理完成）-- 作业的后处理动作，成功完成
>
> `POST_ERR`（后处理出错）-- 作业的后处理动作，出错
>
> `UNKWN`（不确定）-- `mbatchd` 与执行节点机上的 `sbatchd` 失去联系， 从而使作业状态不确定
>
> `WAIT`（等待）-- 对于提交到“块作业“队列中的作业，作为“块作业”的成员所处于的等待状态
>
> `ZOMBI`（僵尸）-- 作业成为僵尸是因为系统失去了与作业执行节点机的联系

5. **队列（queues）：**队列是一个系统范围内的作业容器。作业在被分派到节点机上执行之前，所有的作业都要在队列中等待被调度。队列与单个节点机没有对应关系，一个队列可对应所有 volclava 系统中的服务节点，也可根据设定与一部分的服务节点相对应。当你提交作业到队列中时，你不必为作业指定执行节点。volclava 会把作业分派到集群中最佳的节点上执行。

> 相关命令介绍：
>
> `bqueues` -- 检查所有可用的队列
>
> `bsub -q` -- 把作业提交到指定的队列中
>
> `bparams` -- 检查默认的队列
>
> 相关设置介绍：
>
> 在 `lsb.queues` 文件中，设定各个队列
>
> **注意：**在你的集群中，请保持队列名的唯一性。它不能与任何节点机重名或也不能与集群名称相同。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

6. **服务节点：**服务节点上运行 sbatchd 守护进程，通过它来执行服务请求，实施本地策略。默认的情况下，所有针对 volclava 系统配置的修改都是静态的，就是说必须重启后才能生效。要增加或减少节点机，你必须动手修改配置，并重新启动所有的候选主节点。

```
相关命令介绍：
    lshosts -- 检查哪些节点是客户节点（server=Yes）
    lsload -- 检查节点机上的负载情况
    bhosts -- 检查批处理系统里的服务节点的情况
相关设置介绍：
    在 lsf.cluster。<cluster_name>文件中，通过将“Server“域置为“1”来设定服务节点
```

7. **客户节点：**客户节点就是只能用于提交作业的节点机。客户节点上可以运行 volclava 的命令或使用 bsub 提交作业，但不能执行作业，也不运行 volclava 守护进程。

```
相关命令介绍：
    lshosts -- 检查哪些节点是客户节点（server=No）
    bsub – 用于提交作业
    bjobs – 检查已被提交的作业的情况
相关设置介绍：
    在 lsf.cluster。<cluster_name>文件中，通过将“Server“域置为“0”来设定客户节点
```

8. **主节点：**主节点就是一个作为整个集群协调者的服务节点。在主节点机上，运行着主 LIM 进程与 mbatchd 进程。每个集群只有一个主节点，它要处理所有作业的调度与分派。如果当前的主节点机宕机，那么就会有另一个 volclava 服务节点起来担当主节点。 在主节点上，运行所有的 volclava 服务进程。其中的 LIM 进程是主 LIM 进程。

```
相关命令介绍：
    lsid -- 检查主节点的名称
```

1.1.2 volclava 服务进程

进程	作用

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

mbatchd	作业的请求与分派
query mbatchd	处理查询请求
sbatchd	作业的执行
res	作业的执行
lim	管理节点的信息
pim	管理作业的进程信息

1. **mbatchd**：是批处理系统的主服务进程，只运行在主节点机上。它是由 sbatchd 服务进程来启动的，它负责管理所有作业的作业状态。 mbatchd 接受作业提交请求，处理作业信息查询请求，管理队列中的作业，根据调度决策来分派作业到执行节点上。

2. **query mbatchd**：当使能 LSB_QMBD_PORT 后，mbatchd 会创建该查询子进程，用于分流 bjobs 等查询请求。query mbatchd 不负责作业提交、调度、分派或状态更新；这些写操作仍由主 mbatchd 负责。它的角色是定期更新的 mbatchd 的只读查询侧副本，主 mbatchd 始终是权威数据源。

3. **sbatchd**：sbatch 是批处理作业系统的从服务进程，运行于每个服务节点机上。它接收来自 mbatchd 的作业执行请求，并负责作业在本节点上的执行工作。它负责贯彻本地策略，并负责维护本节点上的作业状态。 sbatchd 会为每一个作业再启动一个 sbatch 进程。这个子 sbatchd 进程会运行一个 res 实例，以创建作业的运行环境。当作业完成后，这个子 sbatch 进程会自动退出。

```
相关命令介绍：
    badmin hstartup –    启动 sbatchd 进程
    badmin hshutdown – 停止 sbatchd 进程
    badmin hrestart –    重启 sbatchd 进程
相关设置介绍：
    在 lsf.conf 文件中，设置 sbatchd 的服务端口 LSB_SBD_PORT
```

4. **res**：是远端执行服务器的简称，运行在每一个服务节点上。它负责接收远端执行服务请求，为作业或任务提供透明的、安全的远端执行服务

```
相关命令介绍：
    lsadmin resstartup – 启动 RES 进程
    lsadmin resshutdown –停止 RES 进程
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
    lsadmin resrestart - 重启 RES 进程
 相关设置介绍：在 lsf.conf 文件中，设置 RES 的服务端口
LSF_RES_PORT
```

5. **lim** ： lim 是负载信息管理者的简称，它运行在每一个服务节点上。它负责收集节点上的负载信息与配置信息，并把这些信息传递给主节点上的主 LIM 服务进程。用户可使用 lsload 与 lshosts 命令来获得 LIM 所汇报的信息。 当 LIM 启动或 CPU 个数改变时，相关的静态指标信息会被更新，这些**静态指标**包括:

- CPU 的个数（ncpus）
- 磁盘的个数（ndisks）
- 内存总量大小（maxmem）
- 交换区总量大小（maxswp）
- 临时目录总量的大小（maxtmp）

LIM 每间隔一段时间还收集一些与节点负载相关的动态信息，这包括:

- 节点的状态（status）
- 15 秒，1 分钟，与 15 分钟的运行队列的长度（r15s，r1m，与 r15m）
- CPU 的使用率（ut）
- （内存的）页调度比率（pg）
- 登陆用户的个数（ls）
- 交互式工作模式下的空闲时间（it）
- 可用的交换空间的大小（swp）
- 可用的内存大小（mem）
- 可用的临时空间大小（tmp）
- 磁盘 IO 比率（io）

```
相关命令介绍：
  lsadmin limstartup - 启动 Lim 进程
  lsadmin limshutdown -停止 Lim 进程
  lsadmin limrestart - 重启 Lim 进程
  lslod - 检查动态的负载值
```

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
  lshosts - 检查静态的节点的各项指标
相关设置介绍：
  在 lsf.conf 文件中，设置 LIM 的服务端口 LSF_LIM_PORT
```

6. **主 LIM** :主 LIM 进程是运行在主节点机上的 LIM 服务进程，它接收其他节点上 LIM 进程所传递来的负载信息。 这些信息会通过主 LIM 进程被传递给 mbatchd 服务进程，mbatchd 会再把这些信息传递给 mbschd 服务进程，最终这些信息成为 mbschd 的决策基础。如果 LIM 不可用了，那么就会从候选主节点列表中，自动选出新的主 LIM。相关命令同上。

7. **PIM** :PIM 是进程信息管理者的简称，它运行于每一个服务节点上。它是由 LIM 进程启动的，LIM 会定期检查 PIM 的状态，如果 PIM 进程死掉了，LIM 会自动重启它。PIM 负责收集节点机上所运行的作业的进程信息，并把这些信息传递给 sbatchd 进程。这些信息包括，作业所使用的 CPU 与内存的情况等。

```
相关命令介绍： bjobs - 检查作业的信息
```

1.1.3 用户与管理员

1. **volclava 用户**：volclava 用户就是有权限向 volclava 集群提交作业的用户帐号。

2. **volclava 管理员**：一般来说，只有 volclava 集群管理员才能执行影响整个 volclava 集群的操作。每个集群都有一个首席 volclava 管理员，这是在安装时就指定的。此外，你还可以设定集群级别与队列级别的管理员。

1.1.4 负载阈值

volclava 管理员可以通过设置两种负载阀值来调度队列中的作业。每个负载阀值可以设置如下的负载指标:

```
LoadSched 用于决定派发挂起作业的负载条件。如果一个节点的负载高于任何一个 LoadSched 值，那么就不会有任何作业被派发到这个节点上来。这个阀值也是恢复挂起作业的条件标准
LoadStop 是决定把作业挂起的条件
```

要把一个作业派发到一个节点上，那么这个节点上的负载情况不仅要满足节点的负载阀值，也要满足派发作业队列的阀值。 负载指标的值可根据负载情况或增或减，

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

具体要看具体负载指标的含义。因此，当对比节点负载情况与负载阈值时，你需要使用大于或小于对比符，具体根据具体的指标而定。

```
相关命令介绍：
    bhosts -l       检查节点的挂起条件
    bqueues -l      检查队列的挂起条件
    bjobs -l          检查具体作业的挂起条件与作业挂起后恢复时所要使用的调度阈值
相关设置介绍：
    lsb.hosts –设置节点的阈值
    lsb.queues –设置队列的阈值
```

1.1.5 资源需求

资源需求用于限定作业可以在哪类节点上运行。只有符合资源需求的节点才能作为作业运行的候选节点。当 volclava 调度作业时，会收集所有候选节点的相关指标，进行对比，volclava 只会把作业派发到所有相关指标都符合的节点上。

```
相关命令介绍：
    bsub –R – 指定作业所需求资源的字符串
相关设置介绍：
    lsb.queues –设置队列的资源需求 RES_REQ
```

volclava 支持同时在作业级（通过 `bsub -R` 指定）和队列级（通过 lsb.queues 中的 `RES_REQ` 指定）配置资源请求。当两者同时存在时，按以下规则合并：

- **select**：默认情况下，作业级与队列级配置会同时生效，作业只会被派发到同时满足两者要求的节点。
    - 如果 bsub 在 select 中没有指定 type 值，默认值一般是 type == local，如下情况默认值是 type == any：
        i. bsub 指定了-m
        ii. select 中指定了字符串或布尔类型资源
    - 我们可以把 select 分成两类一般形式和 xor 形式（select[xxx,xxx]，在调度逻辑上等同与||)），如果 job 和 queue level 都指定了 select，且有出现 xor 形式，则 queue level 的 select 配置将被忽略，只使用 job level 的指定的 select string。
- **order**：作业级覆盖队列级。作业级未定义时取队列级的值；两者都未定义时，使

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

用默认值 order[r15s:pg]。

- **span**：作业级覆盖队列级。作业级未定义时取队列级的值；两者都未定义则不限制 span 要求。
- **rusage**：合并 job level 和 queue level 指定的预留资源，对于交集资源（共同指定的资源），job level 覆盖 queue level 的值，且要求 job level 的值需要小于等于 queue 值；对于 duration 值，job level 指定的值需要小于等于 queue level 指定的值；对于 decay 值，job level 指定的值需要大于等于 queue level 指定的值。如果 job level 没有指定 duration 或者 decay，则继承 queue level 指定的值。如果 job level 的 value/duration/decay 的值不符合上诉要求，job 提交将被拒绝。示例：

```
例子 1:
job-level: rusage[swp=10:mem=10]
queue-level: rusage[tmp=10:mem=20]
merged-resreq: rusage[swp=10:tmp=10:mem=10]
例子 2:
job-level: rusage[mem=10]
queue-level: rusage[tmp=10:mem=20:duration=1m:decay=0]
merged-resreq: rusage[tmp=10:mem=10:duration=1m:decay=0]
例子 3:
job-level: rusage[mem=10:duration=2m:decay=1]
queue-level: rusage[tmp=10:mem=20:duration=1m:decay=0]
merged-resreq:  rusage[tmp=10:mem=10:duration=2m:decay=1]
```

> 注意：目前 volclava rusage 语法只支持“:”方式连接不同资源和关键字，因此不同资源只能共享一个 duration 的值，所以这里 tmp 和 mem 都继承 job level 的 duration，这里的行为和 LSF 不一致，LSF 的 rusage 语法支持“，”方式，可以为不同资源指定不同的 duration。

1.1.6 作业生命周期

1. **提交作业**：你可从 volclava 客户端或服务端，使用 bsub 命令来提交作业。 如果在提交作业时没有指定队列的话，作业会被提交到默认队列中。 作业被放在队列中等待被调度，此时的状态是 PEND。作业实体保留在 VOLCLAVA_TOP/work/logdir/info 目录中。
2. **作业 ID：** 当提交作业时，volclava 会为作业分配的、唯一的作业 ID。
3. **作业名称：** 在提交作业时，你还可以使用-J 选项，为每个作业指定一个作业名称。与作业 ID 不同，作业名称不必是唯一的。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

4. **调度作业：**mbatchd 评估每个作业并根据如下的信息进行调度决策：

   - 作业优先级
   - 调度策略
   - 可用的资源情况

   为作业选择最佳的可运行节点。 资源信息的收集是由主 LIM 每隔一定的时间间隔通过各个服务节点 LIM 来完成的。主 LIM 把这个信息传递给 mbatchd。

5. **派发作业：**执行 mbatchd 的调度决策，就会把作业派发到相应节点上。

6. **运行作业：**sbatchd 负责作业的执行

   - sbatchd 接收来自 mbatchd 的请求
   - sbatchd 会为作业创建一个子 sbatchd 进程
   - sbatchd 创建相应的执行环境
   - 使用 res 启动作业

7. **返回输出：**当作业完成时，如果作业完成且没有任何错误，它的状态会被设定为 DONE； 如果作业运行出错，它的状态会被设定为 EXIT。 sbatchd 会把作业的信息传递给 mbatchd，包括作业错误与其输出。

1.1.7 作业的提交

当你提交一个作业到 volclava 系统中，这个作业的生命周期就开始了。在命令行上，bsub 是用于提交作业的命令，你可以使用很多 bsub 的参数来改变其默认行为。

1. **队列：**队列是一系列的待发作业，它们按照预定的顺序排列并等待着使用资源的机会。队列实施不同的作业调度和控制策略。所有提交到同一队列中的作业， 拥有相同的作业调度与控制策略。队列不与具体的节点机相对应；每个队列都可以使用集群中所有节点，或者可以使用集群中一个设定的子集。 队列是集群范围内的保留作业的地方。作业由 bsub 命令提交到队列中来。volclava 可以设置一个或多个默认队列。如果作业提交时没有指定特定的队列，那么作业会被放入最先接收它的默认队列中。

   队列具有如下的特点：

   - 优先级，数字越大优先级越高
   - 名称，这是队列唯一的标识
   - 队列限制，可以限制节点，作业数目，用户，组，处理器等
   - 标准 linux 限制，内存，交换区，进程，CPU 等

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

- 管理员
- 运行条件

2. **队列样例：**

```
Begin  Queue
QUEUE_NAME = normaL
PRIORITY = 30
STACKLIMIT = 2048
DESCRIPTION = 用于低优先级作业，只在节点负载低的时候才运行
QJOB_LIMIT =60 #队列中作业限制
PJOB_LIMIT =2 #每个处理器的作业限制
ut = 0.2
io = 50/240
USERS = all
HOSTS = all
NICE = 20
END Queue
```

3. **队列优先级**：队列优先级定义队列的搜索次序，以便决定哪个作业应该被处理了。队列的优先级是由 volclava 管理员来指定的，数字越大优先级越高。队列优先级决定了 volclava 系统对它们的服务次序，从高到低。如果多个队列具有相同的优先级， 那么 volclava 会把这些队列中的所有作业，按先到先服务的策略来处理。

4. **自动队列选择是如何工作的**：

   volclava 按照如下的条件选择一个适用的队列：

   - 用户的访问权限限制 – 如果队列不允许这个用户使用，那该队列就不在被考虑的范围之内
   - 节点的限制 – 如果作业明确指定一系列的执行节点，那么所选队列就必须能够向这些节点派发作业
   - 队列状态 – 处于关闭的队列不在考虑的范围之内
   - 排他性执行限制 – 如果作业需要排他性的执行，那么不能指定排他任务的队列就不在考虑的范围之内
   - 作业所需的资源 – 这些需求必须都在队列的资源限制范围之内

   如果有多个队列能满足条件，那么排列在最前面的队列将被选择。这个排列是根据 DEFAULT_QUEUE 参数或 LSB_DEFAULTQUEUE 环境变量而来的。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.1.8 作业的调度与派发

提交的作业在队列中等待调度与派发。当作业提交到 volclava 系统中后，有许多因素会影响到或控制着作业什么时候与派发到哪里运行。

- 队列或节点的活动时间窗口
- 作业所需要的资源条件
- 候选节点的可用性
- 多种的作业槽限制
- 作业依赖条件
- 公平策略限制
- 负载条件

1. **先到先服务的策路（FCFS）**： 默认情况下，队列中作业是按先到先服务的顺序来进行派发的。这就意味着作业是按它们在队列中的位置进行派发的。因为作业在队列中的排序是按着其优先级而定的，这个顺序并不代表它们的提交顺序。这个顺序可由用户与管理员进行调整。

2. **调度与派发：**作业的调度决策有一个固定的时间间隔，默认的是 5 秒钟。一旦作业的调度决定完成，它就可以被立即派发。为了不使任何节点被淹没，volclava 会等待一定的时间间隔再把作业派发到同一节点机上。这个时间间隔是在 lsb.params 文件中，通过设置 JOB_ACCEPT_INTERVAL 参数来控制的。这个参数指定要等待多少秒，才再次向同一节点机派发第二个作业，默认设置是 60 秒。如果这个参数设置为 0，那么就会有多个作业同时在一个节点上启动运行的情况出现。

3. **派发顺序**：作业不一定会按其提交的顺序派发。 在一个队列定义时，volclava 管理员会给它们设定一个优先级。volclava 系统会首先从优先级最高的队列开始派发作业。

    - 针对队列，按从高到低的顺序。如果多个队列具有相同的优先级，那么 volclava 会把这些队列内的作业按照先到先服务的策略进行派发
    - 队列中的每个作业是按先到先服务的顺序派发的
    - 如果任何一个节点都适合运行这个作业，那么就把作业派发到最为适合的节点上去。并在 JOB_ACCEPT_INTERVAL 间隔过去前，将这个节点设置为不接受任何作业

作业的派发有时也不能按正常的顺序，比如预处理条件不能满足，指定节点或资源为忙或不可用，或是用户已经达到了其所拥有的作业槽限制。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.1.9 节点机选择

每当 volclava 试图派发一个作业时，volclava 会检查有哪些节点适合运行这个作业。 有许多条件决定一个节点是否适合:

- 节点派发窗口
- 作业的资源需求
- 队列的资源需求
- 队列的节点列表
- 节点的负载水平
- 节点的作业槽限制

如果所有的条件都满足，作业才可能在这个节点上运行。如果作业已经提交到队列中，并且有一个节点适合运行它，那么这个作业就可以派发到此节点上。如果有两个以上的适合节点，那么就以作业和队列的资源需求为标准， 选择一个最为适合的节点运行这个作业。

如果节点的负载指标（比如 r1m，pg，mem）都在设置的调度阀值之内，那么这个节点就是可用的节点。这里有两种调度阀值，节点调度阀值和队列调度阀值。如果节点的任何一个负载指标超过节点或队列的调度阀值，那么这个节点就是不适合运行任何作业。

使用 `bhosts -l` 命令来显示节点的负载阀值。
使用 `bqueues -l` 命令来显示队列的负载阀值

1.1.10 作业的执行环境

volclava 系统会尽力使得运行环境与提交环境尽可能的相同。volclava 会把提交节点的环境拷贝到执行节点上。

执行环境包括:

- 作业所需的环境变量
- 作业开始运行所需的工作目录
- 其他系统所依赖的环境设置，比如资源使用限制或 UMASK

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

PATH 环境变量：寻找可执行文件的 PATH 环境变量，会原封不动地从提交节点传递到执行节点。在混杂的环境下，最好使用户目录（比如， /usr/bin，/usr/local/bin）在不同的节点上具有相同的名称，从而使得 PATH 环境变量在所有的节点上有效。volclava 的配置文件一般是放置在共享目录下的，这可以减轻管理员的工作。这对系统的执行效率影响不大，因为系统不会经常读取这些文件。

1.1.11 容错

volclava 系统是一个连续可用的系统，即便其中的一些节点不可用了，整个系统依然可以连续运行。由系统中选择一个节点作为主节点运行，如果这个主节点宕机了或不可用了，那么就会有另外一个节点起来担当主节点。只要集群中有一个节点存在，volclava 系统会一直保持可用状态。 volclava 能够承受集群中任何节点或节点组的宕机或失效。当一个节点宕机或失效时，运行在其上所有作业会丢失，其他的待发或运行作业则不会受到任何影响。重要作业可以在提交时使用一些选项，从而使其在运行节点宕机或失效时，让 volclava 自动重启这个作业。

1. **动态主节点：**volclava 系统中，主节点是动态选择的。如果当前主节点宕机或失效了，那么 lsf.cluster.cluster_name 文件主机列表中的第一个当前可用的节点会成为当前的主节点。在选取主节点过程中，volclava 系统会有几分钟不可用，此时所有的节点在等待新的主节点联系它们。 在每个服务节点上，运行着的作业是由 sbatchd 进程管理的。当新的 mbatchd 启动后，它会联系所有服务节点的 sbatchd 进程，从而得知所有作业的当前情况。如果 sbatchd 失效了，然而节点机也确实是运行着的，那么运行在节点上作业也不会丢失。当 sbatchd 重启后，它会继续控制管理这些作业。

2. **网络失败：**如果由于网路失效，使得整个集群分割为若干分区的话，每个分区会选出一个自己的主 LIM。同时只要每个节点都能够访问 volclava 的执行文件，那么交互式的负载分担依然有效。

3. **事件日志（lsb.events**）：volclava 容错机制依赖于事件日志，lsb.events。这个文件一般是保存在主文件服务器上的。系统中所发生的任何事件都登记在其中，包括每个作业的提交，每个作业或节点的状态变更等。如果集群的主节点宕机，LIM 就会选取新的主节点。新主节点上 sbatchd 进程能够启动 mbatchd 进程，这个新的 mbatchd 进程会读取 lsb.events 文件，从而能够恢复系统的状态。 如果有的用户不想只依靠文件服务器来保存恢复信息的话，那么 volclava 还提供了一种日志备份机制。在文件服务器上保存的是备份日志，当前使用的日志是保存在主节点上的。如果当前所用的日志文件不可用了，那么就会使用这个备份的日志。当先前的主节点再次起来是会通过这个备用的日志，同步当前的系统态。

4. **网络分隔：**如果网络被分隔成不同区域，并且只有一个分区能访问到 lsb.events 文件， 那么批处理系统只在这个分区上有效。volclava 使用一个锁文件，以保证在整

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

个集群内只有一个 mbatchd 在运行。

5. **节点失败**：如果一个 volclava 服务节点失效，那么运行在其上的作业就会丢失。其他的作业不会受到影响。作业可以被设置为 rerunnable（可重新运行的）和 checkpointable（可作检查点的）。这样一旦由于运行节点失效造成的作业丢失，volclava 系统会自动重启作业，重新从头运行或从上一个检查点开始运行。

1.2 管理集群

1.2.1 检查集群的信息

volclava 为用户提供了检查集群信息的命令，以检查诸如主节点名，集群名称，集群资源的定义，以及集群管理员等有关集群的信息。

要检查的信息请运行

```
volclava 的版本  - - -    lsid
集群的名称        - - -    lsid
当前的主节点      - - -    lsid
所配置的参数      - - -    bparams
```

检查 volclava 版本，集群名称，以及当前的主节点：

```
[root@master-test ~]# lsid
volclava project 2.2.1, Oct 30 2026

My cluster name is volclava
My master name is master-test
```

运行 **badmin showconf mbd/sbd**，**lsadmin showconf lim** 来显示 volclava 配置文件 lsf.conf 中和 mbatchd、sbatchd、lim 相关的配置信息：

```
 [root@master-test ~]$ badmin showconf mbd
MBD configuration at Fri Oct 30 10:38:13 2026
        LSB_CONFDIR = /software/volclava-2.2/etc/
        LSB_MAILPROG = /usr/lib/sendmail
        LSB_MAILTO = !U
        LSB_MBD_PORT = 6324
        LSB_MIG2PEND = 0
        ......

[root@master-test ~]$ badmin showconf sbd
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
SBD configuration for host <master-test> at Fri Oct 30 10:38:13
2026
        LSB_CONFDIR = /software/volclava-2.2/etc/
        LSB_MAILPROG = /usr/lib/sendmail
        LSB_MAILTO = !U
        LSB_MBD_PORT = 6324
        LSB_MEMLIMIT_ENFORCE = N
        ......

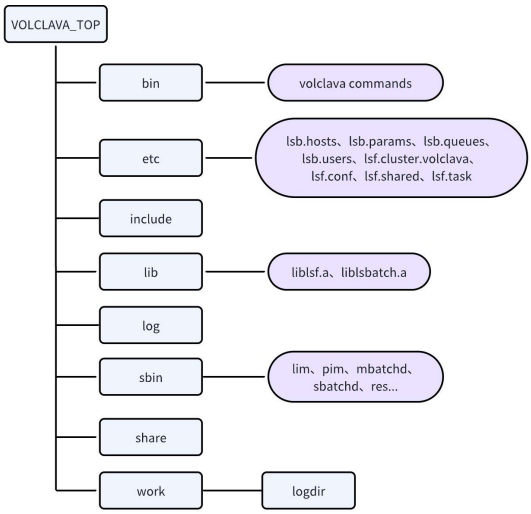
[root@master-test ~]$ lsadmin showconf lim
LIM configuration for host <master-test> at Fri Oct 30 10:38:07
2026
        LSB_SHAREDIR = /software/volclava-2.2/work
        LSF_BINDIR = /software/volclava-2.2/etc/exec/rocky-8-
x86_64/bin
        LSF_CONF_RETRY_INT = 30
        LSF_CONF_RETRY_MAX = 0
        LSF_CONFDIR = /software/volclava-2.2/etc
        ......
```

运行 **bparams** 来显示 volclava 配置文件 lsb.params 中设置的一般信息，其中包括默认的队列、作业派发间隔、作业检查间隔、作业接收间隔等。bparams -l 能够把相应的参数以长格式列出，并能够列出每个相应参数的简介。

```
[root@master-test ~]# bparams
Default Queues: normal
Job Dispatch Interval:  10 seconds
Job Checking Interval:  7 seconds
Job Accepting Interval: 0 seconds
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.2.2 目录结构的示例



1.2.3 集群的管理员

1. **首席集群管理员**：首席管理员是必须存在的。它是第一个集群管理员，是在安装期间定义的。首席 volclava 管理员拥有配置文件与日志文件。首席 volclava 管理员有权执行针对整个集群有效的操作，比如更改配置文件，重新设置集群，以及控制所有用户提交的作业。

2. **增加集群管理员**：

```
在 lsf.cluster.cluster_name 文件中的 clusterAdmins 段内，在
ADMINISTRATORS 后面指定集群管理员
Begin ClusterAdmins
Administrators = volclava
End ClusterAdmins
保存你的更改
运行 lsfadmin reconfig 来重新设置 LIM
运行 badmin mbdrestart 来重新启动 mbatchd
```

1.2.4 控制 volclava 的后台进程

1. **控制后台进程的命令**：下面是一个简要的命令列表，你可以用它们来控制 volclava 的后台进程。执行命令的身份需是 root 用户或是 volclava 管理员。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

后台进程	操作	命令
所有	启动	lsfstartup
	关闭	lsfshutdown
sbatchd	启动	badmin hstartup [host_name ...|all]
	重启	badmin hrestart [host_name ...|all]
	关闭	badmin hshutdown [host_name ...|all]
mbatchd	重启	badmin mbdrestart
	关闭	badmin hshutdown
	重新设置	badmin reconfig
RES	启动	lsadmin resstartup [host_name ...|all]
	关闭	lsadmin resshutdown [host_name ...|all]
	重启	lsadmin resrestart [host_name ...|all]
LIM	启动	lsadmin limstartup [host_name ...|all]
	关闭	lsadmin limshutdown [host_name ...|all]
	重启	lsadmin limrestart [host_name ...|all]
	重新设置	lsadmin reconfig

2. **sbatchd**:重启一个节点上的 sbatchd 进程并不会影响运行在其上的作业。如果 sbatchd 关闭了，这个节点只是不能运行新的作业了。正运行在其上的作业不会停下来，但是其运行的结果只能在 sbatchd 重新启动后，才能被发送给用户。

3. **LIM 和 RES**:重启一个节点上的 LIM 和 RES 进程并不会影响运行在其上的作业。如果某个后台进程不对其相应的网络连接请求作回应的话，那么 lsadmin 会显示相应节点的名称与错误。这样情况下，你就必须手动杀掉并重启相应节点上的这个进程。如果当前主节点上的 LIM 或其他后台进程关闭了，那么就会有其他的节点起来担当主

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

节点的角色。 如果一个节点的 RES 进程关闭了，而此时一个远程的交互式作业还运行在其上，那么这个交互作业会继续，但这个节点不会再接受新的任务。

1.2.5 控制 mbatchd 进程

1. **重新设置 mbatchd**:如果你往某个节点组中增加一个节点，往一个队列里增加一个节点，或是把 lsf.cluster.clustername 文件中的 Hosts 段的设置改变了，那么你就必须重新设置 mbatchd，否则所提交的作业不能利用这个节点。

```
Shell
运行 badmin reconfig
当你重新配置集群时，mbatchd 进程并不会被重启动，只是配置文件会被刷新
```

2. **重启 mbatchd**

```
Shell
运行 badmin mbdrestart
volclava 首先会检查配置文件，如果其中有错误，则将会被打印到标准错误输出 （默认为终端屏幕）。如果没有错误，则下列事件将会发生：
-- 配置文件被重新载入
-- mbatchd 被重新启动
-- lsb.events 文件的事件记录会被读取并载入，这可以恢复上个 mbatchd 的运行状态
注意：当 mbatch 重启期间，它是不能接收服务请求的。在一个大的集群中 lsb.events 中会有很多事件日志，因此重启 mbatchd 会耗费相当多的时间。 为了避免重载所有的事件记录，请使用 badmin reconfig 命令。
```

3. **当重启 mbatchd 时登记注释信息**

```
Shell
使用 badmin mbdrestart 命令的-C 选项，向 lsb.events 文件中登记一条相关的管理员注释信息
比如： badmin mbdrestart -C “Configuration Change”
这个“Configuration Change”注释信息会被记录到 lsb.events 文件中
运行 badmin  mbdhist 命令，可以显示相关 mbatchd 重启的管理员注释信息。
```

4. **关闭 mbatchd**

```
Shell
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

运行 badmin hshutdown 命令，来关闭主节点上的 sbatchd 进程。
比如： **badmin hshutdown hostD**
Shut down sLave batchd daemon on ..... done

运行 badmin mbdresart 命令：
badmin mbdrestart
Checking configuration fiLes ... No errors found.
这个命令会使 mbatchd 和 mbschd 进程退出，并且 mbatchd 不能够重新启动， 因为本节点上 sbatchd 已经关闭了。所有的 volclava 服务都暂时被在中止了，但是正在退出的作业不会受到影响。当最终 sbatchd 重启了 mbatchd，所有先前存储在 lsb.events 中的事件记录被重载入，系统调度就会相应的得到恢复。

5. **query mbatchd 子进程**

C
代码块
query mbatchd 是由主 mbatch fork 的专门处理 query 请求的子进程，需要在 lsf.conf 中正确配置 **LSB_QMBD_PORT** 之后生效。
query mbatchd 默认依赖主 mbatchd 定期重建来同步新的作业信息，达到存活时间或者检测到新作业提交并且超过最小存活时间之后，主 mbatchd 会通知旧 query mbatchd 退出，并启动新的 query mbatchd。
可以通过配置 **LSB_QMBD_JOB_SYNC_MODE** = socket / shm 通过 ipc 的方式实时同步新作业到 query mbatchd。

query mbatchd 相关配置项如下:

配置项	说明
LSB_QMBD_PORT	query mbatchd 提供查询服务的端口，配置之后，所有的查询请求都会向 query port 发起连接。由主 mbatchd 绑定并监听，query mbatchd 复用该监听 socket，默认不配置。
LSB_QMBD_ALIVE_TIME	query mbatchd 默认存活时间。超过该时间后或者在最小窗口期间（5s）之后有新作业提交，主 mbatchd 通知过期的 query mbatchd 退出并且 fork 新的 query mbatchd。
LSB_QMBD_THREAD_NU	query mbatchd 查询处理线程数。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

M	
LSB_QMBD_MAX_TASK_NUM	query mbatchd 查询任务队列容量，任务队列满之后查询请求会 fallback 到主 mbacthd。
LSB_QMBD_JOB_SYNC_MODE	query mbatchd 增量作业同步模式，在配置了合法的 LSB_QMBD_PORT 后生效。
LSB_QMBD_SYNC_SHM_SIZE	shm 模式使用的共享内存大小，仅在 LSB_QMBD_JOB_SYNC_MODE=shm 时使用。

LSB_QMBD_JOB_SYNC_MODE 支持以下取值:

取值	行为
off	关闭增量同步。query mbatchd 只依赖启动时重建的作业快照；新作业需要等待下一轮 query mbatchd 重建后可见。
socket	主 mbatchd 通过内部 socketpair 将新提交作业增量发送给 query mbatchd，默认值是 socket。
shm	主 mbatchd 通过共享内存同步新提交作业。

1.2.6 重新设置你的集群

当你修改了 volclava 的配置文件内容，你必须告诉 volclava 重新读取这些文件并更新相应的配置。请使用下面的命令来重新配置集群:

```Shell
lsadmin reconfig
badmin mbdrestart 或者 badmin reconfig
```

具体使用哪个命令要依据你对哪个文件进行了修改，你可以使用下表来快速查找。

如果你修改了...	请使用...	相应的作用
lsb.hosts	badmin reconfig	重新载入配置文件;

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

lsb.params	badmin reconfig	重新载入配置文件;
lsb.queues	badmin reconfig	重新载入配置文件;
lsb.users	badmin reconfig	重新载入配置文件;
lsf.cluster.cluster_name	lsadmin reconfig ; badmin mbdrestart	重启 LIM， 重新载入配置文件，并重启 mbatchd
lsf.conf	lsadmin reconfig; badmin mbdrestart	重启 LIM， 重新载入配置文件，并重启 mbatchd
lsf.shared	lsadmin reconfig ; badmin mbdrestart	重启 LIM， 重新载入配置文件，并重启 mbatchd
lsf.task	lsadmin reconfig ; badmin reconfig	重启 LIM， 重新载入配置文件

1. **使用 lsadmin 和 badmin 命令，重新设定集群：**要使得配置修改生效，请使用如下这个方法来重设集群。

```
Shell
使用 root 用户身份或是 volclava 管理员的身份，登录到节点
运行 lsadmin reconfig 来重设 LIM
     lsadmin reconfig
     这个 lsadmin reconfig 命令会检查配置错误。如果配置没有错误，那么 volclava 会提示你选择，只重启候选主节点还是重启所有的节点上的 LIM。如果配置中有重大的错误，那么重设过程会被中止。
运行 badmin reconfig 来重设 mbatchd
     badmin reconfig
     这个 badmin reconfig 命令会检查配置错误。如果配置中有重大的错误，那么重设过程会被中止
```

2. **通过重新启动 mbatchd 来重设整个集群** :使用这个方法来重设集群，能够重载并恢复集群的运行状态。

```
Shell
运行 badmin mbdrestart 来重启 mbatchd
     badmin mbdrestart
这个 badmin reconfig 命令会检查配置错误。如果配置没有重大错误，那么
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

volclava 会提示你确认重启 mbatchd，如果配置中有重大的错误，那么这个命令就会被中止，不做任何动作
注意：如果 lsb.events 文件很大，或当前有很多作业在运行，那么重启 mbatchd 则会使用相当多的时间，而此时 mbatchd 是不接收服务请求的

3. **检查配置错误**:

```
Shell
运行 lsadmin ckconfig  -v
运行 badmin ckconfig -v
这些命令会报告所有的错误到你的终端上
```

1.3. 管理节点

1.3.1 节点状态

节点状态表示了一系列有关节点接收与运行作业能力的信息，其中包括后台进程的状态，负载水平，以及所设置的管理控制指标。你可以使用 **bhosts** 与 **lshosts** 命令来显示节点状态。

状态	描述
OK	节点能够接收与运行新的批处理作业。
Unavail	节点处于岩机状态，或是找不到 LIM 和 sbatchd 进程。
Unreach	LIM 进程运行着，但是联系不到 sbatchd 进程。
Closed	节点不接收新的作业，使用 bhosts -l 命令来检查其原因。

1. **bhosts -l** 命令能够显示节点关闭的原因。处于关闭状态的节点不能接收新的作业。

关闭的原因	描述
Closed_Adm	这是由 volclava 管理员或是 root 用户，通过运行 badmin hclose 命令来指定的。运行中的作业不会受到影响

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

Closed_busy	某个运行中负载指标的值超过了所设定的阀值，这个阀值是在 lsb.hosts 中设定的，你可使用 bhosts -l 来显示它。运行中的作业不会受到影响。这个超出设定的负载指标由星号（*） 标示。
Closed_ExcL	有一个排他型的作业运行在这个节点上，比如由 bsub –x 提交的作业。
Closed_FuLL	节点上运行的作业数量已达到所设置的最大值，运行中的作业不会受到影响。
Closed_LIM	sbatchd 进程运行着，但是联系不到 LIM 进程。
Closed_Lock	这是由 volclava 管理员或是 root 用户，通过运行 lsadmin limlock 命令来指定的。运行中的作业会被挂起（SSUSP），使用 lsadmin limunlock 命令来解开当前节点 LIM 的锁定。
Closed_Wind	节点是被派发窗口指定为关闭的，这个派发窗口是在 lsb.hosts 文件中定义的。运行中的作业不会受到影响

2. **lsload** 显示节点当前的状态

状态	描述
OK	节点可用，能够接收与运行批处理作业或远程任务
-OK	LIM 进程运行着，但是 RES 进程联系不到。
busy	不影响批处理作业，只适用于远程作业的调度，例如用 lsrun 命令提交的作业。busy 表示负载指标已经超出所设的阀值，其阀值是在 lsf.cluster.cluster_name 文件中设置的，可通过 lshosts -l 命令来显示。这个超出设定的负载指标由星号（*） 标示
LockW	不影响批处理作业，只适用于远程作业的安排，例如用 lsrun 命令提交的作业。节点是由运行窗口锁定的，运行窗口是在在 lsf.cluster.cluster_name 文件中设置的，可通过 lshosts -l 命令来显示
LockU	既不会接收新的批处理作业也不接收远程作业。这是由 volclava 管理员或是 root 用户，通过运行 lsadmin LimLock 命令来指定的， 或有一个排

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

	他型的作业运行在这个节点上，比如由 bsub –x 提交的作业。使用 lsadmin limunlock 命令来解开当前节点 LIM 的锁定。
Unavail	节点宕机，或是 LIM 进程联系不到

1.3.2 检查节点信息

volclava 使用集群中某些或全部节点作为执行节点。节点列表是由 volclava 管理员来设定的。使用 bhosts 命令来检查节点信息，使用 lsload 命令来检查节点的负载信息。

要检查...	**请运行...**
集群中所有节点以及他们的状态	bhosts
详尽的服务节点信息	bhosts -l 和 lshosts –l
节点的负载信息	lsloads
节点的结构信息	lshosts
节点的历史信息	badmin hhist
节点的类别与型号信息	lsinfo
动态节点的信息	lshosts

1.3.3 控制节点

1. **关闭一个节点**

```
运行 badmin hclose:
badmin hclose hostB
Close  ...... done
如果命令失效，则可能是由于网络连接的问题引起的，也可能是节点上的守护
```

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
进程失效了。
```

2. **开启一个节点**

```
运行 badmin hopen:
badmin hopen hostB
Open  ...... done
```

3. **设置派发窗口：**派发窗口指定一个或多个时间段，在这些时间段内节点是接受作业的，而在其他时间是不接受作业的。派发窗口不影响你提交作业或已经在运行中的作业，这些作业会一直被允许运行到结束。默认情况下，volclava 并没有设置派发窗口。如要设置它，请按下面的步骤。

```
编辑 lsb.hosts
在 DISPATCH_WINDOW 域内，设置一个或多个时间窗口：
Begin  Host
HOST_NAME  r1m  pg  Ls  tmp  DISPATCH_WINDOW
...
hostB         3.5/4.5  15/  12/15   0   (4:30-12:00)
...
End Host
重新设置集群
  - - 使用 lsadmin reconfig 命令来重设 LIM
  - - 使用 badmin reconfig 命令来重设 mbatchd
使用 bhosts -l 来显示所设置的派发窗口值
```

4. **在开启与关闭节点时，登记一条注释信息：**使用 badmin hclose 与 badmin hopen 命令的-C 选项，来向 lsb.events 中添加一条注释信息：

```
badmin hclose –C “WeekLy backup” hostB
```

相关的注释信息“WeekLy backup”将会被登记到 lsb.events 文件中。

使用 **badmin hist** 与 **badmin hhist** 命令来显示在开启或关闭节点时所登记的信息：

```
badmin hhist  Fri Apr 4 10:35:31: Host  Closed by
administrator  Weekly backup.
```

bhosts -l 命令也能够显示所登记的信息。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.3.4 增加一个节点

在 lsf.cluster.cluster_name 文件中，增加节点的信息。

1. 以 root 用户身份或者 volclava 管理员身份，登录到集群的主节点上。
2. 编辑 VOLCLAVA_TOP/etc/lsf.cluster.cluster_name 文件，在 Host 段指定下列信息：

   -- 节点的名称

   -- 相应的类型或使用！ 指定自动检测类型

   -- 在 Server 列，用 1 表示为服务节点，用 0 表示为客户节点

```
Begin Host
HOSTNAME        model        type    server    rlm
RESOURCES
ic-master      IntelI5       linux     1       3.5    ()
ic-cmp02       IntelI5       linux     1       3.5    (cs)
ic-cmp03       IntelI5       linux     1       3.5    (cs)
ic-vnc00       IntelI5       linux     0       3.5    ()
End Host
```

3. 保存修改 。
4. 使用 **lsadmin reconfig** 命令来重新设置 LIM 。
5. 使用 **badmin mbdrestart** 命令来重新启动 mbatchd 。
6. 在新节点上启动 volclava

```
lsadmin limstartup
lsadmin resstartup
badmin hstartup
```

7. 使用 lshosts 与 bhosts 命令来检查你的新配置。

1.3.5 移除一个节点

从 volclava 中移除一个节点，就是要阻止其他作业再在这个节点上运行，从而使节点从集群中分离出来。

注意：请不要把 volclava 的主节点移除。如果你要把当前默认的主节点从 volclava 集群中移除，请先修改 lsf.cluster.cluster_name 文件，把默认主节点指向其

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

他节点。 然后在移出这个节点。

1. 以 root 身份登录到这个 volclava 节点。
2. 使用 badmin hclose 命令关闭此节点。这可以停止向此节点再派发作业， 并且允许现有的作业运行完毕。
3. 手动停止所有的 volclava 后台进程。
4. 在 VOLCLAVA_TOP/etc/lsf.cluster.cluster_name 文件中，Host 部分移出此节点所有相关的设置。
5. 在下列文件中，移出所有与该节点相关的设置。
    - VOLCLAVA_TOP/etc/lsf.shared
    - VOLCLAVA_TOP/etc/lsb.hosts
    - VOLCLAVA_TOP/etc/lsb.queues
6. 从该节点退出，并使用 root 与 volclava 管理员身份登录到其它节点上。
7. 运行 lsadmin reconfig 命令来重设 LIM。
8. 运行 badmin reconfig 命令来重设 mbatchd。
9. 如果你设置了在系统启动时 volclava 服务进程自动加载，请从启动文件中移出 volclava 的相关设置。
10. 如果节点上有用户使用 lstcsh 作为其登录 shell，请更改他们的登录 shell 为 tcsh 或 csh，并从/etc/shells 文件中移除 lstcsh。

1.3.6 向 lsf.shared 中添加节点类型与型号

lsf.shared 文件中有节点类别与节点型号的列表，这其中涵盖了大多数的操作系统。你可以根据自己的需要来修改这个列表，增加或者修改节点的类型与节点的型号。节点的类型或型号可以是最多长达 39 个字符的数字与字符的组合串。

1. **增加一个节点类别或型号**
    a. 以 volclava 管理员的身份登录到集群中任何一个节点上
    b. 修改 lsf.shared 文件：

若是一个新的节点类别，则修改 HostType 段的设置：

```
Begin HostType
TYPENAME              #Keyword
linux
```

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
End HostType
```

若是一个新的节点型号，则修改 HostModel 段的设置：

```
Begin HostModel
MODELNAME    CPUFACTOR    ARCHITECTURE # keyword
# CPU factors are only comparisions
IntelI5      100          (x86_64)
End HostModel
```

增加一个新型号并相对其他型号来设定其 CPU 的速度因子

c. 保存修改好的 lsf.shared 文件。

d. 运行 lsadmin reconfig 命令来重新配置 LIM。

e. 运行 badmin reconfig 命令来重新配置 mbatchd。

1.3.7 登记服务端口

volclava 使用独占的 UDP 与 TCP 端口来进行通信。集群中所有的节点都要使用相同的通信端口。 服务端口可以是从 1024 到 65535 之间的任何没有被其他服务占用的端口。为了保证你所提供的端口是没有被其他应用服务所登记，请检查/etc/services 文件或使用 ypcat services 命令。 默认的 volclava 服务端口是在 lsf.conf 文件中所设定的。你也可以通过修改/etc/services 文件或 NIS 及 NIS+数据库来配置端口。但是，如果你在 lsf.conf 文件中定义了端口，那么在服务端口数据库中定义的端口将被忽略。

1. 以 root 身份登录任何一个节点
2. 修改 lsf.conf 文件中下列设置：
   - LSF_RES_PORT=3878
   - LSF_MBD_PORT=3881
   - LSF_SBD_PORT=3882
3. 在每个节点上的 lsf.conf 文件中，增加同样的端口设置
4. 保存 lsf.conf 文件
5. 运行 lsadmin reconfig 来重新设置 LIM
6. 运行 badmin mbdrestart 来重新设置 mbatchd
7. 运行 lsfstartup 命令来重新启动集群中的所有的进程

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.3.8 节点命名

volclava 需要把节点名与相应的 IP 地址相匹配。volclava 通过下面的途径来查找节点的 IP 地址;

1. 使用/etc/hosts 文件;
2. 使用 NIS 和 YP，Sun 的网路信息服务/黄页服务
3. 使用 DNS 服务，DNS 就是域名查询服务
4. DNS 也叫作伯克利因特网名称服务（bIND）或 named，named 是 bIND 服务的守护进程。

每个节点都可以设置使用一个或多个上述的机制

1.3.9 节点组

你可以在 volclava 内定一个节点组，或使用外部可执行程序来提取节点组成员。使用 bhosts 命令来检查当前的节点列表，使用 bmgroup 命令来检查当前节点组的成员信息。

1. **哪些地方可以设置或要使用节点组**

   volclava 的节点组可在 volclava 配置文件的下列地方进行配置或使用:

   - lsb.queues 文件的 HOSTS 段，目的是指定当前队列所授权使用的节点
   - lsb.hosts 文件的 HostPartion 段，在这里指定节点组的具体成员

2. **配置节点组**

```
使用 volclava 管理员的身份登录到集群的任何一个节点
打开 lsb.hosts 文件;
如果 HostGroup 段不存在，增加该段
     Begin  HostGroup
     GROUP_NAME        GROUP_MEMBER
      groupA           (all)
      groupb           (groupA ~hostA ~hostB)   #使用剔除符号
（~）来剔除指定的节点或节点组
      groupC           (hostX hostY hostZ)
      groupD           (groupC ~hostX)
      groupE           (all~groupC ~hostB)
      groupF           (hostF groupC hostK)
     desk_tops         (hostD hostE hostF hostG)
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
   End HostGroup
在 GROUP_NAME 列上添加一个节点组名
在 GROUP_MEMBER 列上添加节点成员
保存修改
运行 badmin ckconfig 命令来检查节点组的定义
如果有任何错误存在， 请处理相应的问题，然后再次检测
选择下列操作之一：
    A 如果你不想让在你修改配置前提交的作业知道这个节点组，请使用
badmin reconfig 命令。
    b 如果你想让在你修改配置前提交的作业知道这个节点组，请使用
badmin  mbdrestart 命令。
```

1.4. 管理队列

1.4.1 队列状态

可以通过 bqueues 命令来显示队列的状态，队列的状态描述的是队列接收和启动批处理作业的能力，有如下的组合:

状态	描述
Open：Active	接收并派发作业，正常处理中
Open：Inactive	只接收作业但不派发作业，正在收集作业
Closed：Active	只派发作业但不接收新作业，正在排空队列
Closed：Inactive	既不接收新作业，也不派发作业，所有活动都停止

volclava 管理员或 root 用户能够改变队列的状态。 队列的状态也可以通过设置运行窗口与派发窗口来激活或关闭。这些参数是在 lsb.queues 文件中配置的，可使用 bqueues -l 命令来检查其配置情况。 bqueues -l 的输出中，用 Inact_Adm 表示是被 volclava 管理员通过 badmin qinact 命令来关闭的，而用 Inact_Win 表示是被所设定的运行窗口或派发窗口所关闭的。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.4.2 检查队列信息

bqueues 命令能够显示有关队列的信息，而 bqueues -l 选项则还能够提供当前队列中作业的统计信息，比如这个队列中现在一共有多上个作业，以及运行的和挂起的作业分别为多少等等。

要显示...	请运行...
可用的队列	bqueues
队列的状态	bqueues
队列的详细信息	bqueues -l
队列状态改变的历史信息	badmin qhist
队列的管理员	bqueues -l 队列名

1.4.3 控制队列

volclava 管理员或 root 用户，可以通过命令或设置运行窗口或派发窗口来控制队列。

1. **关闭一个队列**

```
运行 badmin qclose 命令：
badmin qclose normal
Queue <normaL> is Closed
若有用户向已经关闭了队列提交作业的时候，他会收到如下的信息：
bsub -q normal …
Normal: Queue has been Closed
```

2. **开启一个队列**

```
运行 badmin qopen
badmin qopen normal
Queue <normal> is opene
```

3. **阻止一个队列**

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
运行 badmin qinact:
badmin qinact normal
Queue <normal> is inactivated
```

4. **激活一个队列**

```
运行 badmin qact:
badmin qact normal
Queue <normal>  is activated
```

5. **设置派发窗口：**队列的派发窗口指定的是一个或多个时间段，在这些时间段内队列向节点派发并运行作业，而在其他的时间内队列是不向节点派发作业的。派发窗口不影响作业的提交或是已经在运行的作业，已经运行的作业可以一直运行到完毕。默认情况下，队列一直是激活状态，当队列处于阻止（inactive）状态时，你则必须在队列上明确指定派发窗口的时间段。

```
设置派发窗口：
修改 lsb.queues
为相应的队列添加 DISPATCH_WINDOW 关键字，并指定一个或多个时间窗口。
    Begin  Queue
    QUEUE_NAME = queue1
    PRIORITY  = 45
    DISPATCH_WINDOW = 4:30-12:00
    End Queue
重新配置集群：
    A 运行 lsadmin reconfig。
    b 运行 badmin reconfig。
运行 bqueues -l 命令来显示派发窗口
```

6. **配置运行窗口：**运行窗口也指定了一个或多个时间段，在这些时间段内队列的作业是允许在节点上运行的。当一个运行窗口关闭时，运行的作业会被挂起，原先挂起的作业将依旧被挂起。当一个运行窗口开启时，挂起的作业会被恢复。默认状态是激活（active），作业一直都可以运行到结束。而当队列是阻止（inactive） 状态时，你则需要指定运行时间段。

```
修改 lsb.queues。
为相应的队列添加 DISPATCH_WINDOW 关键字，并指定一个或多个时间窗口。
    Begin  Queue
    QUEUE_NAME = queue1
    PRIORITY = 45
    RUN_WINDOW = 4:30-12:00
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
    End Queue
重新配置集群：
    A 运行 lsadmin reconfig。
    b 运行 badmin reconfig。
运行 bqueues -l 命令来显示运行窗口。
```

1.4.4 增加与移除队列

1. **增加一个队列**

```
以 volclava 管理员的身份登录到集群中任何一个节点上。
编辑 lsb.queues 文件来添加新队列的定义。你可以从文件中拷贝一个队列的定义，但是请注意修改队列名（QUEUE_NAME）。
保存对 lsb.queues 文件的修改。
运行 badmin reconfig 来重新配置 mbatchd。增加队列并不能影响挂起或运行着的作业
```

2. **移除一个队列**

注意：在移除一个队列前，请检查确认队列中没有作业

如果队列中有作业存在，请将挂起的作业与运行着的作业转移到另一个队列中，然后再移除该队列。如果你将一个有作业的队列移除了，那么这个作业会暂时被转移到 lost_and_found 队列中。lost_and_found 队列中的作业会一直保持挂起状态，直到有用户或 volclava 管理员使用 bswitch 命令把作业转移到一个存在的队列中。在其他队列中的作业不会受到影响。

```
使用 volclava 管理员身份登录到集群中任何一个节点
首先关闭该队列，以避免有新的作业提交到该队列中来；
     badmin qclose night
     Queue <night>  is Closed
把所有运行着的作业及挂起的作业转移到另一个队列中
     bswitch -q night idLe 0
     Job <5308> is switched to queue <idLe>
     Job <5310> is switched to queue <idLe>
    把 night 队列上的 job 都转移到 idLe 队列中去，并指定作业 ID 为 0，这代表所有的作业
修改 lsb.queues 文件，删除或注释掉这个队列的配置
保存对 lsb.queues 文件的修改
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

使用 **badmin reconfig** 命令来重新配置 mbatchd

1.4.5 管理队列

1. **添加队列管理员：**队列管理员是安装完成后的可选配置。他们的权限限于制定的队列，可以管理队列的状态，如开启、关闭、激活以及阻止，还可以管理运行在队列中的作业。队列管理员不能修改配置文件，也不能对 volclava 进程进行操作，同时也不能管理没有指定的队列。 要将作业从一个队列转移到另一个队列，你必须同时具有两个队列的管理权限。

在 lsb.queues 文件中，在相应队列的 Begin Queue 与 End Queue 之间，添加 ADMINISTRATORS 参数并在其后填入管理员列表，你可以使用用户名或用户组名，用空格分开。

```
Begin  Queue
ADMINISTRATORS = User1 GroupA
End Queue
```

1.5. 管理作业

1.5.1 理解作业的状态

使用 bjobs 命令来显示当前作业的状态

1. **正常的作业状态**

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

大多数的作业会是如下的三种状态：

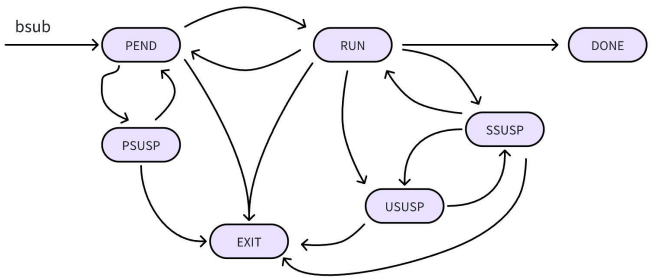
作业状态	描述
PEND	在队中等待派发或调度
RUN	已经被派发并且运行
Done	正常运行完毕，退出值为 0

2. **挂起作业的状态**

作业状态	描述
PSUSP	作业是被用户自己或 volclava 管理员在 PEND 状态时指定挂起的
USUSP	作业是被用户自己或 volclava 管理员在派发后指定挂起的
SSUSP	作业是被 volclava 系统在派发后指定挂起的

3. **状态变化**:

一个作业会经过一系列的状态变换，最终或是被成功完成，或是失败，或是被终止；下图中标识出了作业在其生命周期内所有可能的状态。



4. **挂起的作业**

作业会一直处于挂起状态，直到所有的执行条件满足。下列就是其中一些条件:

- 用户在提交作业时所指定的起始时间

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

- 能够运行该作业的节点的负载情况
- 派发窗口，队列可以在这个指定的时间内向适合的节点派发作业
- 运行窗口，队列可以在这个指定的时间内运行起作业
- 在队列、节点或用户上所定义的作业槽使用数量的限制
- 相对于其他用户与队列的优先级
- 运行所需要资源的可用情况
- 作业的依赖关系以及作业的前处理条件

5. **最大的挂起作业阀值**

如果用户或用户组向 volclava 提交的作业挂起作业阀值，volclava 会拒绝这个用户或用户组的再次提交作业的请求。挂起作业的阀值是在 lsb.users 文件中的 user 段定义的，或是在 lsb.params 文件中定义的。

系统会自动每隔 SUB_TRY_INTERVAL 时间间隔向 volclava 提交作业请求，并会持续尝试 LSB_NTRIES 次。 SUB_TRY_INTERVAL 是在 lsb.params 文件中定义的，LSB_NTRIES 则是一个环境变量。如果没有过定义 LSB_NTRIES 且 volclava 拒绝接收作业的提交，那么系统的默认动作是不停的尝试下去。

6. **挂起作业**

作业可能在任何时候被挂起。作业可能是被用户自己，或是 volclava 管理员，或 root 超级用户，以及 volclava 挂起。在一个作业运行时，volclava 会定时检查运行节点的负载情况，如果有的负载值超过了在节点或队列上所定义的阀值，那么该节点上优先级最低的批处理作业将会被挂起。

如果执行节点或执行节点组的负载过高，可能是批处理作业之间互相干扰或是批处理作业与交互式作业互相干扰，那么就需要把一部分作业停下来，从而保证系统的效率，或保证交互式任务的响应速度。

volclava 根据作业所属队列的优先级来决定挂起哪个作业。当一个节点忙时，volclava 首先会挂起优先级最低的作业，除非是与其相关的调度策略不同意。

作业也可能是由于当前不是队列的运行窗口中的运行时间，从而被 volclava 挂起。系统挂起的作业，在负载下降到足够低的时候或是又到了运行窗口时间内，volclava 会自动恢复这些挂起的作业。

7. **退出的作业**:

一个退出作业，在其结束时是以一个非零值为其退出状态的。

作业的异常终止退出可能是由于多种原因造成的，作业终止也可能在任何状态发

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

生。一个异常终止的作业会被标记为 EXIT。下列的条件将有可能导致作业的异常退出：

- 作业在队列中待发时或是已经被派发到节点上后，被用户自己或是 volclava 管理员取消；
- 作业还没有来得及被派发就已经到达了其最后期限，volclava 会自动取消它
- 作业不能成功启动，例如，用户在提交作业时指定了错误的执行代码；
- 应用程序的退出码为非零值

1.5.2 检查作业的信息

1. 使用 bjobs 命令来显示作业的信息。默认情况下，bjobs 命令只显示触发命令用户的作业信息。

- 运行 **bjobs –u** 来显示所有用户的作业信息，作业按照如下的顺序显示：

  - - 运行中的作业

  - - 挂起的作业，按照它们调度的顺序排列

  - - 高优先级作业排列在低优先级作业的前边

- 运行 **bjobs –u user_name** 来显示指定用户的作业信息
- 运行 **bjobs –r** 来显示运行着的作业信息
- 运行 **bjobs –d** 来显示近期内完成的作业信息
- 运行 **bjobs –p** 来显示挂起作业的原因
- 运行 **bjobs –w all** 来显示所有用户的最大挂起作业的阀值
- 运行 **bjobs –s** 来显示一个作业挂起的原因
- 运行 **bjobs –d** 来显示最近完成作业的信息
- 运行 **bjobs -l** 来以长格式以多行格式显示每个作业的详细信息
- 运行 **bjobs -UF** 来显示未格式化的作业详细信息
- 运行 **bjobs –o** 来显示指定的字段名字
- 运行 **bjobs -o –json** 结合-o 使用，可以将输出内容转换成 json 格式；不可以脱离-o 单独使用

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
当前 bjobs -o 支持的参数：
"JOBID", "USER", "STAT", "QUEUE", "FROM_HOST", "EXEC_HOST",
"JOB_NAME", "SUBMIT_TIME","PROJ_NAME", "CPU_USED", "MEM",
"SWAP", "PIDS", "START_TIME", "FINISH_TIME"
```

```
[root@master-test etc]# bjobs -o 'jobid stat QUEUE EXEC_HOST
USER SUBMIT_TIME FROM_HOST JOB_NAME PROJ_NAME CPU_USED MEM
SWAP PIDS START_TIME FINISH_TIME' -json
{
    "COMMAND":    "bjobs",
    "JOBS": 1,
    "RECORDS":    [{
                "JOBID":        "16",
                "STAT":  "RUN",
                "QUEUE":        "normal",
                "EXEC HOST":    "cmp2-test",
                "USER":  "root",
                "SUBMIT_TIME":  "Oct 31 10:17",
                "FROM_HOST":    "master-test",
                "JOB NAME":     "sleep 100",
                "PROJ NAME":    "default",
                "CPU_USED":     "000:00:-2:00",
                "MEM":  "0",
                "SWAP": "0",
                "PIDS": "-",
                "START_TIME":   "10/31-10:17:44",
                "FINISH TIME":  "-"
            }]
```

2. 查看作业 slots 使用率

作业提交到队列后，占用的计算资源用 slots 来衡量，但"占用了多少 slots"并不等于"用好了这些 slots"。我们可以开启 IDLE_FACTOR 来根据作业的 CPU 情况，帮助用户直观了解作业是否充分利用了其申请的 slots 资源。首先来理解 IDLE_FACTOR 含义。

- 作业空闲因子（IDLE_FACTOR）

  SLOTS 使用率用作业的**空闲因子（IDLE_FACTOR）**来表示，计算公式如下：

  IDLE_FACTOR = (CPU time / n) / run time

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

其中：

- **CPU time**：作业累计使用的 CPU 时间；
- **n**：作业申请的 slots 个数；
- **run time**：作业的实际运行时间。

简单来说，IDLE_FACTOR 反映的是"作业平均每个 slot 用掉的 CPU 时间占运行时间的比例"，数值含义如下：

- 数值越接近 0：说明作业占着 slots 但很少使用 CPU，作业处于空闲状态；
- 数值越接近 1：说明作业 CPU 利用率高，申请的 slots 得到了充分利用；
- 数值大于 1：说明作业实际消耗的 CPU 超过了申请 slots 所能提供的算力，可能是申请的 slots 偏少。

注意：当作业提交时指定使用 1 个 slot 时，**IDLE_FACTOR** 直接代表了该作业的 cpu 使用率。

- 开启该功能

在 **lsb.queues** 配置文件中，为需要统计的队列添加 **JOB_IDLE** 参数：

```
Begin Queue
  QUEUE_NAME = normal
  JOB_IDLE = number
End Queue
```

其中 number 为 0.0 到 1.0 之间的数字，作为判定作业是否空闲的阈值。配置完成后，volclava 即开始计算该队列上所有作业的 IDLE_FACTOR；未配置 JOB_IDLE 的队列不进行计算。

注意：仅配置 JOB_IDLE 时，只开启作业 IDLE_FACTOR 的统计与展示，不会触发任何额外动作。

- 查看 CPU 使用率
  - 运行 **bjobs -l** 查看作业详细信息，资源使用率输出中会显示 **IDLE FACTOR (cputime/runtime)** 字段，即作业当前的 CPU/Slots 使用率（空闲因子）；

```
[root@v-y-volclava-dev1 etc]# bjobs -l 42
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
Job <42>, User <root>, Project <default>, Status <RUN>,
Queue <normal>, Command <job.sh>, Share group charged
</normal/root>
Thu Oct 22 17:15:41: Submitted from host <v-y-volclava-
dev1>, CWD </opt/volclava_test/etc>;
Thu Oct 22 17:15:52: Started on <v-y-volclava-dev1>,
Execution Home </root>, Execution CWD
</opt/volclava_test/etc>;
Thu Oct 22 17:16:02: Resource usage collected.
                     IDLE_FACTOR(cputime/runtime):   0.1
                     MEM: 6 Mbytes;  SWAP: 54 Mbytes
                     PGID: 874123;  PIDs: 874123

 MEMORY USAGE:
 MAX MEM: 6 Mbytes;  AVG MEM: 6 Mbytes

 SCHEDULING PARAMETERS:
           r15s   r1m  r15m   ut      pg    io   ls    it
tmp    swp    mem
 loadSched   -     -     -     -       -     -    -     -
-      -      -
 loadStop    -     -     -     -       -     -    -     -
-      -      -

 RESOURCE REQUIREMENT DETAILS:
 Combined: select[type == local] order[r15s:pg]
 Effective: select[type == local] order[r15s:pg]
```

- 运行 **bqueues -l** 查看队列详细信息，在 **JOB EXCEPTION PARAMETERS** 区域会显示队列配置的 IDLE 阈值。

```
[root@v-y-volclava-dev1 etc]# bqueues -l normal

QUEUE: normal
  -- For normal low priority jobs, running only if hosts
are lightly loaded.  This is the default queue.

PARAMETERS/STATISTICS
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND
RUN SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     0     0
0     0     0    0
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
Interval for a host to accept two jobs is 0 seconds

SCHEDULING PARAMETERS
           r15s   r1m  r15m   ut      pg    io   ls    it
tmp    swp    mem
 loadSched   -     -     -     -       -     -    -     -
-      -      -
 loadStop    -     -     -     -       -     -    -     -
-      -      -

JOB EXCEPTION PARAMETERS
              IDLE(cputime/runtime)
  Threshold   0.30
       Jobs      0

SCHEDULING POLICIES:  FAIRSAHRE
USER_SHARES:  [default,1]

SHARE_INFO_FOR: normal/
USER/GROUP      SHARES    PRIORITY   STARTED    CPU_TIME
RUN_TIME
 root                1       0.333         0         0.0
0

USERS:  all
HOSTS:  all
```

- 空闲作业异常检测（可选）

  当运行中作业的 IDLE_FACTOR 低于队列配置的 JOB_IDLE 阈值时，该作业会被判定为空闲作业。**lsb.params** 文件中以下参数可以赋能 volclava 做更多的事情：

  - DETECT_IDLE_JOB_AFTER = num：开启统计队列上的空闲作业数量。
  - EADMIN_TRIGGER_DURATION/EADMIN_TRIGGER_INTERVAL = num：在出现空闲作业时，开启自动触发通知或告警。

以上参数的详细使用说明请参见 lsb.queues 和 lsb.params 用户手册（例如：在集群环境中 man lsb.queues）。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.5.3 检查历史作业信息

bhist 用于查询作业历史信息的命令，与依赖 mbatchd 内存的 bjobs 不同，它直接读取 lsb.events 事件日志文件，因此具备两项核心能力：一是按时间线还原单个作业从提交、排队、分派、运行、挂起、迁移到结束的完整调度事件，并统计 PEND、RUN、SUSP 等各状态累计耗时，便于排查作业为何迟迟未被调度或中途异常；二是可以翻查那些早已执行完毕、已从 mbatchd 内存中清理掉的历史作业，即使 bjobs -a 也不再显示，仍能通过 lsb.events 及其滚动归档文件回溯作业的执行节点、起止时间、退出码和资源消耗。bhist 是集群中做任务复盘、故障追溯和用量核对的基础工具。以下是常用操作：

- 运行 **bhist -u user_name** 查看指定用户的历史作业
- 运行 **bhist -q queue_name** 查看指定的历史作业
- 运行 **bhist -m host_name** 查看指定的历史作业
- 运行 **bhist -S / -D / -C time0,time1** 按提交、派发、完成时间筛选历史作业
- 运行 **bhist -l job_ID** 以长格式查看指定作业的完整过程
- 运行 **bhist -a / -d / -e / -p / -r / -s** 按作业状态筛选历史作业
- 运行 **bhist -n min_logfile, max_logfile** 指定读取事件文件的范围

bhist 命令通过顺序扫描主调度节点上的 lsb.events 事件日志来获取作业历史，在作业量大、查询时间跨度长或需要追溯归档日志时，存在以下不足：

- 事件文件多、文件大时，**响应时间会明显变慢，大范围查询可能内存不足**。
- 默认只读当前的 lsb.events，**查更早的作业要加 -n**；

为解决这一问题，volclava 提供 bhist 加速服务模块：由后台 loader 进程持续监听 lsb.events 及其滚动归档文件，并将其中的作业事件增量导入本地数据库；用户执行 bhist 查询时，先由本模块根据查询条件（用户、队列、执行节点、作业 ID、提交 / 派发 / 结束时间区间等）在数据库中快速过滤出相关作业的事件记录，再交由标准 bhist 完成后续的格式化输出。对用户而言，命令行用法与原生 bhist 完全一致，无需额外学习成本；**在性能上，常用查询的响应速度可得到显著提升，同时也支持在不明显增加查询时延的前提下，回溯更大时间范围内的作业历史**。

该模块可在部署 volclava 时随主程序一并安装，也支持以热更新方式在已部署环境中单独上线。除本产品自有集群外，标准 LSF 集群亦可接入本加速服务。 下面介绍 bhist 加速服务模块的部署和启动。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

- **安装方式**

方式一：随 volclava 全量安装

1. 在 install.conf 中启用：

```
VOLC_BHIST_SPEEDUP=Y
```

2. 然后按原有流程安装：

```
./volcinstall.sh --file=./install.conf
```

方式二：在已有 volclava/LSF 集群热更新

1. 用 volcinstall.sh 构建并安装到临时目录

```
cd /path/to/volclava-source
cp install.conf.example install.conf
# 编辑 install.conf：
#   VOLC_PREFIX=/tmp/bhist-speedup-stage   # 临时目录，不要指向现有的安装目录
#   VOLC_ADMIN=volclava                    # 与集群管理员一致
#   VOLC_BHIST_SPEEDUP=Y
./volcinstall.sh --setup=pre --file=install.conf
./volcinstall.sh --setup=install --file=install.conf

STAGE_DIR=/tmp/bhist-speedup-stage          # 与 VOLC_PREFIX 相同
```

--setup=pre 会创建管理员用户、安装编译依赖，在 Rocky/CentOS 上还会停止并禁用 firewalld；依赖已经装好时可以跳过这一步。--setup=install 只安装到 VOLC_PREFIX，不会影响集群当前的状态。

2. 停止现有服务（首次部署跳过）。只有服务已经在运行时才需要停止；首次部署还没有启动过服务，直接进行下一步。

```
# 由 systemd 管理时
systemctl stop volclava-bhist-server volclava-bhist-loader
# 手动启动时
bhist-admin server stop
bhist-admin loader stop
```

3. 替换共享安装目录中的文件。LSF_BINDIR、LSF_SERVERDIR 分别指向当前平台的共享命令目录和服务程序目录，LSF_ENVDIR 指向共享配置目录。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
for name in bhist bhist-fast bhist-standard bhist-admin bhist-
speedup-setup; do
    install -m 755 "$STAGE_DIR/bin/$name" "$LSF_BINDIR/.$name.new"
    mv -f "$LSF_BINDIR/.$name.new" "$LSF_BINDIR/$name"
done
for name in bhist-speedup-loader bhist-speedup-server; do
    install -m 755 "$STAGE_DIR/sbin/$name"
"$LSF_SERVERDIR/.$name.new"
    mv -f "$LSF_SERVERDIR/.$name.new" "$LSF_SERVERDIR/$name"
done

# lsb.hist 只在首次部署时安装，已有的配置保留不动
if [ ! -e "$LSF_ENVDIR/lsb.hist" ]; then
    install -m 644 "$STAGE_DIR/etc/lsb.hist"
"$LSF_ENVDIR/lsb.hist"
fi
```

注意：热部署 LSF 集群时， 必须把 LSF 自带的 bhist 改名为 bhist-standard。热更新时不要用 Volclava 构建出的 bhist-standard 覆盖它，即需把复制列表中去掉 bhist-standard。

- **安装或者热更新完成后，选择部署 bhist 加速服务的节点，该主机需要满足如下几个条件**:
  - 身份为集群内节点，且需要配置该主机不执行作业。
  - 有本地存储，数据库路径配置到本地存储，空间至少为管理员配置的时间窗口内，logdir 下所有 lsb.events.* 文件大小之和的两倍（推荐 100GB 以上）；
  - 网络通畅，配置尽量高些，建议 8core 32G。

- **开启 bhist 加速服务的配置**。编辑 $LSF_ENVDIR/lsb.hist 文件，做如下配置:

```
LSB_BHIST_ENDPOINT=host:port #修改 host:port 为你选择的服务节点和自定义端口号
LSB_BHIST_DB_PATH=/var/lib/bhist-speedup/bhist-speedup.sqlite #可用这个值，也可修改成其他路径
```

- **root 登录服务节点，启动服务**:

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

方式一：使用 systemd 管理服务（推荐）

首次运行时，需要执行**$LSF_BINDIR/bhist-speed-setup**，生成 volclava-bhist-loader.service，volclava-bhist-server.service 文件，推荐使用 systemd 管理服务。

```
bhist-speedup-setup    # 首次部署时执行一次，生成 systemd 服务
systemctl start volclava-bhist-loader volclava-bhist-server
```

方式二：手动管理

```
bhist-admin loader start
bhist-admin server start
```

- **验证**

```
在 bhist 加速服务节点运行：
bhist-admin loader status
bhist-admin server status

在集群里任意节点运行：
bhist -l 12345
```

- **管理员注意事项**
  - 如果发现 bhist-admin loader status 返回的内容里 db_status=rebuild_required，请清理数据库并且重新启动 loader 进程；
  - 在手动清理/修改 logdir 下的 lsb.events.* 之后，应该清理数据库并且重新启动 loader 进程；
  - 修改 lsb.hist 配置后，及时重启服务。

更多日常管理以及详细使用说明请参见 bhist-speedup 、 lsb.hist 、bhist 用户手册（例如：在集群环境中 man bhist-speedup）。

1.5.4 在队列中改变作业顺序

默认情况下，在有适当的节点可用时，volclava 安装作业到达的顺序来派发，也

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

就是先到先服务的策略（FCFS）。 使用 bbot 和 btop 命令可以改变待发作业在队列中顺序，或者是改变待发作业数组的成员，从而达到改变作业派发顺序的目标。每个用户只能在其自有作业内部调整顺序，而 volclava 管理员则可以在任何用户作业的顺序。

1. **bbot <JOB_id>**

   把作业移到你的作业队列的尾部； 如果是普通用户发出的指令，bbot 命令将把指定的作业移到优先级相同且由同一个用户最后提交的作业的后边； 如果是 volclava 管理员发出的指令，bbot 命令将把指定的作业移到优先级相同并是最后提交的作业的后边。

2. **btop <JOB_id>**

   把作业移到你的作业队列的头部； 如果是普通用户发出的指令，btop 命令将把指定的作业移到优先级相同且由同一个用户最先提交的作业的前边； 如果是 volclava 管理员发出的指令，bbot 命令将把指定的作业移到优先级相同并是最先提交的作业的前边。

1.5.5 把作业从一个队列转移到另一个队列

通过使用 bswitch 命令，你可以把作业从一个队列中转移到另外一个队列中来。这个命令在很多时候是很有用的，比如你把作业提交进错误的队列中， 或者是当前队列阀值的限制或运行窗口的限制，队列中的作业不能被运行， 而你又希望能够恢复作业的运行。

1. **把一个单一的作业转移到另一队列中**

   运行 `bswitch` 命令，把运行的或待发的作业从一个队列转移到另一个队列。
   在下面的命令中，作业 `5309` 被转移到 `priority` 队列中
   `bswitch priority 5309`

2. **把所有的作业转移到另一个队列中**

   运行 `bswitch -q from_queue to_queue 0` 命令，把一个队列中所有的作业转移到另一个队列
   其中`-q` 选项代表移动队列的操作，使用作业 `ID 0` 表示要移动队列中所有的作业。在下面的例中，要将在 `night` 队列中的所有作业移动到 `idLe` 队列中
   `bswitch -q night idLe 0`

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.5.6 强制作业执行

可以使用 brun 命令来强制执行一个处于待发状态的作业。这个操作只能由 volclava 管理员来进行。 你可以强制一个作业在指定的节点上运行，忽略其他限制条件知道作业运行。

运行 brun -m hostname job_ID 来强制一个待发作业运行。你必须指定在哪个节点上运行作业
例如，在下面的例子中，强制作业 104 在 hostA 上运行；
brun -m hostA 104

1.5.7 挂起与恢复作业

一个作业可以被其用户或 volclava 管理员挂起。这样的作业，由于是由用户挂起的，所以在 bjobs 的输出中会显示为 USUSP。

1. **挂起一个作业**

运行 bstop job_ID
你的作业如果是在运行状态，则会转为 USUSP 状态；
如果作业是在待发状态，则会转为 PSUSP 状态。
bstop 3421   Job <3421> is being stopped
在上例中，作业 3421 被挂起。

bstop 会向作业发送下列信号:

- 针对并行或是交互式作业，发送 SIGSTP 信号---SIGTSTP 会被作业的主进程捕获，并分发给在其他节点上运行的从进程。
- 针对串行作业，发送 SIGSTOP 信号---SIGSTOP 不能被用户程序所捕获。SIGSTOP 信号可以在 lsf.conf 文件中，通过 LSB_SIGSTOP 参数来设置。

2. **恢复一个作业**

运行 bresum job_ID
bresume 3421
Job <3421> is being resumed
在上例中，作业 3421 被恢复。
恢复一个用户挂起的作业可能不能立即就使作业恢复 RUN 状态。如果你的作业挂起前已经处在运行状态，那么 bresume 命令首先把作业恢复到 SSUSP

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

状态，然后等待 sbatchd 根据负载情况来安排调度。

1.5.8 取消作业

使用 bkill 命令可以取消待发的作业，或向运行的作业发终止信号。在 linux 系统上默认的情况 bkill 向运行的作业发送的是 SIGKILL 信号。 在发送 SIGKILL 命令之前，SIGINT 和 SIGTERM 信号会被发送，以便让作业有机会捕获信号并清理退出。信号是由 mbatchd 发送给 sbatchd 的，sbatchd 会等到作业退出后再回过头来汇报作业情况。因为这个过程中会有一些延迟， 所以在使用 bkill 命令后，有一小段时间里 bjobs 命令可能会显示该作业还在运行。

1. **取消一个作业**

```
运行 bkill job_ID 命令
如下例中，使用 bkill 命令来取消作业 3421
bkill 3421
Job <3421> is being terminated
```

2. **取消多个作业**

```
使用 bkill 0 命令来取消所有待发的作业，或使用 bkill 0 加上-g， -J -m -q， 或 -u 选项来取消相应选项的作业。
下面这个命令取消了所有派发到 hostA 节点上的作业：
bkill -m hostA 0
Job <267> is being terminated
Job <268> is being terminated
Job <271> is being terminated
下面这个命令取消了所有属于 groupA 的作业：
bkill -g groupA 0
Job <2083> is being terminated
Job <2085> is being terminated
```

1.6. 管理用户

1.6.1 检查用户与用户组信息

你可以通过使用 busers 和 bugroup 命令来检查有关 volclava 用户与用户组的信息。 busers 命令能够显示用户与用户组的信息。默认情况下所显示的是发处理的用户

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

的相关信息。

1. **busers** 命令能够显示：
    - 在一个处理器上，用户或用户组最多可以执行的作业数目。
    - 在集群上，用户或用户组最多可以使用的作业槽的数目。
    - 在系统上，用户或用户组最多可以有多少个挂起的作业。
    - 所有这个用户提交的作业所需要的作业槽的总数量。
    - 处于 PEND，RUN，SSUSP 和 USUSP 状态的作业，所需要的作业槽的数量。
2. **bugroup** 命令能够显示用户组的信息，以及每个用户组的成员信息。

    注意：all 关键字是 volclava 系统保留的，请检查确保实际用户中没有人使用这个关键字。

    - 检查用户信息，运行 **busers all** 命令。
    - 检查用户挂起阀值，运行 **busers –w** 命令，它会在 busers all 输出的最后一列后追加一列，显示有关用户挂起作业的阀值。
    - 检查用户组信息，运行 **bugroup** 命令。
    - 检查用户共享信息，运行 **bugroup -l** 命令，它能够用长格式来显示有关用户共享组成员的信息。

1.6.2 用户组

用户组就像是一列用户的别名。管理员可以针对用户或用户组，设置一个处于运行状态作业的总数限制。你可以通过如下几个方法来设定 volclava 的用户组。

- 使用配置文件中已经存在的用户组。
- 创建 volclava 专用的用户组。

如果需要，你可以同时使用上述两种方法，但要确保用户和用户组名不同。

1. **使用现存的用户组作为 volclava 的用户组**

在你现有的操作系统上的用户组，一般代表着组织内用户的关系。自然而然，可以使用现有的用户组来控制计算资源的访问。你可以在任何需要使用 volclava 用户组的地方，指定其使用现有的 linux 用户组。

2. **volclava 是如何识别 linux 用户组**

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

volclava 只接受/etc/egroup 文件中所指定的用户成员，或是通过 group.byname 文件表示的 NIS 映射关系；用户在/etc/passwd 文件中所定义的第一用户组会被忽略。

在你第一次指定一个 linux 用户组时，volclava 会自动创建与之同名的 volclava 用户组。 用户组的成员信息是通过使用 getgrnam，在 mbatchd 启动时从主节点机上获取的。在不同节点机上，用户组成员信息可能是不同的。在 volclava 用户组建立之后，你可能又修改了 linux 上的用户组，但是 volclava 不会自动地进行相应的更新。你需要使用 badmin 命令重新配置 volclava，这样才能更新用户组。为了区别用户与用户组重名，请在用户组名后加斜线，例如 group_name/。

3. **必要条件**

请在集群所有节点机上，volclava 所要引用的 linux 用户组定义文件中，保持用户组的定义一致。如果各个节点上所定义的用户组不同，可能会带来不可预知的后果。

4. **volclava 是如何解决用户和用户组重名**

如果一个用户与某个用户组同名，volclava 在解析时认为所要引用的是用户。如果要指定所要引用的是用户组，请在用户组名后加斜线（/）。 例如，在你的系统上存在 admin 用户与 admin 用户组，volclava 把 admin 解释为用户，把 admin/解释为用户组。

5. **哪里需要使用现有的用户组**

在下列 volclava 配置文件中，你可以使用现有的用户组来定义参数：

- 在 lsb.queues 文件中，定义 USERS 参数来指定队列的授权用户。
- 在 lsb.users 文件中，定义 USER_NAME 参数来指定用户作业槽的使用限制。

6. **设置用户组**

```
使用 volclava 管理员身份登录到任何一个节点上
打开 lsb.users 文件
如果 UserGroup 段不存在，请创建它
在 GROUP_NAME 段下，指定用户组的名称
在 GROUP_MEMEBER 段下，指定用户
保存修改
运行 badmin ckconfig 命令来检查用户组的定义。如果错误报告，请修正错误后再次使用命令检查
运行 badmin reconfig 命令来重新配置集群
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1.6.3 取消 default 对用户组的影响

在 lsb.params 中增加一个开关配置：DEFAULT_LIMIT_IGNORE_USER_GROUP，可以取消 lsb.users 中 default 定义对 userGroup 的影响。

解决场景：

实现此功能之前，在 lsb.users 中 default 用户定义的效果是：其 default 配置为所有用户、**用户组**的默认配置；

但是无法满足我们实际的使用场景：

- 定义了 default 用户默认某个限制（如 MXJ）为 1000；
- 为用户 user_A 定义了（如 MXJ）的限制 2,000；
- 但是用户 user_A 也属于用户组 group_A，而 group_A 未做定义；

那此时用户组 group_A 的限制继承于 default 的定义限制 1000，也作用于 user_A 上，那么为用户 user_A 定义了 2,000 则永远打不满；

配置定义：

lsb.params 中新增 DEFAULT_LIMIT_IGNORE_USER_GROUP 的配置，取值范围：

```
DEFAULT_LIMIT_IGNORE_USER_GROUP=y|Y|n|N
```

作用效果：

lsb.users：default 配置是否影响用户组

```
Begin User
USER_NAME       MAX_JOBS        JL/P    MAX_PEND_JOBS
MAX_PEND_SLOTS
default          -               -         -                -
End User
```

效果示例：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

按如下配置：

- lsb.users 定义 default，同时限制 MAX_JOBS、JL/P、MAX_PEND_JOBS、MAX_PEND_SLOTS；
- 再定义 test1 用户组，但是不做额外的限制定义；
- 同时定义对比用户组 eng_users，定义限制作为对比观察；

```
Begin UserGroup
GROUP_NAME       GROUP_MEMBER
develop          (jwang long david ming dev )
system           (all)
test1            (tiger volclava)
eng_users        (zhang ahmedk pangj develop test1)
End UserGroup

Begin User
USER_NAME        MAX_JOBS        JL/P        MAX_PEND_SLOTS
MAX_PEND_JOBS
default          -               -           -
10
root             -               -           -
-
eng_users        5               -           -
7
tiger            -               -           20
20
volclava         5               -           -
200000
End User
```

上述配置调整好后，执行 badmin reconfig，在通过如下命令即可看到相应效果：

- buser -w all 展示：
  - default 配置影响所有用户的默认配置
  - default 配置不影响所有用户组的默认配置
    - test1 用户组：因没有在 lsb.users 中单独配置，采用默认初始配置，不受 default 配置影响
    - eng_users 用户组：启用自身配置
- bparams -l 展示 DEFAULT_LIMIT_IGNORE_USER_GROUP = TRUE

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
[root@n144-252-149 ~]# busers -w all
USER/GROUP   JL/P   MAX  NJOBS   PEND   RUN  SSUSP  USUSP    RSV
MPEND  PJOBS MPJOBS
ahmedk         -     -      0      0     0      0      0      0
-      0      10
david          -     -      0      0     0      0      0      0
-      0      10
default        -     -      -      -     -      -      -      -
-      -      10
dev            -     -      0      0     0      0      0      0
-      0      10
develop        -     -      0      0     0      0      0      0
-      0       -
eng_users      -     5      0      0     0      0      0      0
-      0       7
jwang          -     -      0      0     0      0      0      0
-      0      10
long           -     -      0      0     0      0      0      0
-      0      10
ming           -     -      0      0     0      0      0      0
-      0      10
volclava       -     5      0      0     0      0      0      0
-      0  200000
pangj          -     -      0      0     0      0      0      0
-      0      10
root           -     -      0      0     0      0      0      0
-      0       -
system         -     -      0      0     0      0      0      0
-      0       -
test1          -     -      0      0     0      0      0      0
-      0       -
tiger          -     -      0      0     0      0      0      0
20      0      20
zhang          -     -      0      0     0      0      0      0
-      0      10

[root@n144-252-149 ~]# bugroup -r
GROUP_NAME    USERS
develop       ming jwnag david long dev
system        all users
test1         volclava tiger
eng_users     zhang ahmedk pangj develop test1
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
[root@n144-252-149 ~]# bparams -l | grep
DEFAULT_LIMIT_IGNORE_USER_GROUP -C 1
the limits of 'default' user set in lsb.users would ignore user
group:
    DEFAULT_LIMIT_IGNORE_USER_GROUP = TRUE
```

2. 资源的操作

2.1 理解资源

2.1.1 volclava 资源

1. **查看可用资源。**使用 **lsinfo** 列出你的集群上存在的资源，如下所示:

```
[root@master-test etc]# lsinfo
RESOURCE_NAME   TYPE   ORDER  DESCRIPTION
r15s          Numeric   Inc   15-second CPU run queue length
r1m           Numeric   Inc   1-minute CPU run queue length
(alias: cpu)
r15m          Numeric   Inc   15-minute CPU run queue length
ut            Numeric   Inc   1-minute CPU utilization (0.0 to
1.0)
pg            Numeric   Inc   Paging rate (pages/second)
io            Numeric   Inc   Disk IO rate (Kbytes/second)
ls            Numeric   Inc   Number of login sessions (alias:
login)
it            Numeric   Dec   Idle time (minutes) (alias:
idle)
tmp           Numeric   Dec   Disk space in /tmp (Mbytes)
swp           Numeric   Dec   Available swap space (Mbytes)
(alias: swap)
mem           Numeric   Dec   Available memory (Mbytes)
ncpus         Numeric   Dec   Number of CPUs
ndisks        Numeric   Dec   Number of local disks
maxmem        Numeric   Dec   Maximum memory (Mbytes)
maxswp        Numeric   Dec   Maximum swap space (Mbytes)
maxtmp        Numeric   Dec   Maximum /tmp space (Mbytes)
cpuf          Numeric   Dec   CPU factor
......
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

2. **查看主机资源。**运行 lshosts 获取主机上定义的资源列表，如下所示：

```
[root@master-test etc]# lshosts
HOST_NAME      type    model  cpuf ncpus maxmem maxswp server
RESOURCES
master-test   linux  IntelI5 100.0     4 32012M 10239M    Yes
(cs)
cmp1-test     linux  IntelI5 100.0     4 31993M 10239M    Yes
(cs)
cmp2-test     linux  IntelI5 100.0     4 31993M 10239M    Yes
(cs)
```

2.1.2 资源如何分类

1. **根据值类型分类**
   - **布尔资源：**使用布尔值来表示某些特征是否存在的资源。
   - **数值资源：**使用数字来表示的资源，例如负载量索引，一个节点上处理器的个数，或节点的 CPU 参数。
   - **字符串资源：**使用字符值来表示的资源，例如节点类型、节点模型和节点状态。
2. **根据值的改变分类**
   - **动态资源：**动态改变其值的资源：节点状态和负荷指数。
   - **静态资源：**值不改变的资源：除了节点状态和负荷指数的资源。
3. **根据定义分类**
   - **外部资源：**用户自定义的客户资源：外部负荷指数和 lsf.shared 中定义的资源（共享资源）。
   - **内置资源：**volclava 中一直存在的资源。例如负荷指数、CPU 个数或交换空间。
4. **根据范围分类**
   - **基于节点的资源：**不在节点间共享，依赖于单个节点的资源，例如交换空间、CPU 或内存。应用程序只有运行在特殊的节点上才能访问这些资源。一个节点上的内存耗尽并不影响其他节点的内存。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

- **共享资源：**不依赖于单个节点，而属于整个集群或集群子集的资源，例如流动许可或共享文件系统。应用程序可以从任何一个配置了共享资源的节点上进行访问， 但这么做会影响其它主机看到的这个值。

- **布尔资源：**布尔资源（例如，server 来指定 volclava 服务主机）为一个主机所定义时值为 1，没有被定义时值为 0。使用布尔资源来配置主机属性，并在选定的主机中运行作业。例如：

  i. 节点可能有不同的操作系统类型和版本。

  ii. 节点可能在系统中扮演不同的角色，例如文件服务器或计算服务器。

  iii. 节点可能为一些应用配置特殊目的的设备。

  iv. 一些特定的软件包或者许可证只存在于一部分节点中。

  在作业的资源需求部分指定布尔资源，以此来选择可以运行此作业的节点。布尔资源的一些例子：

资源名称	描述	样例的解释
cs	在集群中的角色	计算节点
fs	在集群中的角色	文件服务节点

5. **查看节点的共享资源**

   运行 bhosts –s 来查看节点的共享资源。

2.1.3 如何使用资源

当作业通过 volclava 系统提交后，它们运行时所使用资源的情况将被记录下来。这些信息是用来加强资源的使用限制和负荷阈值。

1. **volclava 收集如下信息**：

   - 运行本作业的所有进程耗费的总 CPU 时间 。
   - 运行本作业的当前所有进程占有的总内存（以 KB 为单位） 。
   - 运行本作业的当前所有进程占有的总虚拟内存（以 KB 为单位）。
   - 运行本作业的当前活动进程所属组号 。
   - 运行本作业的当前活动进程号 。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

在 linux 中，作业级资源的使用情况是通过一个名为 PIM（Process Information Manager）的特殊进程收集的。PIM 在内部由 volclava 管理。PIM 每 30 秒采样一次资源使用信息，这些信息被 sbatchd 以最大频率为每秒 SBD_SLEEP_TIME（在 lsb.params 文件中配置）收集一次，然后传递给 mbatchd。

2. **查看作业资源使用情况**

运行 bjobs -l 来查看本作业当前资源使用情况。

信息更新仅在以下情况执行：CPU 使用时间、内存使用或虚拟内存使用与上一次更新比较，其变化超过 10%时；或者一个新的进程或进程组被创建时。

3. **查看一个节点的负荷信息**

运行 bhosts –l 来检测本节点上的负荷级别，如果有必要可以调整节点或队列的悬挂条件。 bhosts –l 命令同时显示了最近调度作业所使用的资源值。输出中的短线（-）表示没有定义对应的阈值。如下所示：

```
[root@master-test etc]# bhosts -l master-test
HOST  master-test
STATUS       CPUF    JL/U    MAX  NJOBS    RUN  SSUSP  USUSP
RSV DISPATCH_WINDOW
ok           100.00     -      4      0      0      0      0
0      -

 CURRENT LOAD USED FOR SCHEDULING:
             r15s   r1m  r15m    ut    pg    io   ls    it
tmp   swp   mem  slots
 Total        0.0   0.0   0.0    0%   0.0     1    1     1
3320M   10G   28G      4
 Reserved     0.0   0.0   0.0    0%   0.0     0    0     0
0M    0M    0M      -

 LOAD THRESHOLD USED FOR SCHEDULING:
           r15s   r1m  r15m   ut      pg    io   ls    it
tmp    swp    mem
 loadSched   -     -     -     -       -     -    -     -
-      -      -
 loadStop    -     -     -     -       -     -    -     -
-      -      -
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

2.1.4 负荷指数

负荷指数是内置资源，它用来衡量 volclava 集群中每个节点上的静态、动态或非共享资源的可用性。

定义在 LIM 中的负荷指数在固定周期内会被更新一次。LIM 收集的机器负荷指数如下：

复合指数	测量	单位	变化趋势	在...期间取平均	更新间隔
status	节点状态	字符			15 秒
r15s	运行队列长度	进程个数	增加	15 秒	15 秒
r1m	运行队列长度	进程个数	增加	1 分钟	15 秒
r15m	运行队列长度	进程个数	增加	15 分钟	15 秒
ut	CPU 使用率	百分率	增加	1 分钟	15 秒
pg	内存换页率	每秒的换入与换出和	增加	1 分钟	15 秒
ls	登录用户	用户	增加	N/A	30 秒
it	空闲时间	分钟	减少	N/A	30 秒
swp	可用的交换空间	MB	减少	N/A	15 秒
mem	可用的内存	MB	减少	N/A	15 秒
tmp	可用临时文件系统空间	MB	减少	N/A	120 秒
io	磁盘 I/O (通过 lsload -I 来显示)	KB 每秒	增加	1 分钟	15 秒

机器状态描述如下：

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

状态	描述
OK	节点可以接受远程作业。LIM 可以选择此类节点执行远程作业。
-OK	前面带有短线（-）标明 LIM 存在，但是节点上没有运行 RES 或 RES 没有反应。
Busy	节点超负荷（busy），因为某些负荷指数超过其阈值。星号（*）标出了不合适的指数。LIM 不会选择这类节点来执行交互作业。
lockW	节点被它的运行窗口锁住，使用 lshosts 显示运行窗口
lockU	节点被 volclava 管理员或 root 用户锁
Unavail	节点宕机，或节点上没有运行 LIM，或 LIM 没有反应

1. **CPU 运行队列长度（r15s， r1m， r15m）**

r15s， r1m 和 r15m 是在 15 秒，1 分钟和 15 分钟内 CPU 运行队列的平均长度，这是在给定间隔内准备使用 CPU 的进程的平均个数。

在 linux 中，运行队列长度指数不一定与通过 uptime（1）命令打印出来的平均负荷相同，这是因为 uptime 平均负荷在一些平台上还包括短时间内为等待状态 （如页面调度或读出/入磁盘）的进程。

2. **有效运行队列长度**

在多处理器系统中，同一时刻有不止一个的进程在运行。

volclava 在多处理器系统中同样衡量运行长度值，以使得单处理器和多处理器的 CPU 负荷可比较。这个衡量值称为有效运行队列长度。

使用 lsload –E 来查看有效运行队列长度 。

3. **标准运行队列长度**

volclava 根据处理器的相对速度（CPU 参数）来调整 CPU 运行队列长度。 标准运行队列长度包括调整处理器个数和 CPU 速度。具有最低标准运行队列长度的 CPU 将尽快运行一个加强 CPU 作业。

使用 lsload –N 来查看标准 CPU 运行队列长度。

4. **CPU 利用率**

ut 指数是用来衡量 CPU 的使用情况，它是运行系统和用户代码使用时间的百分数。没有进程运行的节点的 ut 值为 0%；而 CPU 完全使用的 ut 值为 100%。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

5. **页面交换率**

pg 指数是虚拟内存每秒中交换页面的频率。

这个指数与可用的 RAM 大小及节点上运行的进程的总大小密切相关。如果没有足够多的 RAM 满足所有进程，页面交换率就会很高。

页面交换率是节点如何回应交互使用的一个很好衡量，当节点页面交换负载比较重时，反应就很迟钝。

6. **登录用户个数**

ls 指数给出了登录用户的个数。每个用户只计算一次，而不管他们登录节点的次数。

7. **交互空闲时间**

在 linux 上，it 指数是节点的交互空闲时间，单位为分。 空闲时间从上次输入或输出算起，包括直接与终端或支持登录会话的网络虚拟终端的交互。 这不包括直接通过 X 服务的活动，例如 CAD 应用程序或 emacs 窗口。

8. **临时文件夹（tmp）**

tmp 指数是在包含临时目录的文件系统上的可用空间（MB 为单位）。linux 上为 /tmp 。

9. **交换空间（swp）**

swp 指数给出当前存在的虚拟内存（交换空间）大小，以 MB 为单位。 这表示此节点上可以开始的最大进程。

10. **内存 （mem）**

mem 指数是当前对用户进程可用的实际内存大小的估计。这是此节点上可以运行且不需要进行页面交换的最大进程的大概值。

LIM 报告可用空闲内存的大小。LSF 是通过计算物理空闲内存、高速缓冲内存、缓冲内存及调整值的和来估计空闲内存大小。

命令 vmstat 也会报告空闲内存值，但是它分开显示这些值。 由 LIM 和 vmstat 报告的空闲内存大小可能不同，因为虚拟内存的行为在不同操作系统间是有区别的。

可以写一个 ELIM 来覆盖由 LIM 返回的空闲内存值。

11. **输入/输出率**

指数 io 来衡量直接附在本节点上的磁盘 I/O 吞吐量，单位 KB/s。它不包括从其他节点上借来的磁盘 I/O。

12. **查看负荷指数信息**

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

lsinfo -l 命令显示系统中所有存在的负荷指数信息。

lsload -l 命令显示了所有负荷指数的值。

2.1.5 静态资源

静态资源是内置资源，它代表不随时间而改变的节点信息，例如对用户进程可用的最大 RAM 值，或节点中处理器的个数。 大多数的静态资源在 LIM 启动时、或在 volclava 检测到硬件配置改变时就可以确定。

作业可以使用静态资源来选择合适的节点，以满足特殊作业对机器架构的需求。资源 ncpus， maxmem， maxswp， 和 maxtmp 在支持动态硬件配置的 linux 节点上不是静态的。

静态资源由 LIM 汇报，如下表所示：

指数	描述	单位/类型	取决于
type	节点类型	字符串	配置
model	节点结构	字符串	配置
hname	节点名称	字符串	配置
cpuf	CPU factor	相对值	配置
server	节点可以运行远程作业	布尔值	配置
rexpri	执行优先级	Nice (2) 参数	配置
ncpus	处理器个数	处理器	配置
ndisks	本地磁盘个数	磁盘	LIM
maxmem	最大 RAM 值	MB	LIM
maxswp	最大交换空间	MB	LIM

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

maxtmp	/tmp 的最大空间	MB	LIM

1. **节点类型（type）**

节点类型名称通常基于硬件名称和操作系统的组合。 所有运行在同样的计算机架构并且具有相同操作系统的计算机属于同一个类型。 可以在 lsf.shared 的 HostType 区域添加客户节点类型。这种包含文字和数字的值最长可达 39 个字符。如节点类型 LINUX86。

2. **节点结构 (model)**

节点模型包含节点类型和 CPU 速度（CPU 参数）。 所有的具有相同类型和 CPU 速度的节点都可指定为同一种节点结构。可以在 lsf.shared 的 HostModel 区域添加客户节点类型。这种包含文字和数字的值最长可达 39 个字符。如节点模式 InteL_IA64。

3. **CPU 参数（cpuf）**

CPU 参数（经常简写为 cpuf）表示相对于集群中其他节点的 CPU 速度。 例如，如果一个处理器速度是另外一个的两倍，它的 CPU 参数就应该是另外一个的两倍大。对于多处理器节点，CPU 参数是单个处理器的速度，volclava 将自动衡量节点的 CPU 负荷来处理额外的处理器。

CPU 参数自动检测或由管理员定义。

4. **服务器（server）**

表示节点功能类型。在 lsf.cluster.cluster_name 文件的 Host 配置中，可以为节点定义 server 列的值，如下：

- 如果节点作为管理节点或者执行作业节点用来运行作业，配置值为 1；
- 如果节点是 volclava 客户端，只用来向其他节点提交作业，配置值为 0；

5. **CPU 个数（ncpus）**

默认情况下，CPU 个数表示节点上物理处理器的个数。 由于大多数 CPU 都由多核、多线程及多处理器组成，所以 ncpus 可以由管理员 （全局或单个节点）定义为下列值：

- 处理器个数。
- 处理器的内核个数 。

6. **磁盘个数（ndisks）**

磁盘个数指的是一台节点上的本地磁盘个数，由 LIM 确定。

7. **最大内存（maxmem）**

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

最大内存是一台节点上存在的总内存大小，以兆字节（MB）衡量。

8. **最大交换空间（maxswp）**

   最大交换空间是一台节点上拥有的总交换区大小，以兆字节（MB）衡量。

9. **最大临时空间（maxtmp）**

   最大临时空间是一台节点上拥有的总临时空间大小，以兆字节（MB）衡量。

2.1.6 自动检测硬件配置

1. **maxmem， maxswp， maxtmp 动态变化**

   假如动态地改变了 maxmem， maxswp，或 maxtmp，而处理器的数量却没有发生改变，你就必须通过命令 lsadmin limrestart 重启本地 LIM，使得它能够识别这些变化。

   如果动态地改变了处理器数量以及 maxmem， maxswp，或 maxtmp，volclava 会自动识别这些变化。 当它检测到处理器数量的变化时，本地 LIM 也会去检测 maxmem， maxswp，maxtmp。

2. **查看硬件的动态变化**

   ls-command 命令在 ls-command 识别到这些变化时，大约会有 2 分钟的时延（例如，在 lshosts 显示变化前）。

   b-command 在 b-command 命令识别到这些变化时，大约会有 2 + 10 分钟的时延（例如，在 bhosts -l 显示变化前） 。

   这是由于 mbatchd 会每隔 10 分钟与 master LIM 通讯一次。

2.2 添加新资源

2.2.1 如何给集群添加新资源

下面概述集群添加新资源的步骤:

1. 在集群中的任何一台节点上以 volclava 管理员身份登录。

2. 在 lsf.shared 中的 Resource 段定义新资源。 至少要指定一个名字和一个简要描述，它们将会被用户以 lsinfo 指令的方式显示。请参考 lsf.shared 文件中 Resource Section 配置。

3. 对于静态布尔资源及静态或动态 string 资源、任何有新资源的节点，将其资源名称添加到 lsf.cluster.cluster_name 文件中 Host 段的 RESOURCES 栏目中。参考

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

lsf.cluster.cluster_name 文件中 Host Section 配置。

4. 对于共享资源、任何有新资源的节点，把资源和节点相关联（或许你也有理由在这一段配置成非共享资源类型）。参考 lsf.cluster.cluster_name 文件中 ResourceMap Section 配置。

5. 重新配置你的集群。

2.2.2 如何配置 lsf.shared 的 Resource 段

除了内置资源，集群内的所有资源都需要定义在 lsf.shared 文件中的 Resource 段内。 共享与非共享资源的配置没有区别。 至少使用关键字 RESOURCENAME 和 DESCRIPTION 定义一个名字和对该资源的描述。 配置示例如下:

```
Begin Resource
RESOURCENAME  TYPE    INTERVAL INCREASING  DESCRIPTION         #
Keywords
   fs         Boolean ()       ()          (File server)
   cs         Boolean ()       ()          (Compute server)
#  nio        Numeric 60       Y           (Network I/O in
Kbytes/second)
#  console    String  60       ()          (Console user name)
End Resource
```

- 资源名字不能够以数字开头。
- 资源名不能包含下列的字符。

  : . ( ) [+ - * / ! & | < > @ = ¨

- 资源名不能是下列任何保留关键字之一:

  cpu cpuf io Logins Ls idLe maxmem maxswp maxtmp type Model status it mem ncpus nprocs ncores nthreads define_ncpus_cores define_ncpus_procs define_ncpus_threads ndisks pg r15m r15s r1m swap swp tmp ut

- 为了避免与第三方中的 inf 和 nan 关键字发生冲突，资源名字不能以 inf 或 nan 开头（包括大写和小写）

  诸如-R "infra" 或 -R "nano"此类的资源请求串将导致错误。

  使用-R "defined（infxx）" or -R "defined（nanxx）"来指定资源名称。

- 资源名是大小写敏感的。
- 资源名最长可达到 39 个字符。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

你也可以指定

- 资源类型（TYPE = Boolean | String | Numeric），默认为 Boolean。
- 对于动态资源，需要定义更新间隔（INTERVAL， 以秒为单位）
- 对于数值类型资源，可以定义趋势含义。如果一个较高的值意味着较高的负荷，则定义 INCREASING 列为 Y，反之为 N。
- 可选列 RELEASE，用于定义资源是否可以被释放。对于数值型的共享资源，当一个正在使用该资源的作业被挂起时，volclava 就释放该资源（RELEASE = Y）。

当没有指定可选的属性时，该资源就被视为静态及布尔类型

- 可选列 CONSUMABLE，用来定义可消耗的资源。

如果一个资源为可消耗的，就配置 CONSUMABLE 列为 Y。 动态和静态数值资源都可以被指定为可消耗的。

可消耗属性的默认值为：

- 内置指数

Ø 以下为可消耗的：r15s， r1m， r15m， ut， pg， io， ls， it， tmp， swp， mem。

Ø 所有其它内置静态资源是不可消耗的（例如，ncpus， ndisks， maxmem， maxswp， maxtmp， cpuf， type， model， status， rexpri， server， hname）。

- 外部共享资源

Ø 任何数值类型资源是可消耗的

Ø 字符串和布尔类型资源是不可消耗的。

- 你只能在资源请求字符串中的 rusage 段指定可消耗的资源。rusage 段中的不可消耗资源会被忽略掉。
- 不可消耗的资源不能够被释放。
- 不可消耗的数值型资源应当能够在一个资源请求字符串的 order， select 段中使用。

2.2.3 如何配置 lsf.cluster.cluster_name 中的 Host 段

在 lsf.cluster.cluster_name 中只有 Host 段是必需的。它列出所有在该集群中的节

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

点以及每台节点的配置信息。 在 lsf.shared 中的 Resource 段中定义的资源可以配置在 RESOURCES 列中。

你可以在 RESOURCES 列中配置任意数量的资源（用圆括号括起来，并由空格符或者制表符分隔）。对于动态数值和字符串资源，直接使用 resource_name。字符串资源不能包含空格。静态数值或者字符串资源使用下列的语法格式：resource_name=resource_value 。如下：

```
Begin  Host
HOSTNAME   Model    type     server     r1m   mem   swp
RESOURCES #Keywords
Lemon     PC200    LINUX86   1          3.5     1     2
(!Linux dataSize=big)
pLum      !        NTX86     1          3.5     1     2       (nt
res=5)
Host_name  !       NTX86     1          -       -     -       ()
End Host
```

如果需要定义跨节点的共享资源，应当使用 ResourceMap 段。 如果在 Host 段和 ResourceMap 段的 resource 列都定义了资源，resource 列中的定义将起作用。

2.2.4 配置 lsf.cluster.cluster_name 中 ResourceMap 段

在 lsf.cluster.cluster_name 的 ResourceMap 段为节点定义了资源，这些资源与节点相关联。

对于每个资源，你必须指定拥有它的节点名字。 如果 ResourceMap 段没有被定义，那么在 lsf.shared 中指定的任何动态资源就不与节点绑定，而在集群中被所有的节点共享。

2.3 资源分配限制

2.3.1 关于资源分配限制

1. **资源分配限制是做什么用的**

默认情况下，资源消费者如：用户，机器，队列或项目运行作业时没有资源限制。资源分配限制是配置在 lsb.queues 中的约束条件。

- 在不同类型的作业调度过程中，能分配给一个作业而使之运行的资源的最大

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

值。

- 该限制应用于哪个资源消费者。

如果所有的资源都被消耗掉，不能启动更多的作业，直到一些资源被释放。 例如，限制你的每一台机器上能使用的最大内存量，使你的系统的处理能力最优。通过定义往特定队列和指定的一些机器提交作业的用户的内存限制， 你可以防止这些用户耗尽系统中的所有内存。

2. **资源分配限制和资源使用限制**

资源分配限制和资源使用限制不同，后者在作业运行中起作用。例如：你设置 CPU 限制，内存限制和其他限制，它们在作业运行起来后起作用。

3. **volclava 是如何计算资源的**

节点机上的资源如果被已经启动且还没完成的作业使用就是不可用的。这就意味着运行中和挂起的作业所使用的资源不能超过其上所设定的限制。限制可设置在队列，节点机，项目以及处理器等资源上。

4. **作业 slot 限制**

作业 slot 限制是在任一时刻最多可运行的作业数。

例如：一个队列如果没有可用的作业 slot 就不能运行作业；机器没有可用的 slot 也不能运行作业。

限制如下：

- QJOB_LIMIT(lsb.queues)
- HJOB_LIMIT(lsb.queues)
- UJOB_LIMIT(lsb.queues)
- MXJ(lsb.hosts)
- JL/U(lsb.hosts)
- MAX_JOBS(lsb.users)

限制作业 slot 的数量。当负载是串行的时，作业 slot 通常等于作业数。对于并行作业或分布式应用来说，这些才是真正的作业 slot 限制而不是作业限制。

5. **转移的作业可以超过资源分配限制**

如果转移（bswitch）的作业还没有被派发出去，那么作业就好像重新被提交到新队列上，JOBS 限制也是遵从新队列的限制。

6. **资源消费者的限制机器组**

如果给机器组指定限制，将计算机器组中的所有机器使用的资源总和。如果一台

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

机器不止属于一个组，那么在该机器上运行的作业在他所属于的每个组中都被计算进去。

7. **用户和用户组限制**

正常情况下，作业按先到先服务（FCFS）排队。一些用户有可能提交大量的作业滥用系统，其他用户必须等这些作业完成。对用户进行资源限制可以防止用户垄断整个系统的资源。

用户可以无限提交作业，但如果任何资源限制达到了，剩下的作业处于等待调度状态，直到一些运行的作业完成或者有资源变得可用。 如果给用户组指定限制，将计算用户组中的所有用户使用的资源总和。如果一个用户不止属于一个组，那么该用户运行的作业在他所属于的每个组中都被计算进去。

用关键字 all 来配置应用于集群中每一个用户或用户组的限制。在一个大的集群中，如果你想从限制的定义中排除几个用户，这是很有用的。

8. **用户和用户组中每个用户的限制**

每个用户的限制对每个用户或用户组中的单个用户起作用。如果一个用户组包含子组，限制也应用到每个子组的成员。

每个用户的限制，如果用关键字 all，将应用到集群中的每一个用户。如果用用户组，限制将应用到每个组成员，而不是整个组。

2.3.2 配置资源使用限制

资源使用限制是在作业运行期间对使用的系统资源进行的一种限制，例如设置 CPU 限制、内存限制等，是在作业开始运行之后才生效的限制。

可以由 volclava 管理员（在 lsb.queues 中）为队列指定限制，也可以在提交时为作业指定限制。在队列级别指定的限制是硬限制，而在作业提交时指定的限制是软限制。

限制	作业语法（bsub）	队列语法（lsb.queues）	格式/单位
CPU time Limit	-c （小写）	CPULIMIT	[hours:]minutes[/host_name | /host_Model]
FiLe size Limit	-F	FILELIMIT	KB
Memory Limit	-M	MEMLIMIT	KB

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

Data segment size Limit	-D	DATALIMIT	KB
Stack segment size Limit	-S	STACKLIMIT	KB
Core fiLe size Limit	-C（大写）	CORELIMIT	KB
Run time Limit	-W	RUNLIMIT	[hours:]minutes[/host_name | /host_Model]
VirtuaL memory Limit	-V	SWAPLIMIT	KB

示例：
```
CPULIMIT=10/hostA
```

如果主机 A 的 CPU 因子为 2，主机 b 的 CPU 因子为 1 （主机 b比主机 A 慢），则这指定主机 A 或任何其他 CPU 因子为 2 的主机上的实际时间限制为 10 分钟。但是，如果 hostB 是执行主机，则 hostB 上的实际时间限制为 20 分钟（10 * 2 / 1）。

2.3.3 通用资源配额限制

2.3.3.1 它能解决什么问题

通用资源配额限制是 volclava 提供的一套**集中式**资源分配管理功能，管理员可以在一个统一的配置文件 lsb.resources 中，为“资源消费者”（谁在用资源，包括主机/主机组、队列、用户/用户组、项目）设置资源使用上限，例如某个用户在某个队列上最多能占用多少个作业槽位（slots）、多少内存（mem）、运行多少个作业（jobs），以及能使用多少自定义资源（如 license、GPU）等。

当某个资源使用者的资源用量达到配额上限后，该资源使用者新提交的作业将不再被调度，保持 PEND（排队等待）状态，直到有资源被释放。系统会在 bjobs -p/lp 中显示对应的 pending reason，告诉你具体是哪种资源超限、被哪条限制挡住。

应用通用资源配额限制，就不需要在队列上（lsb.queues 中）、主机上（lsb.hosts 中）和用户上（lsb.users 中）分散的定义 2.3.1 中提到的资源分配限制了，都可以集中在 lsb.resources 文件中定义资源分配限制。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

2.3.3.2 如何配置使用

- **配置文件与生效方式**

  在配置文件 lsb.resources（存放在 LSF_ENVDIR 目录下）中定义限制规则，该文件可选，如果不存在，集群不启用任何资源配额限制。修改文件后，执行 **badmin reconfig** 或 **badmin mbdrestart** 使新配置生效。新配置会在下一次调度时生效，不会影响已经在运行的作业。

- **一条限制的组成**

  每条限制以 **Begin Limit** 开头、**End Limit** 结尾，里面是一行一行的“关键字 = 值”配置项，以 # 开头的行是注释。下面是一条限制的完整骨架：

```
限额配置骨架
Begin Limit
NAME = demo_limit            # 限制名称（必填，全局唯一）
HOSTS = PER(hostA hostB)     # 生效的主机/主机组（可选）
QUEUES = SHARED(normal)      # 生效的队列（可选）
USERS = SHARED(user1)        # 生效的用户/用户组（可选）
PROJECTS = PER(proj1)        # 生效的项目（可选）
RESOURCES = [SLOTS, 10] [MEM, 50%]    # 限制的资源及上限（必填）
DESCRIPTION = "这是说明文字" # 说明（可选）
End Limit
```

  其中 **NAME** 和 **RESOURCES** 必填；其余四项不写时表示“不按这个维度限制”（等价于 SHARED(all)，即对所有对象生效）。

- **语法要点**

  - **NAME = strings**：限制的名字，只能使用字母、数字、下划线（_）和连字符（-），不能有空格，最长 60 个字符；名字全局唯一，重复定义会被忽略。

  - **HOSTS / QUEUES / USERS / PROJECTS（资源使用者）：**这四个关键字用来定义“这条限额管谁”，即限制作用的资源使用者范围，作业占用的资源会分别算在这些对象上。不写某个资源使用者，默认对所有对象生效（等价于 SHARED(all)）。资源使用者的具体含义和用法如下：

    - **HOSTS = PER | SHARED(主机名 主机组名 ... | all [~]主机名 ... | all [~]主机组名 ...)**：按机器限制，作用于在指定主机/主机组上运行的作业；

    - **QUEUES = PER | SHARED(队列名 ... | all [~]队列名 ...)：**按队列限制，

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

作用于提交到指定队列的作业；

- **USERS = PER | SHARED(用户名 用户组名 ... | all [~]用户名 ... | all [~]用户组名 ...)：**按用户限制，作用于指定用户/用户组提交的作业；
- **PROJECTS = PER |SHARED(项目名 ... | all [~]项目名 ...)：**按项目限制，作用于指定项目下的作业。
- **value 语法格式说明**：资源使用者的对象列表写在关键字后面的括号里，括号内以空格分隔。每个维度都必须从 PER(...) 和 SHARED(...) 中二选一来定义取值；不同维度之间可以自由组合，各自独立选择 PER 或 SHARED，互不影响。详细说明如下：
  - PER(...) 与 SHARED(...)：定义限制计算类型
    - PER（各自计数）：对列表中的每个对象分别计数。例如 HOSTS=PER(hostA hostB) 表示 hostA、hostB 每台机器各自有独立的配额；
    - SHARED（共享计数）：列表中的所有对象共用一个计数。例如 USERS=SHARED(userA userB) 表示 userA 和 userB 加在一起总共只能用这么多。
  - all 与排除符 ~：all 表示“所有”；~名字 表示从范围内排除。例如 USERS=SHARED(all ~user1) 表示“除 user1 外的所有用户共享一个配额”。
  - 组名：可以直接写主机组/用户组名（如 hg1、ug1），或者末尾加斜杠（如 hg1/）显示表示组。

- **RESOURCES=[keyword, value] [keyword,value] ...：**规定资源消费者可用的资源上限，由一个或多个 `[keyword，value]` 组成。配置多个时，表示同时规定多项资源上限，其中任何一项达到上限了，资源使用者都无法运行更多的作业。keyword 是资源类型，value 是对应的上限值，支持以下类型：
  - SLOTS： [SLOTS, integer]

    资源消费者可用的最大作业槽位数量。请指定一个大于或等于 0 的正整数。
  - JOBS：[JOBS, integer]

    资源消费者可用的作业数量上限，包括正在运行或被挂起（RUN、SSUSP、USUSP）的作业。请指定一个大于或等于 0 的正整数。
  - MEM：[MEM, integer[%]]

    资源消费者可用的最大内存量。单位默认 MB（或 lsf.conf 中

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

LSF_UNIT_FOR_LIMITS 设置的单位），可以指定大于等于 0 的整数，也可填百分比（如 50%）。

- TMP：[TMP, integer[%]]

  资源消费者可用的最大 tmp 量。单位同上，可填整数或百分比。

- SWP：[SWP, integer[%]]

  资源消费者可用的最大 swp 量。单位同上，可填整数或百分比

- SLOTS_PER_PROCESSOR：[SLOTS_PER_PROCESSOR，float | !]

基于主机上的处理器数量计算的每处理器作业槽位限制。如果指定了 SLOTS_PER_PROCESSOR，则需要配置 HOSTS=PER(...)，否则该资源限额配置域将被忽略。

这个资源的取值是个非负数，可以是正整数，也可以是个小数，例如 0.5，因此它也可以作为多处理器机器上每颗 CPU 的限制。对于一台主机，该数值会向上取整为大于或等于其总作业槽位限制的最近整数。例如，如果 SLOTS_PER_PROCESSOR 为 0.5，那么在一台 8-CPU 的多处理器主机上，用户在任何时候最多只能使用 4 个作业槽位。而在单处理器机器上，用户最多可以使用 1 个作业槽位。 使用感叹号 ! 可以使作业槽位数量等于主机上的 CPU 数量。

- resourceName：[resoure_name, integer]

  限制用户自定义共享资源的数量。在 lsf.shared 中定义的数字类型的共享资源。指定一个大于或等于 0 的正整数。

注意事项：MEM/SWP/TMP 使用百分比、或使用 SLOTS_PER_PROCESSOR 时，必须配合 HOSTS=PER(...) 使用，否则该条限制会被忽略；

- **DESCRIPTION = "strings"**： 说明资源限制的用途等，长度限制为 4095 个字符。

- **配置示例**

示例 1：normal 队列上的作业，在 hg1 主机组的每台机器上，最多可使用 20 个作业槽位和 50% 的内存。

```
Begin Limit
NAME = demo_limit
HOSTS = PER(hg1)
QUEUES = SHARED(normal)
RESOURCES = [SLOTS, 20] [MEM, 50%]
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
DESCRIPTION = "This is demo"
End Limit
```

示例 2：userA 和 userB 一起最多使用 8 个 license；除 userA 外的每个用户在各个队列上最多可运行 10 个作业：

```
Begin Limit
NAME = shared_lic
USERS = SHARED(user1 user2)
RESOURCES = [lic, 8]
End Limit

Begin Limit
NAME = per_user_jobs
QUEUES = PER(all)
USERS = PER(all ~user1)
RESOURCES = [JOBS, 10]
End Limit
```

2.3.3.3 查看资源配额限制

- 使用 **blimits** 命令可以查看资源限制的配置情况以及使用情况：

blimits -c 查看限制的配置信息；用 -q（队列）、-u（用户）、-m（主机）、-p（项目）、-n（限额名）等参数过滤。

```
[root@v-y-volclava-dev1 etc]# blimits -c
 Begin Limit
   NAME          = demo_limit
   HOSTS         = PER(hg1/)
   QUEUES        = SHARED(normal)
   RESOURCES     = [SLOTS,20] [MEM,50%]
   DESCRIPTION   = "This is demo"
 End Limit

 Begin Limit
   NAME          = per_user_jobs
   QUEUES        = PER(all)
   USERS         = PER(all ~user1)
   RESOURCES     = [JOBS,10]
 End Limit
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
 Begin Limit
   NAME          = shared_lic
   USERS         = SHARED(user1 user2)
   RESOURCES     = [lic,8]
 End Limit
```

blimits：可以查看有运行作业的资源限制使用情况，同样可以用 -q（队列）、-u（用户）、-m（主机）、-p（项目）、-n（限额名）等参数过滤。

```
[user1@v-y-volclava-dev1 ~]$ blimits
INTERNAL RESOURCE LIMITS:

      NAME            USERS            QUEUES           HOSTS
PROJECTS      SLOTS    MEM      TMP      SWP     JOBS
 per_user_jobs         root             normal             -
-            -       -        -        -      1/10

EXTERNAL RESOURCE LIMITS:

      NAME            USERS            QUEUES           HOSTS
PROJECTS         lic
   shared_lic     (user1 user2)          -                -
-             1/8
```

blimits -a：显示所有资源分配限制的信息。对于有正在运行作业的限制，blimits 显示限制使用情况信息。对于没有任何正在运行作业的限制，blimits 显示资源限制配置信息。

```
[user1@v-y-volclava-dev1 ~]$ blimits -a
INTERNAL RESOURCE LIMITS:

      NAME            USERS            QUEUES           HOSTS
PROJECTS      SLOTS    MEM      TMP      SWP     JOBS
   demo_limit            -               normal           @hg1/
-          -/20    -/50%      -        -        -
 per_user_jobs         root             normal             -
-            -       -        -        -      1/10

EXTERNAL RESOURCE LIMITS:

      NAME            USERS            QUEUES           HOSTS
PROJECTS         res2
   shared_lic     (user1 user2)          -                -
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
-              1/8
```

更全面 blimits 用法，可以在集群环境中查看 blimits 的 manpage 手册。

```
[volclava@master-test ~]$ man blimits
```

- 作业因配额被挡住时，用 **bjobs -p 或者-lp** 查看 pending reason，会明确提示是哪种资源超限、对应哪条限额。

```
[volclava@master-test ~]$ bjobs -p
JOBID   USER    STAT  QUEUE       FROM_HOST      JOB_NAME
SUBMIT_TIME
313     volclava PEND  short       v-y-volclava-d sleep 300
Aug 10 17:08
 RESOURCES limit defined on queue has been reached (RESOURCES:
slots, Limit Name: test_limit1, Limit Value: 4);
```

2.3.3.4 与 LSF 的兼容性

volclava 的 lsb.resources 同时兼容传统 LSF 的限制定义格式。volclava 会按“限制块”自动识别格式：同一个 Begin Limit ... End Limit 块内不能混用两种格式。新配置建议优先使用 volclava 格式（这也是默认推荐格式）。

volclava 格式与 LSF 格式的主要差异：

对比项	volclava	LSF
资源使用者写法	HOSTS = PER(...) / SHARED(...)，队列、用户、项目同理	HOSTS = 列表 或 PER_HOST = 列表，队列、用户、项目同理
资源上限写法	统一写在 RESOURCES = [key, value] [key, value]	用 SLOTS = xx、JOBS = xx 等独立关键字
关键词	只支持复数 RESOURCES	RESOURCE

volclava 暂不支持以下配置项。某个限制如果使用了其中任意一项，整个限额将被忽略；未使用这些配置项的 LSF 限额，可以正常兼容解析：

- **APPS / PER_APP**：volclava 目前不支持 application 配置；

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

- **ELIGIBLE_PEND_JOBS**、**INELIGIBLE**：尚未支持计算作业的合理 pending 时间；
- **JOBS_PER_SCHED_CYCLE**：暂不支持；
- **LIC_PROJECTS / PER_LIC_PROJECT**：不支持 license project 概念。

2.4 预留资源

2.4.1 关于资源预留

当一个作业被调度出去以后，系统假定该作业消耗的资源能反映到负载信息中。但是，很多作业刚启动时并没有消耗它所要求的资源。实际上它们在一定时间周期内使用资源。

例如：一个作业要求 100MB 交换区空间，它被调度到一台有 150MB 可用交换区的机器上。该作业初始化分配了 5MB，在 30 分钟内逐渐增长到 100MB。在这段时间内，其他要求 50MB 以上交换区的作业不能在同一个机器上启动，以免过量的消耗资源。

资源可以被预留，以防止 volclava 过量使用。资源预留需求可以在提交作业的时候作为资源需求的一部分指定，或者配置在队列的资源的需求中。

1. **资源预留是如何工作的**

当决定是否把一个作业调度到一台机器上，volclava 考虑先前提交到该机器上的作业所预留的资源。对于每一个负载项，在一个机器上的所有作业预留的总量， 从 LIM 上报的当前值中扣除（如果负载项是增长性的，则增加）后便得到调度新作业可用的资源数量，算式如下：available amount = current value - reserved amount for all jobs 。

```
例如: bsub -R "rusage[tmp=60:duration=60:decay=1]" my job
```

将为作业预留 60MB 的临时空间。当作业运行起来，预留的数量大约以每分钟 1MB 的速度递减直到 60 分钟后预留量变成 0。

2. **队列级和作业级预留**

队列级资源需求参数 RES_REQ 也可能指定资源预留。如果队列预留一定数量的资源，作业级就不能预留比队列更多的资源。

例如：如果 bqueues –l 命令输出中包含：RES_REQ: rusage[mem=40:swp=80:tmp=100]，下面的作业提交将被拒绝，因为它要求的资源超过队列中的值。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
bsub -R "rusage[mem=50:swp=100]" myJOB
```

2.4.2 使用资源预留

1. **队列级资源预留**

在队列级，资源预留使你能够在队列上为作业指定一定数量的资源。它同时也作为用户提交作业时指定的资源预留的上限。

2. **队列级资源预留和等待调度的原因**

使用 RES_REQ 影响到 bjobs 显示的作业等待调度的原因。如果 RES_REQ 定义在队列中并且 LoadSched 阀值没有指定，个别负载项的等待调度的原因将不被显示。

3. **在队列级配置资源预留**

队列级配置资源预留可以作为 RES_REQ 参数的一部分。在资源需求字符串中用资源使用（rusage）段来指定一个作业启动后将预留的资源。

例如：

```
Begin  Queue
RES_REQ = seLect[type==any] rusage[swp=100:mem=40:duration=60]
End Queue
```

这将允许一个作业被调度到队列所配置的任一台机器上，预留 100MB 交换区，在 60 分钟的周期内预留 40MB 内存。

```
Shell
Begin  Queue
RES_REQ = seLect[type==any] rusage[mem=20||mem=10:swp=20]
End Queue
```

这将允许一个作业被调度到队列所配置的任一台机器上。该作业试图预留 20MB 内存，或当 20MB 内存不可用的时候预留 10MB 内存和 20MB 交换区。

4. **作业级预留**

在作业级指定资源预留，用 bsub –R 并且在资源需求字符串中包括资源使用字段。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

2.4.3 定义资源预留方式

默认情况下，作业请求的基于主机类型资源（如 memory），按着主机个数计算预留；共享类型资源（如 license 资源）按着 job 来计算预留。在 lsb.params 文件中可以使用 RESOURCE_RESERVE_PER_TASK=Y/N，用来控制资源预留方式，修改参数值后，运行 badmin mbdrestart 使参数生效，bhosts -l 可以查看机器上资源使用情况。

- 当 RESOURCE_RESERVE_PER_TASK=N(default): 基于主机类型资源按执行机器计算，共享类型资源按 job 计算。例如：

```
bsub -n 4 -R "rusage[mem=500:lic=1]" my_job #lic 是 shared 资源
```

如果 my_job 运行在两个机器上：host1 和 host2 各占用 2 个 slots，那么 my_job 会在 host1 上预留 **500MB** 的内存，host2 上预留 **500MB** 内存，全局预留 1 个 lic 资源。

- 当 RESOURCE_RESERVE_PER_TASK=Y: 基于主机类型资源和共享类型资源都按指定的 tasks 个数（-n）计算。例如：

```
bsub -n 4 -R "rusage[mem=500:lic=1]" my_job #lic 是 shared 资源
```

如果 my_job 运行在两个机器上：host1 和 host2 各占用 2 个 slots，那么 my_job 会在 host1 上预留 **500m*2=1G** 的内存，host2 上预留 **500m*2=1G** 内存，预留 1*4=4 个 lic 资源。

2.4.4 查看资源预留信息

1. **查看机器级别的资源信息（bhosts）**

   用 bhosts –l 来显示在每台机器上的资源预留量。

```
[root@master-test etc]# bhosts -l ic-cmp02
HOST  ic-cmp02
STATUS       CPUF    JL/U    MAX  NJOBS    RUN  SSUSP  USUSP
RSV DISPATCH_WINDOW
ok           100.00     -    128      1      1      0      0
0      -

 CURRENT LOAD USED FOR SCHEDULING:
             r15s   r1m  r15m    ut    pg    io   ls    it
tmp   swp   mem  slots
 Total        0.0   0.0   0.0    0%   0.0    74    1     1
89G    0G  995G    127
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
 Reserved     0.0   0.0   0.0    0%   0.0     0    0     0
0M    0M  500M      -

 LOAD THRESHOLD USED FOR SCHEDULING:
           r15s   r1m  r15m   ut      pg    io   ls    it
tmp    swp    mem
 loadSched   -     -     -     -       -     -    -     -
-      -      -
 loadStop    -     -     -     -       -     -    -     -
-      -      -
```

3. 作业调度和派发

3.1 作业调度策略

3.1.1 先来先服务（FCFS）

在不应用任何调度策略时，volclava 按着队列的优先级调度作业，一个队列或者相同优先级队列上的作业按着先来先服务进行调度。优先级越高的队列上提交时间越早的作业，越先被调度派遣。

3.1.2 公平共享调度策略

按着先来先服务(FCFS)策略调度派遣作业，在多个用户同时使用系统时，可能会出现以下问题：如果某个用户一次性提交大量的作业，那么集群资源会都被先提交作业的用户占用着，后提交作业的用户只能等着前面人的大量作业都运行完，才能用。用户间使用资源的权利被作业提交时间严重影响，对后提交作业的用户不够公平。公平调度就是为了解决多用户对计算资源的竞争问题，确保集群资源在不同用户、用户组之间公平地分配和使用，避免某些用户或用户组过度占用资源。

3.1.2.1 功能介绍

在队列上应用公平共享策略，为队列上的用户和用户组分配资源份额，以此来代表用户或者用户组的重要性。然后根据用户或者用户组的资源份额、资源使用历史来动态计算用户和用户组当前的优先级。资源使用较少的用户或用户组获得更高的优先

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

级，有更多机会获得资源来运行作业；而资源使用较多的用户或用户组，其优先级会相应降低，以此保证资源分配的公平性。

在调度阶段，我们会根据队列优先级，逐个队列的进行作业调度。对应用了公平共享调度策略的队列，选取队列上优先级最高的用户最早提交的作业来调度，如果用户的动态优先级相同，那么选择最早提交的作业，作业调度成功后，更新用户资源占用信息，应用动态优先级计算公式降低该用户及其父辈用户组的优先级，然后继续选取当前队列上当前优先级最高的用户的作业进行调度，直到队列上的 PEND 作业都被考虑了一遍。

系统重启后，volclava 只会为尚且需要保留在系统内存中的作业，进行公平共享调度策略信息的恢复。因此那些曾经提交过作业但是现在在内存中已经没有作业的 user，将不会回溯回来，以此保持公平共享调度策略信息的精简。

3.1.2.2 配置说明

1. 在队列上添加基于用户的公平共享策略的配置，为队列上的用户或用户组分配资源份额，USER_SHARES 字段内可以定义多个用户或者用户组的资源份额，如下：

```
Begin Queue
...
FAIRSHARE=USER_SHARES[[user, num_shares][...]...]
...
End Queue
```

[user, num_shares]语法含义：

user: 指定当前队列中分配资源份额的用户或者用户组。有如下几种形式：

- 分配给单个用户（指定用户名）；
- 分配给用户组（指定用户组名）；
- 分配给某个组内的每个用户，指定 “group_name@”，如果组内用户和其他 usershare 重复配置了，那么 group_name@配置优先；如果使用@符号为某个组的成员分别分配资源份额，那么该组内部就不能再进行进一步的分层公平共享。资源份额将递归分配给所有子组的所有成员，而不考虑 lsb.users 文件中定义的进一步资源份额分配情况。
- 分配给未包含在其他任何资源份额分配中的用户，可按个体分配（指定关键字 “default”），也可集体分配（指定关键字 “others”）；

num_shares: 指定一个正整数，该整数表示分配给用户的集群资源份额数量。只

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

有当将分配给每个用户的资源份额数量与分配给其他用户的份额数量进行比较，或者与总份额数量进行比较时，这个数量才有意义，它反应了用户或者用户组的重要性，比如 FAIRSHARE=USER_SHARES[[user1, 2][default, 1]]，意味着 user1 比其他任何用户都重要 2 倍。资源份额将被用在计算用户动态优先级的公式中。

2. 分层公平共享配置：

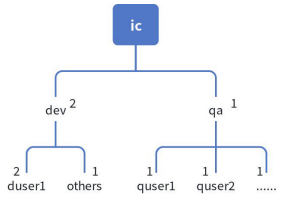
要定义分层公平共享策略，通常需在 lsb.queues 文件中配置顶层资源份额分配。然后，对于受公平共享策略影响的任何用户组，在 lsb.users 文件中 UserGroup 部分配置资源份额数。这能明确资源份额如何在各个用户或子组之间进行分配。

你可以选择为部分用户组定义分层资源份额，而不为其他组定义。如果你没有为组或子组定义资源份额，该组成员以及所有子组成员将根据先到先得（FCFS）策略竞争资源。如下 lsb.users 文件中 UserGroup 定义：

```
Begin Queue
QUEUE_NAME=q1
FAIRSHARE=USER_SHARES[[ic, 1]]
...
End Queue

Begin UserGroup
GROUP_NAME       GROUP_MEMBER               USER_SHARES
dev     (all)    ([duser1, 2][others, 1])
qa      (quser1 quser2)    ([default, 1])
ic      (dev qa)   ([dev, 2] [qa, 1])
End UserGroup
```

以下是用户组资源份额树形结构示意图：



修改配置后，需要运行 badmin mbdrestart 使之生效。默认不在队列上应用公平共享策略。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

3.1.2.3 动态优先级计算

根据资源份额的分配方式，为单个用户或用户组计算动态的用户优先级。这种优先级是动态的，因为公式中的任何变量一旦发生变化，优先级就会随之改变。默认情况下，用户的动态优先级会在作业启动后逐渐降低，而在作业完成时会立即提高。默认情况下，基于以下标准为每个用户计算动态优先级：

1. 分配给用户的资源份额数量
2. 该用户作业所使用的资源情况，具体如下：
    - 已预留和正在使用的 slots 数量
    - 正在运行作业的运行时长（run_time）
    - 累计实际 CPU 使用时间（未归一化），此时间经过调整，近期使用的 CPU 时间比很久以前使用的 CPU 时间权重更大。

公式如下：

动态优先级=number_shares/(cpu_time*CPU_TIME_FACTOR + run_time*RUN_TIME_FACTOR + (1+job_slots)*RUN_JOB_FACTOR)

number_shares: 配置中分配给用户或者用户组的资源份额

cpu_time: 用户累计使用的 CPU 时间（以小时为单位计量）。计算累计 CPU 时间时，会使用实际的（未归一化的）CPU 时间以及一个衰减因子。具体而言，在 lsb.params 文件中由 HIST_HOURS 指定的一段时间间隔（默认值为 5 小时）之后，作业最近使用的 1 小时 CPU 时间会衰减为 0.1 小时。cpu_time 的值是已经结束作业和 running 作业衰减后的 cputime 和当前未到衰减周期的 cputime 之和。

run_time: 已经开始运行的作业的有效运行时间之和（小时为单位）。

job_slots: 该用户或者用户组正在占用的作业 slots 之和。

以下是 lsb.params 中定义的影响动态优先级计算的影响因子：

- CPU_TIME_FACTOR=number，默认值 0.7。此参数也可以在 lsb.queues 文件中针对单个队列进行设置。如果进行了定义，队列设置的值将优先使用。
- RUN_TIME_FACTOR=number，默认值 0.7。 此参数也可以在 lsb.queues 文件中针对单个队列进行设置。如果进行了定义，队列设置的值将优先使用。
- RUN_JOB_FACTOR=number，默认值 3.0。此参数也可以在 lsb.queues 文件中针对单个队列进行设置。如果进行了定义，队列设置的值将优先使用。
- HIST_HOURS=hours，默认值 5。此参数仅适用于公平共享调度策略，它用于确定累计 CPU 时间、运行时间和历史运行时间的衰减率。为了计算用户的动态

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

优先级，调度器会使用一个衰减因子对实际 CPU 时间和运行时间进行缩放。也就是说，经过指定的小时数后，最近使用的 1 小时时间相当于 0.1 小时。在结合衰减后的运行时间和历史运行时间来计算用户动态优先级时，调度器会使用相同的衰减因子对已完成作业的累计运行时间和正在运行作业的运行时间进行缩放。同样，经过指定的小时数后，最近使用的 1 小时时间相当于 0.1 小时。当 HIST_HOURS 设置为 0 时，正在运行作业所累计的 CPU 时间和运行时间不会衰减。此参数也可以在 lsb.queues 文件中针对单个队列进行设置。若有设置，队列中的该参数值将优先使用。

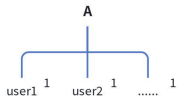
3.1.2.4 几种常用的公平共享调度策略配置

- **第一种经典方式：平铺的资源份额分配方式**

这种配置方式中，资源份额分配只有一层。当用户组资源份额分配中配置了 default，系统将只为活动的用户创建用户资源份额数据结构，活动用户是指当前系统中记录上有过作业提交的用户。当很多用户组中配置了 all 的情况下，这个设计不仅能节省内存空间，还能简化公平调度策略相关的数据结构，使操作这些数据结构花费更少的时间，如下：

```
Begin Queue
QUEUE_NAME   = A
PRIORITY     = 30
NICE         = 20
FAIRSHARE=USER_SHARES[[default,1]]
USER         = all
End Queue
```

以下是该配置中，队列上用户的资源份额树形结构示意图：



系统初始时，队列上显示的公平共享策略信息：

```
[root@master-test logdir]# bqueues -r A

QUEUE: A
  -- For normal low priority jobs, running only if hosts are
lightly loaded.

PARAMETERS/STATISTICS
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND   RUN
SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     0     0     0
0     0    0
Interval for a host to accept two jobs is 0 seconds

SCHEDULING PARAMETERS
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

SCHEDULING POLICIES: FAIRSAHRE
USER_SHARES:  [default,1]

USERS:  all
HOSTS:  all
```

有作业时，**alive user 自动添加，如下命令输出所示：**

```
[root@master-test logdir]# bqueues -r A

QUEUE: A
  -- For normal low priority jobs, running only if hosts are
lightly loaded.

PARAMETERS/STATISTICS
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND   RUN
SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     3     0     3
0     0    0
Interval for a host to accept two jobs is 0 seconds

SCHEDULING PARAMETERS
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

SCHEDULING POLICIES: FAIRSAHRE
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
USER_SHARES:  [default,1]

SHARE_INFO_FOR: A/
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 user1               1     0.166        1     54.0        70
 user2               1     0.111        2     49.0        60

USERS:  all
HOSTS:  all
```

- **第二种经典方式：分层资源份额分配方式**

实例

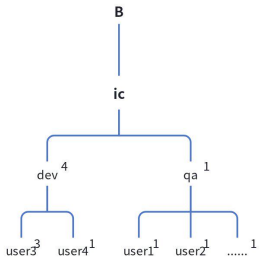
配置文件 lsb.queues 中：

```
Begin Queue
QUEUE_NAME=B
FAIRSHARE=USER_SHARES[[ic, 1]]
...
End Queue
```

配置文件 lsb.users 中：

```
Begin UserGroup
GROUP_NAME       GROUP_MEMBER              USER_SHARES
dev              (user3 user4)             ([user3, 3][user4, 1])
qa               (user1 user2)             ([default, 1])
ic               (dev qa)                  ([dev,4] [qa,1])
End UserGroup
```

以下是该配置中，队列上用户的资源份额树形结构示意图：



以下是该配置下，使用 **bqueues -r** 命令分层显示队列 B 上配置的公平共享策略中用户的实时优先级详情：

```
[root@master-test logdir]# bqueues -r B

QUEUE: B
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
  -- For normal low priority jobs, running only if hosts are
lightly loaded.

PARAMETERS/STATISTICS
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND   RUN
SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     0     0     0
0     0    0
Interval for a host to accept two jobs is 0 seconds

SCHEDULING PARAMETERS
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

SCHEDULING POLICIES: FAIRSAHRE
USER_SHARES:  [ic,1]

SHARE_INFO_FOR: B/
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 ic                  1     0.333        0       0.0        0

SHARE_INFO_FOR: B/ic
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 dev                 4     1.333        0      0.0         0
 qa                  1     0.333        0      0.0         0

SHARE_INFO_FOR: B/ic/dev
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 user3               3     1.000        0      0.0         0
 user4               1     0.333        0      0.0         0

SHARE_INFO_FOR: B/ic/qa
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 user1               1     0.333        0      0.0         0
 user2               1     0.333        0      0.0         0

USERS:  all
HOSTS:  all
```

- **第三种配置：节点是用户组，用户组成员共享份额，成员间按着 FCFS 来选择作**

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

业。

配置样例

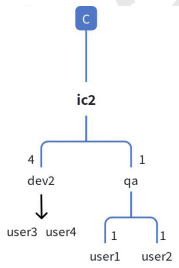
配置文件 lsb.queues 中：

```
Begin Queue
QUEUE_NAME=C
FAIRSHARE=USER_SHARES[[ic2, 1]]
...
End Queue
```

配置文件 lsb.users 中：

```
Begin UserGroup
GROUP_NAME       GROUP_MEMBER              USER_SHARES
dev2             (user3 user4)             ()
qa               (user1 user2)             ([default, 1])
ic2              (dev2 qa)                 ([dev2,4] [qa,1])
End UserGroup
```

以下是该配置中，队列上用户的资源份额树形结构示意图：



以下是该配置下，使用 **bqueues -r** 命令分层显示队列 C 上配置的公平共享策略中用户的实时优先级详情：

```
[root@master-test logdir]# bqueues -r C

QUEUE: C
  -- For normal low priority jobs, running only if hosts are
lightly loaded.

PARAMETERS/STATISTICS
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND   RUN
SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     0     0     0
0     0    0
Interval for a host to accept two jobs is 0 seconds

SCHEDULING PARAMETERS
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

SCHEDULING POLICIES: FAIRSAHRE
USER_SHARES:  [ic2,1]

SHARE_INFO_FOR: C/
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 ic2                 1     0.333        0       0.0        0

SHARE_INFO_FOR: C/ic2
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 dev2                4     1.333        0      0.0         0
 qa                  1     0.333        0      0.0         0

SHARE_INFO_FOR: C/ic2/qa
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 user1               1     0.333        0      0.0         0
 user2               1     0.333        0      0.0         0

USERS:  all
HOSTS:  all
```

- **一些关键字的使用**

default:

```
配置例子
配置文件 lsb.queues 中：
Begin Queue
QUEUE_NAME=xx
FAIRSHARE=USER_SHARES[[jenkins_user,100][default, 1]]
...
End Queue
```

others:

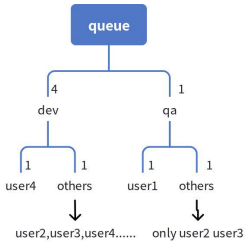
```
配置例子
配置文件 lsb.queues 中：
Begin Queue
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
QUEUE_NAME=xx
FAIRSHARE=USER_SHARES[[dev,4] [qa,1]]
...
End Queue

配置文件 lsb.users 中：
Begin UserGroup
GROUP_NAME       GROUP_MEMBER              USER_SHARES
dev              (all)                     ([user4,1][others,1])
qa               (user1 user2 user3)       ([user1,1][others,1])
End UserGroup
```

以下是该配置中，队列上用户的资源份额树形结构示意图：

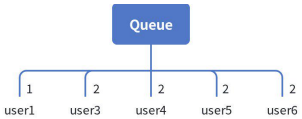


group@：

```
配置例子
配置文件 lsb.queues 中：
Begin Queue
QUEUE_NAME=xx
FAIRSHARE=USER_SHARES[[dev@,2][user1, 1]]
...
End Queue

配置文件 lsb.users 中：
Begin UserGroup
GROUP_NAME       GROUP_MEMBER              USER_SHARES
dev1             (user5 user6)             ([default,1])
dev              (user3 user4 dev1)        ([user4,1][others,1])
qa               (user1 user2)             ([default,1])
End UserGroup
```

以下是该配置中，队列上用户的资源份额树形结构示意图：



Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

3.1.2.5 信息展示

可使用如下命令进行公平共享调度策略信息的相关查询。

- 展示队列上的公平共享配置和负载使用情况：bqueues -l/-r

bqueues -l 会展示顶层的公平共享配置和负载使用情况，如下所示：

```
[root@master-test logdir]# bqueues -l B

QUEUE: B
  -- For normal low priority jobs, running only if hosts are
lightly loaded.

PARAMETERS/STATISTICS
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND   RUN
SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     0     0     0
0     0    0
Interval for a host to accept two jobs is 0 seconds

SCHEDULING PARAMETERS
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

SCHEDULING POLICIES: FAIRSAHRE
USER_SHARES:  [ic,1]

SHARE_INFO_FOR: B/
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 ic                  1     0.333        0       0.0        0

USERS:  all
HOSTS:  all
```

bqueues -r 递归展示各层公平共享配置和负载使用情况，如下所示：

```
[root@master-test logdir]# bqueues -r B
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
QUEUE: B
  -- For normal low priority jobs, running only if hosts are
lightly loaded.

PARAMETERS/STATISTICS
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND   RUN
SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     0     0     0
0     0    0
Interval for a host to accept two jobs is 0 seconds

SCHEDULING PARAMETERS
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

SCHEDULING POLICIES: FAIRSAHRE
USER_SHARES:  [ic,1]

SHARE_INFO_FOR: B/
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 ic                  1     0.333        0       0.0        0

SHARE_INFO_FOR: B/ic
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 dev                 4     1.333        0      0.0         0
 qa                  1     0.333        0      0.0         0

SHARE_INFO_FOR: B/ic/dev
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 user3               3     1.000        0      0.0         0
 user4               1     0.333        0      0.0         0

SHARE_INFO_FOR: B/ic/qa
USER/GROUP      SHARES  PRIORITY  STARTED  CPU_TIME RUN_TIME
 user1               1     0.333        0      0.0         0
 user2               1     0.333        0      0.0         0

USERS:  all
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
HOSTS:  all
```

- 展示用户组配置的资源份额信息：bugroup -l

```
[root@master-test logdir]# bugroup -l
GROUP_NAME:  dev2
USERS:       user4 user3

GROUP_NAME:  dev
USERS:       user4 user3
SHARES:      [user3,3][user4,1]

GROUP_NAME:  qa
USERS:       user1 user2
SHARES:      [default,1][user1,1][user2,1]

GROUP_NAME:  ic
USERS:       dev/ qa/
SHARES:      [dev, 4][qa,1]

GROUP_NAME:  ic2
USERS:       dev2/ qa/
SHARES:      [dev2, 4] [qa, 1]
```

- 展示作业关联的公平共享信息：使用 bjobs -l 展示已运行作业的公平共享信息，作业结束后将不再有公平共享信息。

```
[root@master-test logdir]# bjobs -l 4

Job<4>, User<user3>, Project<default>,Status <RUN>,Queue <B>, Job
Priority <50>, Command <sleep 100>, Share group charged
</B/ic/dev/user3>
Sun Jun 8 15:40:14: Submitted from host <master-test>,CWD
</opt/volclava_test/etc>;
Sun Jun 8 15:40:17: Started on <cmp1-test>, Execution Home
</home/user3>, Execution CWD </tmp>;
Sun Jun 8 15:40:22:Resource usage collected.
                   MEM: 7 Mbytes;  SWAP: 141 Mbytes
                   PGID: 30291;    PIDs: 30291

 MEMORY USAGE:
 MAX MEM:7Mbytes; AVG MEM:7Mbytes
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
 SCHEDULING PARAMETERS:
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

 RESOURCE REQUIREMENT DETAILS:
 Combined: select[type == local] order[r15s:pg]
 Effective: select[type == local] order[r15s:pg]
```

- 展示影响公平共享动态优先级因素：bparams -l

```
Run time weighting factor for fairshare scheduling policy:
    RUN_TIME_FACTOR = 0.700000

CPU time weighting factor for fairshare scheduling policy:
    CPU_TIME_FACTOR = 0.700000

Job slots weighting factor for fairshare scheduling policy:
    RUN_JOB_FACTOR = 3.000000

The number of hours of resource consumption history::
    HIST_HOURS = 3.000000
```

3.1.2.6 相关日志

类别	LC_FAIR	
级别	LOG_DEBUG	打印偶发关键操作信息，比如在公平共享结构中添加新 user，公平共享结构初始配置信息
	LOG_DEBUG1	打印作业级别操作，比如调度算法中对作业的操作、作业负载更新
	LOG_DEBUG2	会展示公平共享结构优先级计算细节，开发者细粒度分析使用

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

	LOG_DEBUG3	开发测试使用，观察调度每个作业时公平共享结构的变化，在**生产环境中请不要开启，严重影响调度性能**

3.1.3 用户指定的作业优先级

用户指定的作业优先级在一个队列中提供了允许用户排列其作业顺序的控制。要确定所派发的作业，首先考虑作业顺序。 有相同优先级的作业按先来先服务的策略进行排序。 作业拥有者可以改变他们自己作业的优先级，volclava 和队列管理员可以改变队列中所有作业的优先级。

在集群中，用户指定作业优先级为所有的队列开启，并且可以配置为作业优先级自动增加。这样就可以自动增加已经挂起了指定时间长的作业的优先级。

1. **配置作业优先级**:

   - 要配置用户指定的作业优先级，编辑 lsb.params 并定义 MAX_USER_PRIORITY。
   - 在 cluster 中，这个配置适用于所有的队列。用 bparams -l 来显示 MAX_USER_PRIORITY 的值

   语法:

   MAX_USER_PRIORITY=max_priority

   其中：max_priority 规定了一个用户可以给作业指定的最大优先级。 合法的值为正整数。值越大代表优先级越高；1 为最小。 volclava 及队列管理员能在超出 max_priority 的范围外指定优先级。

   例如:

   MAX_USER_PRIORITY=100

   规定了用户能够指定的最大作业优先级为 100

2. **指定作业优先级：**

   在使用 bsub 提交作业时指定作业优先级，在提交后用 bmod 修改作业优先级。一个没有指定优先级而提交的作业，默认优先级为 MAX_USER_PRIORITY/2。

   - bsub -sp priority JOB_ID
   - bmod [-sp priority | -spn] JOB_ID

   -sp priority 规定了该作业的优先级。优先级的合法值为任何在 1 到

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

MAX_USER_PRIORITY（由 bparams -l 显示）之间的正整数。不正确的作业优先级会被拒绝。 volclava 和队列管理员可以在超过 MAX_USER_PRIORITY 的范围指定优先级。

-spn 把作业优先级设置为默认值为 MAX_USER_PRIORITY/2（由 bparams -l 显示）。

3. **查看作业优先级信息**
   - bhist -l job_id
   - bjobs -l job_id
   - bparams -l

3.2 派发及运行窗口

3.2.1 派发及运行窗口

派发和运行窗口两者都是控制 volclava 作业何时启动与运行的时间窗口。

- 派发窗口可以在 lsb.hosts 中定义。派发和运行窗口在 lsb.queues 中定义
- 节点可以仅有派发窗口。队列可以有派发窗口和运行窗口
- 两个窗口都会影响作业的启动；仅有运行窗口会影响作业的停止
- 派发窗口定义何时节点及队列是激活或未激活的。它不能控制作业的提交。
- 运行窗口定义何时作业是否能够运行。当一个运行窗口关闭时，volclava 不能启动放置在该队列中的任何作业，也不能结束任何已经开始运行的作业。
- 当一个派发窗口关闭时，运行的作业将继续运行到结束，且新作业不能派发给节点或从该队列中派发。 当一个运行窗口关闭时，volclava 挂起运行的作业，但是新作业仍然能提交到该队列

3.2.2 运行窗口

1. **配置运行窗口**：

   要配置一个运行窗口，在 lsb.queues 中设置 RUN_WINDOW。

   例如：要想指定一个将从早上 `4：30` 到中午打开的运行窗口

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
Begin Queue
...
RUN_WINDOW = 4:30-12:00
...
End Queue
```

您可以指定多个时间窗口。用 **bqueues -l** 来显示关于运行窗口队列信息。

3.2.3 派发窗口

1. **配置节点派发窗口**

给一个节点配置派发窗口，在 lsb.hosts 中设定 DISPATCH_WINDOW，且指定一个或多个时间窗口。如果不配置节点派发窗口，则该窗口一直打开。

2. **配置队列派发窗口**

给一个队列配置派发窗口，在 lsb.queues 中设置 DISPATCH_WINDOW 并且指定一个或者多个时间窗口。如果没有配置队列派发窗口，则该窗口一直打开。

- 使用 **bqueues -l** 显示队列派发窗口
- 使用 **bhosts -l** 显示节点派发窗

3.3 作业重队列和作业重运行

3.3.1 关于作业队列重排

网络计算环境很容易受到任何故障、网络服务或处理器资源的临时状态的影响。例如，你可能会得到 NFS 陈旧句柄错误，磁盘已满错误，进程表已满错误， 或网络连接问题。你的应用程序也可能受外部条件的影响，如软件许可的问题，或者偶尔由你的应用程序中的 bug 引起的故障。

这种错误是暂时的，时有时无，或在某个机器上发生而在另一个上并不发生。 当你所有的作业由于这种临时错误而退出，并且直到 12 小时后你才明白其中原因，你可能会感到不安。

volclava 提供了一种自动从临时错误恢复的方法。你可以配置特定退出值，例如，如果一个作业退出值是其中一个，这个作业将被自动队列，就像它还没有被分发一样。稍后这个作业会被再次运行。你也可以配置你的队列使得一个重排的作业不会被调度到先前运行失败的机器上。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1. **配置作业自动重队列**

   在队列定义 REQUEUE_EXIT_valueS（lsb.queues），并指定导致作业重队列的退出代码。

```
Begin  Queue
REQUEUE_EXIT_valueS = 99 100
End Queue
```

   这个配置使退出值是 99 或 100 的作业重队列。

2. **控制作业重队列次数**

   默认情况下，如果一个作业失败并且其退出值在 REQUEUE_EXIT_valueS 中，volclava 自动重队列作业。重复失败的作业默认最多重队列 5 次，限制失败作业重队列的次数。在集群范围中设置 MAX_JOB_REQUEUE （lsb.params），在队列定义中（lsb.queues）也可以定义。

3. **排斥的作业重队列**

   你可以配置自动作业重队列，这样一个失败的作业不会重新运行在相同的机器上。

4. **限制**

   如果 mbatchd 被重新启动，此功能可能无法正常工作，因为 volclava 忘记哪些主机已被排除。如果作业运行在某个机器上并且在 mbatchd 重新启动前以一个排斥的退出代码退出，这个作业可能会在 mbatchd 重新启动后被分发到同一台机器上。

5. **配置排斥的作业队列**

   在队列定义中设置 REQUEUE_EXIT_valueS（lsb.queues）并使用括号和关键字 EXCLUDE 定义退出代码。

   示例：

```
Begin  Queue
REQUEUE_EXIT_valueS=30 EXCLUDE(20)
HOSTS=hostA hostB hostC
End Queue
```

   这个队列中的作业可以被分发到 hostA，hostB 或 hostC 上。

   如果一个运行在 hostA 上的作业的退出值是 30 并且被重队列，这个作业可以被分发到 hostA，hostB，或 hostC 上。但是，如果一个运行在 hostA 上的作业的退出值是 20 并且被重队列，它只能被分发到 hostB 或 hostC 上。

   如果这个作业运行在 hostB 上并且再次以退出值 20 退出，它只能被分发到 hostC 上。最后，如果这个作业运行在 hostC 上并以退出值 20 退出，它就不能再被分发到

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

任何机器上了，所以它将一直等待下去。

6. **重队列一个作业**

   使用 brequeue 重队列作业。

   - 只能对运行（RUN）、用户挂起（USUSP） 或系统挂起（ SSUSP ）作业使用 brequeue。
   - 用户只能重队列他们自己的作业；只有 root 用户和 volclava 管理员可以重队列由其他用户提交的作业。
   - 你不能对交互式批处理作业使用 brequeue。

3.4 作业组

volclava 提供一种叫做作业组的结构。它允许一系列的作业共享相同的执行体和资源请求但拥有不同的输入文件，并作为一个单一单元被提交、控制和监视。 使用标准 volclava 命令，你也能控制、监视单个作业和从一个作业组提交的作业的组。 在作业组被提交后，volclava 独立地调度和分发每个单个作业。每个从作业组提交的作业共享相同的、作业组的作业 ID，单个作业可以通过组索引被单独引用。 作业组的维度和结构在作业组创建的时候被定义。

3.4.1 创建作业组

在作业提交时使用 bsub -j 选项创建作业组。

例如，下面的命令创建一个由 1000 个作业组成的名为“myArray”的作业组。

```
bsub -J "myArray[1-1000]" myJOB
```

1. **语法：**

   下面的 bsub 语法被用于创建一个作业组：

```
bsub -J "arrayName[indexList, ...]" myJOB
```

其中： -J "arrayName[indexList, ...] 命名并创建作业组。indexList 两边的方括号[]必须像上面那样使用并且作业组名字规范必须放在引号中。逗号（,）被用于分开多个 indexList。这个规范的最大长度是 255 个字符。

- arrayName : 用户指定的字符串，用于识别作业组。以下字符的任意组合都是

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

有效的：

```
a-z | A-Z | 0-9 | . | - | _
```

- indexList = start[-end[:step]]：指定作业组的大小和维度。其中：
  - start：指定索引范围的开始值。也能被用于指定单个的索引。有效值是唯一的正整数。例如，[1-5]和[1， 2， 3， 4， 5]都指定了索引为 1 到 5 的 5 个作业。
  - end：指定索引范围的结束值。有效值是唯一的正整数。
  - step：指定索引在范围内的步长。索引从 start 值开始，以 step 值为增量，并且大小不超过 end 值。默认值是 1。有效值是正整数。例如，[1-10:2]指定了 1 －10 的索引范围，步长是 2，即索引是 1、3、5、7 和 9。

在作业组被创建后（提交后），可以使用作业组名或作业 ID 以及索引值引用每个单独的作业。例如，下面两个作业组语句系列都引用了从名为“myArray”的作业组提交的作业，这个作业组由 1000 个作业组成并且作业 ID 是 123，可以使用下面两种方法来引用单独的作业：

```
myArray[1], myArray[2], myArray[3], ..., myArray[1000]
123[1], 123[2], 123[3], ..., 123[1000]
```

3.4.2 处理输入、输入文件

1. **重定向标准输入**

当执行体从标准输入读时，使用 bsub 的-i 选项和%I 变量。 为了使用%I，所有输入文件必须被命名一致并有一个对应组索引的变量部分。

例如： input.1， input.2， input.3， ..., input.N

下面的命令提交一个包含 1000 个作业的作业组，它的输入文件被命名为 input.1，input.2， input.3， ...， input.1000，位于当前工作目录：

```
bsub -J "myArray[1-1000]" -i "input.%I" myJOB
```

2. **重定向标准输入和错误输出**

当执行体向标准输出和错误输出写时，使用 bsub 的-o 选项和%I、%J 变量。

为了为每个从作业组提交的作业创建一个输出文件，需要指定%I 为输出文件名的一部分。 例如，下面的命令提交一个包含 1000 个作业的作业组，它的输出文件被命

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

名为 output.1， output.2，output.3， ...， output.1000，位于当前工作目录：

```
bsub -J "myArray[1-1000]" -o "output.%I" myJOB
```

为创建作业组 ID 是文件名一部分的输出文件，需要指定%J。 例如，下面的命令提交一个包含 1000 个作业的作业组，它的输出文件被命名为 output.123.1，output.123.2， output.123.3， ...， output.123.1000，位于当前工作目录，作业组的作业 ID 是 123：

```
bsub -J "myArray[1-1000]" -o "output.%J.%I" myJOB
```

3.4.3 查看作业组

1. **显示作业组状态**

为了显示从作业组提交的当前运行作业的统计信息，使用 bjobs 的–A 选项。 例如，一个作业 ID 是 123 的包含 10 个作业的作业组：

```
bjobs -A 123
```

单个作业的状态，查看作业 ID 是 123 的作业组：

```
bjobs 123
```

显示特定作业的当前状态，作业 ID 为 123 的作业组的第五个作业的状态

```
bjobs "123[5]"
```

3.4.4 控制作业组

1. **控制整个组**

为了杀死作业 ID 为 123 的作业组的第五个作业 **bkill 123。**

2. **控制单个作业**

为了杀死作业 ID 为 123 的作业组的第五个作业 **bkill "123[5]"。**

3. **控制作业群**

为了杀死作业 ID 为 123 的作业组的第 1－5，239 和 487 号作业，**bkill "123[1-5，**

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

239， 487]"。

3.4.5 重新排队作业组

1. **重队列 DONE 状态作业**

使用 brequeue 的-d 选项重队列 DONE 状态作业。 例如，**brequeue -J "myarray[1-10]" -d 123** 命令重队列作业 ID 为 123、DONE 状态的作业。

2. **重队列 EXIT 状态作业**

使用 brequeue 的-e 选项重队列 DONE 状态作业。 例如，**brequeue -J "myarray[1-10]" -e 123** 命令重队列作业 ID 为 123、EXIT 状态的作业。

3. **重队列作业组中所有状态的作业**

一个已提交的作业组包含各种状态的作业。使用 brequeue 的-a 选项重队列作业组中所有状态的作业。 例如，**brequeue -J "myarray[1-10]" -a** 命令重队列作业 ID 为 123 的作业组中所有状态的作业。

4. **重队列 RUN 作业到 PSUSP 状态**

使用 brequeue 的-H 选项重队列 RUN 作业到 PSUSP 状态。 例如，**brequeue -J "myarray[1-10]" -H 123** 命令重队列作业 ID 为 123 的 RUN 作业到 PSUSP 。

5. **重队列 RUN 状态作业**

使用 brequeue 的-r 选项重队列 RUN 状态作业例如，**brequeue -J "myarray[1-10]" -r 123** 命令重队列作业 ID 为 123、RUN 状态的作业。

3.4.6 作业组的作业槽限

作业组的作业槽限制被用于指定从作业组提交的作业的最大同时运行数。作业组允许在一个命令中提交大量的作业，可能潜在地使系统崩溃。作业槽限制提供了一种限制作业组对系统影响的方法。作业组的作业槽限制使用下面的语法指定:

```
bsub -J "JOB_array_name[index_List]%JOB_slot_Limit" myJOB
```

其中： %JOB_slot_Limit 指定了最大同时运行数。这个百分号（%）必须被正确的使用。有效值是小于作业组的最大索引值的正整数。

1. **在提交时设置作业组槽限制**

使用 bsub 命令在提交时设置作业槽限制为包含 1000 个作业的作业组设置一个

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

100 个作业的作业组的作业槽限制，例如：

```
bsub -J "JOB_array_name[1000]%100" myJOB
```

2. **在提交后设置作业组槽限制**

使用 bmod 命令在提交后设置作业槽限制例如，为作业 ID 为 123 的组设置一个 100 个作业的作业组的作业槽限制，例如：

```
bmod -J "%100" 123
```

3. **改变作业组的作业槽限制**

改变作业组的作业槽限制与提交后设置是一样的。 1. 使用 bmod 命令在提交后改变作业槽限制。 例如，改变作业 ID 为 123 的作业组的作业槽限制为 250，例如：

```
bmod -J "%250" 123
```

4. **查看作业组的作业槽限制**

使用 bjobs 的-A 和-l 选项查看作业组的作业槽限制。

3.5 运行并行作业

3.5.1 提交并行作业

当提交请求多个处理的并行作业时，你能指定需要的处理器的确切数。 使用 bsub –n 并指定作业需要的处理器数。 例如：

```
bsub -n 4 myJOB
```

提交 myJOB 为并行作业。当有四个可用作业槽时，这个作业被启动。

3.5.2 并行作业的作业槽限制

作业槽是 volclava 中处理器分配的基本单元。一个串行作业使用一个作业槽。一个具有 N 个组件（作业）的并行作业使用 N 个作业槽，这些作业槽可能跨多个机器。

默认配置中，运行的、挂起的作业占用了相关队列、用户、机器和处理器的作业槽限制。 如果使用了处理器预留，被 pending 作业预留的作业槽也占作业槽限制。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

3.5.3 指定处理器的最小、最大数

默认的，当调度一个并行作业时，在每个机器上分配的槽数将不超过机器的 CPU 数，即使机器 MXJ 被设置得比 CPU 数大。当提交一个并行作业时，你也能指定最小、最大处理器数。

如果你指定了最小、最大处理器数，当最小处理器数满足时作业启动，但是作业最多使用最大处理器数，实际使用情况依赖于当前可用处理器数。一旦作业开始运行，即使稍后有更多可用处理器也不会为作业分配额外的处理器。

如果作业请求的处理器数小于作业提交到的队列中定义的最小 PROCLIMIT，或者大于最大 PROCLIMIT，请求将被拒绝。所以请注意在作业请求最小、最大处理器时，最大请求数不能小于最小 PROCLIMIT，并且最小请求数不能大于最大 PROCLIMIT。

如果一个自动可变大小的作业在一个 2CPU、4 槽的机器上请求 1 到 4 个槽，这个作业最后将使用到 4 个槽。例如：

```
bsub -n 4,16 myJOB
```

最多可以分配 16 个处理器给这个作业。如果可运行作业的处理器少于 16 个， 只要可用处理器数大于等于 4 个时这个作业就能被启动。

3.5.4 控制跨机器的处理器分配

有时你可能需要控制并行作业的处理器选择，可以指定如何在集群中跨机器为其选择处理器。 你能在作业级或者队列级控制它。作业级的配置覆盖队列级的配置。

1. **在作业级指定并行作业运行位置**

volclava 默认从可用的处理器集中为作业分配请求的处理器。 一个并行作业可能跨多个机器，其指定每个机器分配的处理器数。一个作业可能被调度到一个多处理器的机器上以充分利用这台机器上有效的共享内存，或者被调度到多个机器以利用他们的内存和交换空间。灵活的跨机器也能被用于完成并行 I/O。通过以资源请求串方式使用 span 部分（bsub –R 或者 lsb.queues 的队列定义中 RES_REQ），你能指定“在同一机器上为并行批处理作业选择所有的处理器”，或者“不要在同一机器上选择超过 n 个处理器” 。

2. **语法**

span 串支持以下语法：

span[hosts=1] 表示所有分配给这个作业的处理器必须都来自同一个机器。

span[ptile=value] 表示每个机器应该分配给这个作业的处理器数。其中 value 是以

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

下中的一个：¨

- 缺省 ptile 值，由 n 处理器指定。在下面的例子中，作业从每个可用机器上请求 4 个处理器，不管机器有几个处理器：span[ptile=4]
- 预定义 ptile 值，由'！'指定。下面的例子使用 lsb.hosts 中预定义的最大作业槽限制（MXJ 每机器类、型）作为 ptile 的值： span[ptile='！']

提示：如果机器或者机器类型没有定义 MXJ，缺省的预定义 ptile 值是 1。

3. **实例**

```
bsub -n 4 -R "span[hosts=1]" myJOB
```

在一个当前至少拥有 4 个可运行该作业的四个组件的处理器的机器上运行作业。

```
bsub -n 4 -R "span[ptile=2]" myJOB
```

在两个机器上运行作业，每个机器使用两个处理器。每个机器可能拥有超过 2 个的可用处理器。

```
bsub -n 4 -R "span[ptile=3]" myJOB
```

在两个机器上运行作业，第一个机器使用 3 个处理器，第二个机器使用 1 个处理器。

```
bsub -n 4 -R "span[ptile=1]" myJOB
```

在四个机器上运行作业，即使这四个机器中的某些当前拥有超过 1 个可用处理器。

3.5.5 限制分配的处理器数

使用 lsb.queues 中的 PROCLIMIT 参数限制能被分配给并行作业的处理器数

1. **语法**

PROCLIMIT = [minimum_Limit [defauLt_Limit]] maximum_Limit

所有的限制必须是大于或等于 1 的正整数并满足下面的关系：

1 <= minimum <= defauLt <= maximum

你最多能在 PROCLIMIT 参数指定 3 个限制：

如果你指定...	那么...

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

一个限制	指定的是最大处理器限制,最小和缺省限制被置为 1
两个限制	第一个是最小处理器限制，第二个是最大处理器限制。缺省限制被置为与最小一样。最小限制必须小于或者等于最大限制
三个限制	第一个是最小处理器限制，第二个是缺省处理器限制,并且第三个是最大处理器限制。最小值必须小于缺省值和最大值

2. **PROCLIMIT 如何影响并行作业的提交**

bsub 的-n 选项指定被并行作业使用的处理器数，受限于队列的处理器限制。指定少于最小 PROCLIMIT 或大于最大 PROCLIMIT 的处理器数的作业将被拒绝。

如果为 PROCLIMIT 指定了一个缺省值，没有使用-n 提交的作业使用缺省值。 如果队列只为 PROCLIMIT 指定了最小和最大值，处理器数等于最小值。如果仅为 PROCLIMIT 指定了最大值或者未指定 PROCLIMIT， 处理器数等于 1。不正确的处理器限制将被忽略，并且在 volclava 被重配置或重启动时显示警告信息。当 volclava 被启动时，警告信息也被写入 mbatchd 日志文件。

3. **修改 PROCLIMIT**

如果你修改类 PROCLIMIT 参数，新的处理器限制不影响正在运行的作业。没有处理器需求的等待作业使用新的默认 PROCLIMIT 值。如果等待作业不满足新的处理器限制，这个等待作业将继续处于 PEND 状态并且等待原因变成： JOB no Longer satisfies PROCLIMIT configuration 。

如果 PROCLIMIT 指定不正确（例如，多余的参数），一个重配置错误信息被发出。重配置发生并且不正确的 PROCLIMIT 被忽略。

4. **大小可变作业**

大小可变作业的资源分配请求服从队列中的 PROCLIMIT 定义。当最大作业槽请求大于 PROCLIMIT 中定义的最大作业槽定义，LSF 选择两者中的最小值。例如，如果一个作业请求-n 1，4，但是 PROCLIMIT 被定义为 2 2 3，这个作业的最大槽请求是 3 而不是 4。

5. **自动队列选择**

当你提交一个没有指定队列名的并行作业，volclava 自动地从队列列表（lsb.params 中的 DEFAULT_QUEUE 参数或者 LSB_DEFAULTQUEUE 环境变量）中选择最合适的队列。自动队列选择考虑所有自动选择可用队列的最大、最小 PROCLIMIT 值。如果你指定了-n 但是没有指定一个队列， 那么第一个满足作业处理器请求的队列被使用。如果没有队列满足处理器请求，作业将被拒绝。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

例如，在 lsb.queues 中定义队列的 PROCLIMIT 值如下：

- queueA with PROCLIMIT=1 1 1
- queueb with PROCLIMIT=2 2 2
- queueC with PROCLIMIT=4 4 4
- queueD with PROCLIMIT=8 8 8
- queueE with PROCLIMIT=16 16 16

在 lsb.params 中：DEFAULT_QUEUE=queueA queueb queueC queueD queue
对于以下作业：

```
bsub -n 8 myJOB
```

volclava 自动选择 queueD 运行 myJOB。

```
bsub -n 5 myJOB
```

作业 myJOB 失败，因为没有缺省的队列拥有正确的处理器数。

6. **实例最大处理器限制**

lsb.queues 中的缺省队列指定 PROCLIMIT 的值为： PROCLIMIT = 3 ，能为队列分配的最大处理器数是 3。

下面是使用例子：

```
bsub -n 2 myjob      作业 myjob 运行在 2 个处理器上
bsub -n 4 myjob      作业 myjob 被队列拒绝，因为它请求超过为队列配置的最大处理器数（3）
bsub -n2,3 myjob     作业 myjob 运行在 2 到 3 个处理器上
bsub -n2,5 myjob     作业 myjob 运行在 2 到 3 个处理器上,取决于机器上当前可用槽数
bsub myjob           没有配置缺省或最小值，所以作业 myjob 运行在一个处理器上
```

7. **最小和最大处理器限制**

lsb.queues 中指定 PROCLIMIT 为： PROCLIMIT = 3 8，能为队列上的作业分配的最小处理器数是 3，能为队列上的作业分配的最大处理器数是 8。

下面是使用例子：

```
bsub -n 5 myjob      作业 myjob 运行在 5 个处理器上
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
bsub -n 2 myjob     作业 myjob 被队列拒绝，因为它请求少于为队列配置的最小处理器数（3）
bsub -n 4,5 myjob   作业 myjob 运行在 4 到 5 个处理器上
bsub -n 2,6 myjob   作业 myjob 运行在 3 到 6 个处理器上
bsub -n 4,9 myjob   作业 myjob 运行在 4 到 8 个处理器上
bsub myjob          缺省处理器数等于最小数（3）。作业 myjob 运行在 3 个处理器上。
```

8. **最小、缺省和最大处理器限制**

   lsb.queues 中指定 PROCLIMIT 为：PROCLIMIT = 4 6 9，能为队列分配的最小处理器数是 4；队列的缺省处理器数是 6；能为队列分配的最大处理器数是 9。

   例子： bsub -n 5 myjob 因为配置了默认值，作业 myjob 运行在 6 个处理器上。

3.6 作业包

3.6.1 功能描述

使用 volclava 作业包可加快大量作业的提交速度，通过提交包含多个作业请求的单个文件来提交作业。必须启用作业包功能（通过在 lsb.conf 中定义 LSB_MAX_PACK_JOBS）才能使用-pack 选项。

将作业分组可以保持性能。当 volclava 在正在处理作业包时，mbatchd 被阻止处理其他请求。限制每个包中的作业数量可确保其他作业提交的合理响应时间。作业包大小是可配置的。

3.6.2 启用并配置作业包

1. **编辑 lsf.conf**
2. **定义参数 LSB_MAX_PACK_JOBS=100**

   执行此操作可启用该功能并设置作业包大小。建议初始包装尺寸为 100 个。如果值为 1，则单独提交文件中的作业，就像直接使用 bsub 命令提交一样。如果值为 0，则禁用作业包。

3. **（可选）定义参数 LSB_PACK_SKIP_ERROR=Y**

   如果希望 LSF 处理作业提交文件中的所有请求，并且即使某些请求有错误也继续

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

执行，请执行此操作。

4. **重新启动 mbatchd 以使更改生效**

   运行 badmin mbdrestart 命令。

3.7 定义最大排队任务数

3.7.1 功能描述

定义 MAX_PEND_JOBS 和 MAX_PEND_SLOTS 配置功能，限制用户错误的提交大量任务在排队，防止用户脚本写错了提交了很多任务，避免系统失控。

定义 MAX_PEND_JOBS 和 MAX_PEND_SLOTS 后，如果提交过程中排队的任务数/slots 数，超过了定义的最大值，则拦截提交请求：

- 对于 bsub 会触发自动重试；
- 对于 bsub -pack 会直接返回提交失败。

简单示例效果：

```
[root@volclava-master ~]$ bsub -R"select[mem=1024000]" sleep 1
sleep 1
User <root>: Pending job threshold reached. Retrying in 60
seconds...
User <root>: Pending job threshold reached. Retrying in 60
seconds...
User <root>: Pending job threshold reached. Retrying in 60
seconds...
User <root>: Pending job threshold reached. Retrying in 60
seconds...
User <root>: Pending job threshold reached. Retrying in 60
seconds...
User <root>: Pending job threshold reached. Retrying in 60
seconds..
User permission denied. Job not submitted.
```

3.7.2 配置定义

新增配置：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1. 在 lsb.users 中定义 MAX_PEND_JOBS 或 MAX_PEND_SLOTS（整型），可以对某个用户/用户组限制排队的任务数/slot 数；

lsb.users 示例：

```
Begin User
USER_NAME        MAX_JOBS        JL/P        MAX_PEND_JOBS
MAX_PEND_SLOTS
dev              20              8           1000              -
default          20              8           1                 -
root             5               -           15                20
test1            5               -           10                10
eng_users        5               -           4
openlava         5               -           20
#develop@        20              8
#support         50              -
End User
```

2. 在 lsb.params 中定义 MAX_PEND_JOBS 或 MAX_PEND_SLOTS（整型），可以对集群整体限制排队的任务数/slot 数；

3. 在 lsb.params 中定义 SUB_TRY_INTERVAL（默认 60，单位：秒），设置自动重试的时间间隔；

lsb.params 示例：

```
Begin Parameters
DEFAULT_QUEUE  = normal   #default job queue name
MBD_SLEEP_TIME = 10       #mbatchd scheduling interval (60 secs is
default)
SBD_SLEEP_TIME = 7        #sbatchd scheduling interval (30 secs is
default)
JOB_ACCEPT_INTERVAL = 0   #interval for any host to accept a job
                          # (default is 1 (one-fold of
MBD_SLEEP_TIME))
SUB_TRY_INTERVAL = 10
MAX_PEND_JOBS = 13
MAX_PEND_SLOTS = 17
End Parameters
```

关联现有配置：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1. 解析环境变量 LSB_NTRIES（默认 6），设置重试次数；

环境变量设置示例：

```
export LSB_NTRIES=3

[root@master-test ~]# cat /opt/volclava-1.0/etc/lsb.params | grep
SUB_TRY_INTERVAL
SUB_TRY_INTERVAL = 10
```

以下是使用例子：

```
[root@n144-252-149 volclava]# export_LSB_NTRIES=3
[root@n144-252-149 volclava]# echo $LSB_NTRIES
3
[root@n144-252-149 volclava]# time bsub -R "select[mem=1024000]" -
J "test[1-11]" sleep 1
User <root>: Pending job threshold reached. Retrying in
10seconds...
User <root>: Pending job threshold reached. Retrying in 10
seconds...
User <root>: Pending job threshold reached. Job not submitted.

real    0m20.014s
user    0m0.006s
sys     0m0.006s
```

3.7.3 功能效果

bsub 单个任务：

定义 MAX_PEND_JOBS 或 MAX_PEND_SLOTS 之后，bsub 提交如果触及最大排队限制，将被拦截并进入自动重试。

```
[root@master-master ~]# bsub -R "select[mem=1024000]" sleep 1
sleep 1
User <root>: Pending job threshold reached. Retrying in 10
seconds...
User <root>: Pending job threshold reached. Retrying in 10
seconds...
User <root>: Pending job threshold reached. Retrying in 10
seconds...
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
User <root>: Pending job threshold reached. Retrying in 10
seconds...
User <root>: Pending job threshold reached. Retrying in 10
seconds...
User <root>: Pending job threshold reached. Retrying in 10
seconds...
User permission denied. Job not submitted.
```

bsub -J：

会在开始提交时就计算产生的 PEND 的任务数，并阻塞批量任务整体的下发。下面是使用事例：

```
[root@master-test ~]# busers -w
USER/GROUP   JL/P    MAX  NJOBS   PEND    RUN  SSUSP  USUSP  RSV
MPEND  PJOBS MPJOBS
root          -        5      0      0      0      0      0    0
20      0     15

[root@master-test ~]# bsub -J "test[1-10]" -R
"select[mem=1024000]" sleep 1
Job <1> is submitted to default queue <normal>.

[root@master-test ~]# bsub -J "test[1-10]" -R
"select[mem=1024000]" sleep 1
User <root>: Pending job threshold reached. Retrying in 10
seconds..
User <root>: Pending job threshold reached. Retrying in 10 seconds
^C

[root@master-test ~]# busers -w
USER/GROUP   JL/P    MAX  NJOBS   PEND    RUN  SSUSP  USUSP  RSV
MPEND  PJOBS MPJOBS
root          -        5     10     10      0      0      0    0
20     10     15
```

bsub -pack：

按行提交，触发 threshold 的作业不再进行阻塞重试，继续尝试下一行作业的提交。下面是使用事例：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
[root@master-test ~]# busers -w
USER/GROUP   JL/P    MAX  NJOBS   PEND    RUN  SSUSP  USUSP  RSV
MPEND  PJOBS MPJOBS
root          -        -     15     15      0      0      0    0
20      3     15

[root@master-test ~]# cat pack.txt
-R "select[mem=1024000]" sleep 1
-n 7 -R "select[mem=1024000]" sleep 1      ---> 会触发
MAX_PEND_SLOTS 限制
-R "select[mem=1024000]" -J "test[1-10]" sleep 1
-R "select[mem=1024000]" -J "test[1-10]" sleep 1

[root@master-test ~]# bsub -pack pack.txt
Line#1 Job <35> is submitted to default queue <normal>
Line#2 User <root>: Pending job slots threshold reached. Job not
submitted.
Line#3 User <root>: Pending job slots threshold reached. Job not
submitted.
Line#4 User <root>: Pending job slots threshold reached. Job not
submitted.
4 lines parsed, 1 jobs submitted, 3 errors found.
```

busers/ -w：

新增 MPJOBS MPEND PJOBS 三列数据：lsb.users 中定义的 MAX_PEND_JOBS、MAX_PEND_SLOTS 的配置，以及统计出当前用户的 PEND 任务数，分别展示在 busers -w 的 MPJOBS MPEND PJOBS 三列。

```
[root@master-test ~]# busers
USER/GROUP   JL/P    MAX  NJOBS   PEND    RUN  SSUSP  USUSP  RSV
root          -        5     0       0      0      0      0    0

[root@master-test ~]# busers -w
USER/GROUP   JL/P    MAX  NJOBS   PEND    RUN  SSUSP  USUSP  RSV
MPEND  PJOBS MPJOBS
root          -        5      0     0      0      0      0    0
20      0      15
```

bparams -l/-a：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

展示 MAX_PEND_JOBS/MAX_PEND_SLOTS/SUB_TRY_INTERVAL

```
[root@master-test ~]# bparams -l | grep -E
"MAX_PEND_SLOTS|MAX_PEND_JOBS|SUB_TRY_INTERVAL" -B 1
Job submission retry interval:
    SUB_TRY_INTERVAL = 20 seconds
System wide max pending jobs:
    MAX_PEND_JOBS = 13
System wide max pending slots:
    MAX_PEND_SLOTS = 17
```

3.8 自定义资源限制的单位

3.8.1 功能描述

volclava 中可以定义相关资源限制，比如 bsub -R，bsub -C core_limit，lsb.queues 中定义的 CORELIMIT、MEMLIMIT，这些资源限制的单位默认是 KB（文档说明是 KB；实测-R 的 rusage/select 是 MB，CORELIMIT、MEMLIMIT、STACKLIMIT、SWAPLIMIT 是 KB）。

但是对于大存储的今天，默认单位较小，使用起来太不方便，也无法灵活调整。比如默认单位是 kb，我要 bsub -M 限制 2G 的 mem_limit，就要写-M 2097152，算起来或写起来都比较麻烦。

因此通过定义 LSF_UNIT_FOR_LIMITS 配置，可以自定义资源默认单位，增加配置可读性、修改便捷性。

3.8.2 配置定义

lsf.conf 新增配置 LSF_UNIT_FOR_LIMITS：

lsf.conf 增加 LSF_UNIT_FOR_LIMITS 的配置，取值范围：

```
Syntax
       LSF_UNIT_FOR_LIMITS=unit

       *  MB or M (megabytes)
       *  GB or G (gigabytes)
       *  TB or T (terabytes)
       *  PB or P (petabytes)
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
        *  EB or E (exabytes)
```

作用范围

绿色背景表示：目前支持的使用 LSF_UNIT_FOR_LIMITS 调整参数单位的“命令/配置文件”。

1. LSF_UNIT_FOR_LIMITS 影响的命令及参数

命令\参数	**-R res_req**	**-C core_limit**	**-M mem_limit**	**-S stack_limit**	**-v swap_limit**
bsub					
bhosts					
bmod					lsf 支持该参数，但是 volclava 不支持
lsloadadj					
lshosts					
lsload					
lsmon					
lsplace					

2. LSF_UNIT_FOR_LIMITS 影响的配置文件及配置项

配置文件\配置项	**RES_REQ**	**RESUME_COND**	**CORELIMIT**	**MEMLIMIT**	**STACKLIMIT**	**SWAPLIMIT**
lsb.queues						

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

配置文件\配置项	**loadSched**	**loadStop**
lsb.hosts		
lsb.queues		

3.8.3 功能效果

1. **影响命令参数中的-R ResReq**

   包括 bsub、lsload、bhosts 中的-R ResReq 表达式，示例：

- bsub -R 示例：

在 lsf.conf 前后修改 LSF_UNIT_FOR_LIMITS，然后分别提交 bsub -R，发现任务会因为单位不同而触发 PEND 或 RUN。

```
测试过程如下：

LSF_UNIT_FOR_LIMITS=MB
提交作业：
bsub -R "select[tmp>10240] rusage[mem=1024]" sleep 11
bsub -R "select[tmp>10240&&mem>=1024]" sleep 11

修改为 LSF_UNIT_FOR_LIMITS=GB
提交作业：
bsub -R "select[tmp>10240] rusage[mem=1024]" sleep 11
bsub -R "select[tmp>10240&&mem>=1024]" sleep 11
```

期望结果：

- 任务执行环境符合 resreq mem>1G，进而进入 RUN 的执行状态；符合预期

下面是事例输出：

```
[root@master-test etc]# grep UNIT ./lsf.conf
LSF_UNIT_FOR_LIMITS=MB
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
[root@master-test etc]# bsub -R "select[tmp>10240]
rusage[mem=1024]" sleep 11
Job <12>is submitted to default queue <normal>.
[root@master-test etc]# bsub -R
"select[tmp>10240&&mem>=1024]sleep 11
Job <13>is submitted to default queue <normal>.

[root@master-test etc]# bjobs
JOBID   USER    STAT  QUEUE      FROM_HOST   EXEC_HOST
JOB_NAME   SUBMIT_TIME
12      root    RUN   normal     master-test cmp2-test   sleep
11   Apr 18 18:01
13      root    RUN   normal     master-test cmp1-test   sleep
11   Apr 18 18:01
```

- 修改单位为 GB，重启服务，任务因为资源不足 mem>1T 而 PEND；符合预期

```
[root@master-test etc]# grep UNIT ./lsf.conf
LSF_UNIT_FOR_LIMITS=GB

[root@master-test etc]# bsub -R "select[tmp>10240]
rusage[mem=1024]" sleep 11
Job <14>is submitted to default queue <normal>.
[root@master-test etc]# bsub -R
"select[tmp>10240&&mem>=1024]sleep 11
Job <15>is submitted to default queue <normal>.

[root@master-test etc]# bjobs -p
JOBID   USER    STAT  QUEUE      FROM_HOST     JOB_NAME
SUBMIT_TIME
14      root    PEND  normal     master-test   sleep 11   Apr
21 10:53
 Job's resource requirements not satisfied:2hosts;
15      root    PEND  normal     master-test   sleep 11   Apr
21 10:53
 Job's resource requirements not satisfied:2hosts;
```

- lsload -R、bhosts -R 示例：

```
[root@master-test etc]# lsload
HOST_NAME       status  r15s   r1m  r15m   ut    pg  ls    it
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
tmp    swp    mem
master-test          ok    0.0    0.1    0.1    9%    0.0    1      0    66
0M    12G
cmp1-test            ok    0.0    0.1    0.0    0%    0.0    1      0    66
0M 8536M
cmp2-test            ok    0.0    0.1    0.0    0%    0.0    1      0    66
0M 8824M

[root@master-test etc]# grep UNIT ./lsf.conf
LSF_UNIT_FOR_LIMITS=GB

[root@master-test etc]# lsload -R "select[mem>10]"
HOST_NAME       status  r15s   r1m  r15m   ut    pg  ls    it
tmp    swp    mem
master-test          ok    0.0    0.1    0.1    9%    0.0    1      0    66
0M    12G

[root@master-test etc]# bhosts -R "select[mem>10]"
HOST_NAME          STATUS       JL/U    MAX  NJOBS    RUN  SSUSP
USUSP    RSV
master-test        ok              0      8      0      0      0
0      0
```

2. **影响参数、配置中的 core_limit，mem_limit，swap_limit，stack_limit**

- bsub 中的 -C / -S / -M / -v:

```
bsub -C 1024 -v 2047  sleep 600
```

通过上述 bsub 命令提交任务，再通过 bjobs -l 可以查看对任务定义的 core_limit，mem_limit，swap_limit，stack_limit 信息，其中 bjobs -l 会把数值单位转换为 KB 展示。如下输出展示:

```
[root@master-test etc]# bsub -C 1024 -v 2047 sleeep 600
Job <3> is submitted to default queue <normal>.
[root@master-test etc]# bjobs -l 3

Job <3>, User <root>, Project <default>, Status <PENND>, Queue
<normal>, Command <sleep 600>
Wed May 7 11:29:45: Submitted from host <master-test>,CWD <$HOME>;

 CORELIMIT SWAPLIMIT
 1048576 K  2096128 K
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
 PENDING REASONS:
 New job is waiting for scheduling: 1 host;

 SCHEDULING PARAMETERS:
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

 RESOURCE REQUIREMENT DETAILS:
 Combined: select[type == local] order[r15s:pg]
 Effective: -
```

如果 core_limit，mem_limit，swap_limit，stack_limit 中传入的参数，结合 lsf.conf 中定义的 LSF_UNIT_FOR_LIMITS，数值超过 2T，则数据视为无效，同时打印 stderr 以便排查。如下输出：

```
[root@master-test etc]# bsub -C 1024 -v 2048 sleep 600
Bad rlimit specification, valid value 0<= n < 2T with
LSF_UNIT_FOR_LIMITS in lsf.conf. Job not submitted.
```

- lsb.queues 中定义的 CORELIMIT，STACKLIMIT，MEMLIMIT，SWAPLIMIT

```
STACKLIMIT   = 1024
MEMLIMIT   = 1
SWAPLIMIT   = 2048
CORELIMIT    = 2
```

bqueues -l 可以看到定义的 CORELIMIT，STACKLIMIT，MEMLIMIT，SWAPLIMIT，并按照 lsf.conf 中定义的 LSF_UNIT_FOR_LIMITS 转化为单位 KB；

```
QUEUE: normal
  -- For normal low priority jobs, running only if hosts are
lightly loaded.  This is the default queue.

PARAMETERS/STATISTICS
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND   RUN
SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     0     0     0
0     0    0
Interval for a host to accept two jobs is 0 seconds
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
 STACKLIMIT CORELIMIT MEMLIMIT SWAPLIMIT
 1048576 K     2048 K    1024 K 2097152 K

SCHEDULING PARAMETERS
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-      -
 loadStop    -     -     -     -       -     -    -     -     -
-      -

USERS:  all
HOSTS:  all
```

同时如果 CORELIMIT，STACKLIMIT，MEMLIMIT，SWAPLIMIT 中定义的数值，结合 lsf.conf 中定义的 LSF_UNIT_FOR_LIMITS，数值超过 2T，则数据视为无效，同时打印错误 log 以便排查，输出如下：

```
May 7 11:16:58 24110 3 20 addQData: Invalid qPtr->rLimits[8]value
2048, beyond [0~2T), in queue <normal>; ignored
```

3. **影响配置中的 loadSched、loadStop**

包括 lsb.hosts/lsb.queues 的 loadSched、loadStop

前置条件：

lsf.conf 修改 LSF_UNIT_FOR_LIMITS

```
LSF_UNIT_FOR_LIMITS=GB
```

lsb.queues 增加 loadSched、loadStop

```
mem                 = 350/241
```

lsb.hosts 增加 loadSched、loadStop

```
HOST_NAME      MXJ JL/U   r1m    pg    ls     tmp   DISPATCH_WINDOW
demo-test      !     ()     ()    ()    33/23     ()
```

验证步骤：

```
bhosts -l
bqueues -l normal
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
bsub sleep 600
```

bhosts -l 可以看到配置文件中定义的 loadSched/loadStop 33/23 已经可以结合 lsf.conf 中定义的 LSF_UNIT_FOR_LIMITS=GB 相乘了。

```
[root@master-test etc]# bhosts -l demo-test
HOST  demo-test
STATUS       CPUF    JL/U    MAX  NJOBS    RUN  SSUSP  USUSP
RSV DISPATCH_WINDOW
ok           1.00       -      8      0      0      0      0
0      -

 CURRENT LOAD USED FOR SCHEDULING:
             r15s   r1m  r15m    ut    pg    io   ls    it   tmp
swp   mem  slots
 Total        0.0   0.0   0.0   15%   0.0     0    1     1   66G
0G   12G      8
 Reserved     0.0   0.0   0.0    0%   0.0     0    0     0    0M
0M    0M      -

 LOAD THRESHOLD USED FOR SCHEDULING:
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     - 33792M
-      -
 loadStop    -     -     -     -       -     -    -     - 23552M
-      -
```

bqueues -l 可以看到配置文件中定义的 loadSched/loadStop 350/241 也已经可以结合 lsf.conf 中定义的 LSF_UNIT_FOR_LIMITS=GB 相乘了。

```
[root@master-test etc]# bqueues -l normal
QUEUE: normal
  -- For normal low priority jobs, running only if hosts are
lightly loaded.  This is the default queue.

PARAMETERS/STATISTICS
PRIO NICE STATUS          MAX JL/U JL/P JL/H NJOBS  PEND   RUN
SSUSP USUSP  RSV
 30   20  Open:Active       -    -    -    -     0     0     0
0     0    0
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
Interval for a host to accept two jobs is 0 seconds

 STACKLIMIT CORELIMIT MEMLIMIT SWAPLIMIT
 1048576 K     2048 K    1024 K 2097152 K

SCHEDULING PARAMETERS
           r15s   r1m  r15m   ut      pg    io   ls    it    tmp
swp    mem
 loadSched   -     -     -     -       -     -    -     -     -
-   1382400M
 loadStop    -     -     -     -       -     -    -     -     -
-   246784M

USERS:  all
HOSTS:  all
```

4. 控制作业的执行

4.1 运行时资源使用限制

4.1.1 关于资源使用限制

资源使用限制能控制运行中作业消耗资源的数量。如果作业使用了超过指定数量的资源，系统将向其发送信号并被降低其优先级。

能被 volclava 管理员指定的限制：

- 队列级：lsb.queues
- 作业级：在提交作业时

例如，通过定义一个高优先级的短队列，你能允许短作业先于长作业被调度。 为了防止一些用户提交长作业至这个短队列，你可以为这个队列设置 CPU 限制以便提交至该队列的作业的运行时间不能超过这个限制。

在队列级指定的限制是硬限制，在作业提交时指定的限制是软限制。

1. **资源使用限制和资源分配限制**

“资源使用限制”不同于“资源分配限制”， “资源使用限制”在作业调度和作业被分发前是不被执行的。你能设置“资源分配限制”，以在为不同级别的作业调度时限制一个给定资源的可用数，并限制其所适用资源的消费者。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

限制	作业语法（bsub）	语法（lsb.queues）	格式/默认值
Core 文件大小限制	-C core_Limit	CORELIMIT=Limit integer	KB
CPU 时间限制	-c cpu_Limit	CPULIMIT=[defauLt] maximum	[hours:]minutes[/host_name |/host_Model]
数据段大小限制	-D data_Limit	DATALIMIT=[defauLt] maximum	KB
文件大小限制	-F fiLe_Limit	FILELIMIT=Limit	KB
内存限制	-M mem_Limit	MEMLIMIT=[defauLt] maximum	KB
处理器限制	-p process_Limit	PROCESSLIMIT=[defauLt]maximum	整数
运行时间限制	-W run_Limit	RUNLIMIT=[defauLt] maximum	[hours:]minutes[/host_name |/host_Model]
堆栈段大小限制	-S stack_Limit	STACKLIMIT=Limit	KB
虚拟内存限制	-v swap_Limit	SWAPLIMIT=Limit	KB

2. **资源使用限制优先级**

   如果没有在作业提交指定限制，那么下面的适用于所有提交至该队列的作业：

如果...	那么...
缺省和最大限制被定义	缺省值被执行
只有最大值被定义	最大值被执行

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

没有限制在队列或作业提交被指定	没有限制被执行

3. **不正确的资源使用限制**

不正确的限制将被忽略，并且在集群被重配置或重启动时显示一条警告信息。 当 volclava 被启动时，mbatchd 日志文件也会记录一条警告信息。 如果没有在作业提交时指定限制，那么下面的适用于所有提交至该队列的作业：

如果...	那么...
缺省限制是不正确的	缺省值被忽略并且这个最大限制被执行
缺省和最大限制被定义，并且最大值是不正确的	最大值被忽略并且这个资源没有最大限制，只有一个缺省限制
缺省和最大限制都是不正确的	缺省和默认限制都被忽略并且没有限制被执行

在作业提交时指定的资源使用限制必须小于在 lsb.queues 中指定的最大值。如果用户指定的限制大于队列级的最大值，这个作业提交将被拒绝并且发出下面的信息：

Cannot exceed queue’s hard Limit(s). JOB not submitted

4.1.2 资源使用的单位设置

测量资源使用限制单位，下面资源使用限制的缺省单位是 KB：

- Core 限制（-C 和 CORELIMIT）
- 内存限制（-M 和 MEMLIMIT）
- 堆栈限制（-S 和 STACKLIMIT）
- 交换区限制 （-v 和 SWAPLIMIT）

4.1.3 指定资源使用限制

队列能对运行中的作业执行资源使用限制。volclava 支持大多数下层操作系统支持的限制。另外，volclava 也支持一些下层操作系统不支持的限制。使用 lsb.queues 中的参数指定队列级的资源使用限制。

1. **仅指定最大值**

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

使用资源限制可以仅指定最大值。

例如，为了指定一个运行时间限制的最大值，为 lsb.queues 中的 RUNLIMIT 参数指定一个值：

RUNLIMIT = 10

这个队列的运行时间限制最大值是 10 分钟。作业不能运行超过 10 分钟。作业处于 RUN 状态超过 10 分钟会被 volclava 杀死。如果只有一个运行时间限制被指定，使用 bsub –W 提交的作业如果指定的运行时间限制超过了运行时间限制的最大值，这个作业不被允许运行。不是通过 bsub –W 提交的作业被允许运行，但是在作业处于 RUN 状态超过指定的运行时间限制最大值时这些作业会被杀死。

2. **指定缺省值和最大值**

如果你指定了这两个限制，第一个是队列中作业的缺省（软限制）限制，第二个是最大（硬）限制。缺省和最大限制必须是正整数。缺省限制必须小于最大限制。如果缺省限制大于最大限制，缺省限制被忽略。指定缺省限制是为了避免在 bsub 命令里指定资源使用限制。

例如，为了指定一个缺省和最大运行时间限制，为 lsb.queues 中的 RUNLIMIT 参数指定两个值：

RUNLIMIT = 10 15

- 第一个数字是缺省运行时间限制，适用于该队列中所有没有在作业级指定运行时间限制（没有使用 bsub -W）的作业。
- 第二个数字是最大运行时间限制，适用于该队列中所有在作业级指定了运行时间限制（使用 bsub -W）的作业。缺省运行时间限制必须小于最大运行时间限制。

你能为下面 lsb.queues 中的资源使用限制指定缺省值和最大值。

- CPULIMIT
- DATALIMIT
- MEMLIMIT
- PROCESSLIMIT
- RUNLIMIT
- THREADLIMIT

3. **两个限制的样本机设定**

如果为 CPU 时间限制或者运行时间限制指定了缺省和最大限制，只能指定一个样本机（不能同时指定两个机器为样本）。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

例如:下面的 CPU 时间限制是正确的（并且有同样的效果）

- CPULIMIT = 400/hostA 600
- CPULIMIT = 400 600/hostA

下面的 CPU 时间限制是不正确的:

CPULIMIT = 400/hostA 600/hostB

下面的运行时间限制是正确的（并且有同样的效果）:

- RUNLIMIT = 10/hostA 15
- RUNLIMIT = 10 15/hostA

下面的运行时间限制是不正确的:

RUNLIMIT = 10/hostA 15/hostB

4.1.4 实例

1. **队列级限制**

   - CPULIMIT = 20/hostA 15

   第一个数字是缺省的 CPU 时间限制，第二个数字是最大 CPU 时间限制。 但是，这个缺省的 CPU 时间限制被忽略了，因为缺省 CPU 时间限制大于最大 CPU 时间限制。

   - CPULIMIT = 10/hostA

   在这个例子中，缺少第二个数字，这表示没有缺省 CPU 时间限制。这个指定的数字被作为缺省和最大 CPU 时间限制。

   - RUNLIMIT = 10/hostA 15

   第一个数字是缺省运行时间限制。第二个是最大运行时间限制。第一个数字指定:缺省的运行时间限制被用于那些没有在提交时指定运行时间限制（没有使用 bsub 的-W 选项）的作业。

   - RUNLIMIT = 10/hostA

   没有缺省运行时间限制被指定。这个指定的数字被作为缺省和最大运行时间限制。

2. **作业级限制**

   - bsub -M 5000 myJOB 提交作业时指定内存限制为 5000KB
   - bsub -W 14 myJOB 作业被期望运行 14 分钟。如果使用 bsub –W 指定的运

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

行时间限制超过了队列中定义的值，作业被拒绝。

4.1.5 CPU 时间和运行时间标准化

为了使用平台无关的方式为作业设置 CPU 时间限制和运行时间限制，volclava 通过有关的机器的 CPU 因子度量这个限制。当一个作业被分发至一个执行机器， 那么这个限制根据执行机器的 CPU 因子被格式化。

当一个格式化的 CPU 时间或运行时间被给定，这个在执行机器上的实际时间被指定为定义的限制时间乘以标准化机器（度量机器）的 CPU 因子，再除以执行机器的 CPU 因子。

如果 lsb.params 中指定了 ABS_RUNLIMIT=Y，与这个应用程序关联的作业的运行时间限制和运行时间估值将不根据机器的 CPU 因子来进行标准化。绝对时钟时间被用于所有在提交时配置了运行时间限制的作业， 或运行时间限制。

1. **标准化样机**

如果没有为 CPU 时间或运行时间指定机器或机器型。volclava 使用在队列级定义的缺省 CPU 时间格式化机器（如果 lsb.queues 中配置了 DEFAULT_HOST_SPEC），否则使用在集群级定义的缺省 CPU 时间格式化机器（如果 lsb.params 中配置了 DEFAULT_HOST_SPEC），否则使用作业提交机器。

例如：

CPULIMIT=10/hostA 如果 hostA 的 CPU 因子是 2，并且 hostB 的 CPU 因子是 1（hostB 比 HostA 慢），这里指定了在 hostA 上，或其它 CPU 因子是 2 的机器上的实际运行时间限制是 10 分钟。但是，如果 hostB 是执行机器，那么 hostB 上的实际运行时间是 20 分钟 （10*2/1）。

2. **缺省的 CPU 限制和运行时间限制的标准化样机**

第一个指定的有效 CPU 因子被用于 CPU 限制和运行时间限制。为了有效，机器的指定必须是 volclava 集群中某个机器的有效名字。这个机器的 CPU 因子被使用，即便指定的限制是无效的。如果 CPU 限制和运行限制有不同的机器指定， CPU 限制机器指定被执行。如果没有为 CPU 时间或运行时间指定机器或机器型，LSF 根据下面的优先级决定缺省的标准化样机：

- lsb.queues 中配置的 DEFAULT_HOST_SPEC
- lsb.params 中配置的 DEFAULT_HOST_SPEC
- 如果 lsb.queues 和 lsb.params 中都没有配置 DEFAULT_HOST_SPEC，最大 CPU 因子的机器被用作标准化样机

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

4. **CPU 时间限制（bacct， bhist， bqueues）**

标准化的 CPU 时间在 bqueues 的输出中限制。CPU 时间在 bacct 和 hist 的输出中没有被标准化。

4.2. 负载阈值

4.2.1 自动作业挂起

在 volclava 环境下运行的作业能基于执行机器上的负载情况被挂起。每个机器和队列上能配置一组挂起条件。如果执行机器上的负载条件超过了相应的机器或队列的挂起条件，那么一个或多个运行在这个机器上的作业将被挂起以降低负载。

当 volclava 挂起一个作业时，volclava 调用 SUSPEND 动作。缺省的 SUSPEND 动作是发送 SIGSTOP 信号量。

缺省情况下，当负载等级降低至挂起条件以下时，作业被恢复。每个机器和队列上都能配置挂起条件，以便挂起的可检查点的或可重运行的作业能被自动迁移至其它机器上。

如果没有为某个负载指标配置挂起的阀值，当要决定是否挂起作业时，volclava 不会检查这个负载指标的值。

挂起的阀值也能被用于执行队列内部的优先级。例如，如果你配置了一个低优先级的队列为 0.25 的 r1m（1 分钟 CPU 运行队列长度）调度阀值和 1.75 的挂起阀值。当机器是空闲的时候，这个队列启动一个作业。如果这个作业消耗 CPU 严重，这个作业快速地增加运行队列长度从 0.25 到 1.25。一个配置了 1.5 的调度阀值和没有限制的挂起阀值的高优先级队列也发送了一个作业至同一机器，这时增加运行运行队列长度至 2.25。这超出低优先级作业的挂起阀值， 所以这个作业将被挂起。这个运行队列的长度保持 0.25 以上，直到高优先级作业退出。在高优先级作业退出这个运行队列后，指标降低至空闲水平，这时这个低优先级作业才可能被恢复。

当作业运行在一个机器上时，volclava 周期性地检查那个机器上的负载水平。如果任何负载指标超出某个作业的相应的机器或队列的挂起阀值时，volclava 挂起这个作业。这个作业保持挂起状态直到负载级别满足调度阀值。

在定期的间隔里，volclava 从某个机器获取负载级别。这个周期在 lsb.params 中的 SBD_SLEEP_TIME 参数定义。那么，对每个运行在这个机器上作业来说，volclava 把负载级别与机器挂起条件和队列挂起条件比较。如果任何相应机器或队列级别的挂起条件被满足，这个作业被挂起。只有对于那些特定的作业挂起阀值的负载级别太高，作业才会被挂起。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

在一些特殊情况下，volclava 不会因为负载级别自动挂起作业。volclava 不挂起作业：

- 使用 brun –f 强制运行。
- 如果只有这一个作业运行在这个机器上，除非这个机器被交互式使用。
- 当只有一个作业运行在这个机器上时，它不会因为任何原因被挂起，除了这个机器不是交互式地空闲（交互式的空闲时间负载指标小于一分钟）。这意味着：一旦一个作业被启动在一个机器上，至少有一个作业继续运行，除非这个机器上有一个交互式用户。一旦这个作业被挂起， 它将不被恢复直到所有调度条件都被满足，所以它应该没有干扰这个交互式用户。
- 由于内存分页率，除非这个机器正在被交互式使用。当一个机器有交互式用户时，volclava 挂起高分页率的作业，以便为机器上的交互式用户提高相应速度。当一个机器是空闲时，这个 pg（分页率）负载指标被忽略。 lsb.params 中的 PG_SUSP_IT 参数控制这个行为。如果这个机器已经处于空闲超过 PG_SUSP_IT 分钟，这个 pg 负载指标不被检查挂起阀值。

4.2.2 挂起条件

volclava 提供配置挂起条件的其他方式。在机器级配置的挂起条件作为负载阀值，尽管在队列级配置负载阀值，或者使用 lsb.queues 中的 STOP_COND 参数，或者在两个地方都配置。

最常使用作为挂起条件的负载指标是 CPU 运行队列长度（r15s，r1m 和 r15m）、分页率（pg）和空闲时间（it）。在挂起作业时（swp）和（tmp）指标也被考虑。 为了给交互式用户优先级，设置 it（idLe time）负载指标的挂起阀值为非零。 当有任意用户的活跃的时作业被停止，并且在机器处于空闲的时间是 it 调度条件中配置的时间时作业恢复。

为了调节分页率的挂起阀值，需要知道你的应用程序的行为。在另外一个空闲的机器上使用 lsload 检查分页率，并启动你的作业。观察应用程序运行时的分页率。通过程序活跃是的分页率减去空闲时的分页率，得到是你的应用程序的分页率。挂起阀值应该允许至少 1.5 倍于你的应用程序的分页率。在任何高于调度阀值的分页率时一个作业能被调度，所以这个挂起阀值应该至少是调度阀值加上 1.5 倍的应用程序分页率。这防止系统在调度一个作业后又因为它的分页率立即挂起它。

有效的 CPU 运行队列长度条件应该像分页率一样被配置。对于 CPU 使用率高的作业，有效的运行队列长度指标为每个作业近似地加一。对于那些使用超过一个进程的作业，你应该做一些测试运行以决定你的作业对运行队列长度指标的影响。而且，

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

挂起阀值应该至少等于调度阀值加上 1.5 倍的作业负载。

1. **大小可变作业**

如果为大小可变作业添加新机器，LSF 考虑这些新机器的负载阀值调度。如果机器从分配中被移除，volclava 不为大小可变作业应用负载阀值调度。

2. **在队列级配置负载阀值**

队列定义（lsb.queues）能包含从零个或多个负载指标的阀值。任何没有配置阀值的负载指标不会影响作业调度。

3. **语法**

每个负载指标在单独的一行里被配置，格式如下：

Load_index = LoadSched/LoadStop

制定负载指标的名字，例如 r1m 是一分钟 CPU 运行队列长度或者 pg 是分页率。LoadSched 是这个负载指标的调度阀值。LoadStop 是挂起阀值。在一个作业被分发至一个机器前和能被恢复前，这个机器必须满足这个 LoadSched 条件。如果 LoadStop 添加被满足了，一个作业被挂起。

LoadSched 和 LoadStop 阀值允许使用 AND/OR 逻辑指定条件。例如，设置：

- MEM=100/10
- SWAP=200/30

解释为“mem>=100 && swap>=200”LoadSched 条件和“mem < 10 || swap < 30” LoadStop 条件

4. **理论**

- r15s、r1m 和 r15mCPU 运行队列长度条件被与 lsload –E 报告的有效队列长度（为多处理器机器格式化）。应该为单处理器机器在适当层级设置这些参数的阀值。
- 如果一个低优先级队列比一个高优先级队列拥有更高的挂起阀值，那么在较高优先级队列中的作业先于低优先级队列中的作业被挂起。

5. **在机器级配置负载阀值**

在 lsf.cluster.cluster_name 文件的 Host 部分，一个共享的资源不能被使用作为一个负载阀值。

6. **在队列级配置挂起条件**

挂起作业的条件能使用队列级 STOP_COND 参数指定。使用一个资源需求串定义。当停止一个作业时，只有资源需求串的 seLect 部分被考虑。所有其它部分被忽略掉。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

这个参数为 LoadStop 提供相似的、但是更灵活的功能。如果 LoadStop 阀值被已经被指定了，那么如果 STOP_COND 为 TURE 或者 LoadStop 阀值被超过了那么一个作业被挂起。

例如：

这个队列基于桌面机器的空闲时间和计算服务器的可用交换区和内存挂起一个作业。假设 cs 是一个在 lsf.shared 文件中定义的布尔资源并在 。

lsf.cluster.cluster_name 文件中被配置以指示机器是否是计算服务器:

```
Begin  Queue
STOP_COND= select[((!cs && it < 5) || (cs && mem < 15 && swap
< 50))]
End Queue
```

7. **查看机器级和队列级挂起条件**

使用 bhosts –l 和 bqueues –l 命令显示挂起条件。

8. **查看挂起原因**

bjobs –Lp 命令显示导致 volclava 挂起作业的负载阀值（包括调度参数）。

STOP_COND 的使用影响 bjobs 命令显示的挂起原因。如果在队列级指定了 STOP_COND，LoadStop 阀值没有被指定，那么每个单独的负载指标的挂起原因不被显示。

9. **恢复挂起作业**

作业被挂起以防止机器过载，以防止批处理作业干扰交互式用户，以允许一个更急迫的作业运行。当这个机器不再过载时，挂起作业应该继续运行。

当 volclava 自动恢复一个作业时，volclava 调用 RESUME 动作。RESUME 的缺省动作是发送 SIGCONT 信号。

如果在机器中存在任何挂起的作业，volclava 在每次调度时检查负载等级。

如果负载等级在队列和机器调度阀值内，并且队列的所有的恢复条件（lsb.queues 中的 RESUME_COND）被满足，这个作业被恢复运行。

如果没有定义 RESUME_COND，那么 LoadSched 阀值被用于控制作业的恢复: 作业被恢复的所有 LoadSched 阀值必须被满足。如果定义了 RESUME_COND，LoadSched 阀值会被忽略。高优先级队列中的作业首先被检查。为了防止机器再次过载，每次调度只有一个作业被恢复。

10. **指定恢复条件**

使用 lsb.queues 中的 RESUME_COND 指定如果一个被挂起的作业需要恢复时在

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

一个机器上必须满足的恢复条件。 当恢复一个作业时，只有资源需求串的 seLect 部分被考虑，所有其它部分被忽略。

11. **查看恢复阀值**

    bjobs –L 命令显示控制作业何时被恢复的调度阀值。

4.3. 前处理和后处理命令

作业在提交时能指定“前处理”和“后处理”命令。一个“前处理”或“后执行”命令是指在作业启动前或在作业完成后要运行的、任意的命令。“先执行”和“后处理”命令是在与作业分离的环境中被执行的。

4.3.1 关于前处理和后处理命令

有些批处理作业需要某些 volclava 不直接支持的资源。例如，适当的前处理和/或后处理命令能被用来处理以下各种情况：

- 预留设备，如磁带驱动
- 为作业创建和删除 scratch 目录
- 定制的调度
- 检查软件许可证的可用性
- 分配作业运行在 SMP 机器上的某个指定处理器上
- 转移需要作业处理的数据文件
- 在作业运行前和运行后修改系统配置文件

缺省情况下，前处理和后处理命令运行在和批处理作业相同的用户 ID、环境、home 和工作目录下。如果命令不在你的正常执行路径下，命令的全路径名必须被指定。

对于并行作业来说，命令被运行在第一个被选择的机器上。 这个命令路径能在 Linux 最多包含 4094 个字符。

1. **前处理命令**

   前处理命令支持作业启动决定：不能在 volclava 中被直接配置。volclava 支持作业级、 队列级前处理命令。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

这个前处理命令把它的退出状态返回给 volclava。当一个前处理命令被指定时，这个作业被保持在队列中直到这个指定的前处理命令正常返回，退出状态零 （0）。

如果前处理名退出的状态时非零，这个批处理作业不被分发。这个作业回到 PEND 状态，并且 volclava 会试图分发其它作业至那个机器。当这个作业在等待时，其它作业能先于这个等待作业被处理。下次 volclava 试图分发作业时重复这个过程。

如果前处理命令的退出值是 99 时，这个作业不会回到 PEND 状态，而是退出。如果前处理命令失败就终止作业，从而提高了作业控制的灵活性。 volclava 假设前处理命令运行没有副作用。例如，如果前处理命令预留一个软件许可证或者其它资源，为了不重复预留这个资源，你必须保证不为同一个批处理作业预留这个相同资源超过一次。

2. **后处理命令**

如果指定了一个后处理命令，那么这个命令会在作业完成后运行，不论这个作业的退出值是什么。后处理命令通常被用于清理一些由前处理命令和作业执行留下的状态。volclava 支持作业级、队列级后处理命令。

3. **作业级命令**

bsub –E 选项指定一个任意的在批处理作业启动前运行的命令。当 volclava 发现一个适合运行一个作业的机器时，前处理命令在那台机器上被执行。如果先执行命令成功运行，这个批处理作业被启动。

bsub –Ep 选项也可以用来指定作业级的后处理命令，在作业完成后在执行机器上运行的命令。

4. **后处理作业状态**

一些作业可能在作业的后续处理被执行后才被认为完成。例如，一个作业可能需要从一个后处理作业脚本中退出，清理作业文件，或者在作业完成后转移作业输出。

缺省情况下，DONE 或 EXIT 作业状态不表示作业运行后的处理是否被完成。 这些依赖于处理的作业可能会被过早地启动。你可以在 bsub –w 命令中使用 post_done 和 post_err 关键字来指定作业后处理的依赖条件。相应的 POST_DONE 和 POST_ERR 显示后处理的完成情况。后处理的资源使用也不统计在作业资源使用里。在作业完成后，你不能在后处理上执行任何作业控制。 后处理退出代码不向 volclava 报告。

4.3.2 配置前处理和后处理命令

前处理命令以及后处理命令均能在作业级、队列中配置。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1. **作业级命令**

   作业级前处理和后处理命令不需要配置

   使用 busb –E 选项指定任意一个在作业启动前运行的命令

   使用 busb –Ep 选项指定任意一个在作业完成后运行的命令

2. **实例**

   下面的例子显示一个需要磁带驱动器的批处理作业。用户编写了一个 tapeCheck 程序：如果指定的磁带驱动器是准备好的，退出值为零

```
bsub -E "/usr/share/bin/tapeCheck /dev/rmt01" myJOB
```

3. **队列级命令**

   在队列定义中（lsb.queues）使用 PRE_EXEC 和 POST_EXEC 关键字指定先执行和后处理命令。

当在队列级设置前处理和后处理命令时，下面的要点应该被考虑：

- 如果前处理命令的退出值非零，那么这个命令被认为执行失败了并且作业被重队列至队列头。这个特性能被用来实现定制的调度：如果调度这个作业的条件不被满足，让前处理命令失败。
- 为作业设置的其他环境变量也为前处理和后处理命令设置了。
- 当作业被从一个包含后处理命令的队列分发时，volclava 为这个作业记住这个后处理命令定义。如果稍后这个作业被转换至其他队列或者这个队列的后处理命令被改变时，volclava 仍然为这个作业运行这个原始的后处理命令。
- 当后处理命令被运行时，环境变量 LSB_JOBEXIT_STAT 被设置为作业的退出值。
- 如果一个作业因为作业的执行环境被设置失败而被重队列，或者如果作业退出值是队列中 REQUEUE_EXIT_VALUES 值中的一个，执行命令也被运行。如果作业被重队列，LSB_JOBPEND 环境变量将被设置。如果作业的执行环境不能被设置，LSB_JOBEXIT_STAT 被设置为 0。
- 在一个可重运行的作业重启动时，运行后处理命令可能不是每次都是需要的；例如，如果后处理命令删除某些文件、或者做一些清除工作，这些操作应该仅仅在作业成功完成时才能发生。使用 lsf.conf 中的 LSB_DISABLE_RERUN_POST_EXEC=Y 防止作业被重运行时后处理命令被运行。
- 如果队列级和作业级的前处理命令都被指定了，作业级前处理命令在队列级

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

前处理命令后运行。

- 如果应用程序级和作业级后处理命令都被指定了，作业级后处理命令覆盖应用程序级后处理命令。队列级后处理命令在应用程序级后处理命令和作业级后处理命令之后运行。

4. **实例**

下面的队列指定前处理命令/usr/share/lsf/pri_prexec 和后处理命令 /usr/share/lsf/pri_postexec

```
Begin  Queue
QUEUE_NAME = priority
PRIORITY = 43
NICE = 10
PRE_EXEC = /usr/share/lsf/pri_prexec
POST_EXEC = /usr/share/lsf/pri_postexec
End Queue
```

5. **linux 和 Linux 上的前处理和后处理命令**

前处理和后处理命令的配置的整个内容被运行在/bin/sh –c 下，所以 shell 的特性能被用在命令中。

例如，下面的是有效的:

PRE_EXEC = /usr/share/lsf/misc/testq_pre >> /tmp/pre.out

POST_EXEC = /usr/share/lsf/misc/testq_post | grep -v "Hey!"

上面的前处理命令和后处理命令被运行在/tmp 下。

标准的输入、输出和错误输出被设置为/dev/nuLL

前处理和后处理命令的输出能显式地被重定向至一个文件（处于 debug 目的）。

PATH 环境变量能被设置为:

PATH='/bin /usr/bin /sbin /usr/sbin'

6. **限制和副作用**

作业查询命令（bjobs、bhist）显示作业处于 RUN 状态直到后处理处理被完成，即使这个作业本身已经完成。

作业控制命令（bstop、 bkill、 bresume）不受影响。可重运行作业可能在作业事实上已经完成后重运行，因为在后处理处理完成前机器变得不可用了，但是 mbatchd 认为这个作业仍然处于 RUN 状态。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

4.4 作业启动器

4.4.1 关于作业启动器

一些作业必须运行在特定的环境下，或者需要一些类型的设置在作业运行前被执行。在一些 shell 环境中，作业设置经常被写入一个封装 shell 脚本文件， 这个文件本身包含了一个启动这个作业的调用。

一个作业启动器是一个指定的封装脚本或者可执行程序，它通常为作业执行环境设置，然后调用作业本身，这个作业继承了由作业启动器创建的执行环境。volclava 控制这个作业启动器进程，而不是这个作业本身。一个典型作业启动器的使用是定制 volclava 使用特定的应用程序环境，例如 ALias 渲染器或者 IBM RationaL ClearCase。

运行作业启动器的两种方式在 volclava 中有两种方式使用作业启动器。你能用其中任意一种完成相似的事情， 但是他们的功能细节稍微有些不同。

1. **命令级**

是用户定义的。他们运行使用 lsrun、lsgrun 或者 ch 提交的交互式作业。命令级作业启动器对批处理作业（包括使用 bsub –I 运行的交互式批处理作业）没有影响。使用 LSF_JOB_STARTER 环境变量为交互式作业指定一个作业启动器。

2. **队列级**

由 volclava 管理员定义，并且运行提交至定义了 JOB_STARTER 参数集的队列的批处理作业。使用 bsub 提交作业至包含作业级作业启动器的队列。

3. **前处理命令不是作业启动器**

一个作业启动器不同于一个前处理命令。一个前处理命令必须在 volclava 作业启动器之前成功运行并且退出。它通知 volclava 分发作业，但是因为前处理命令是一个与作业不相干的进程，所以它不控制作业或影响作业的执行环境。但是一个作业启动器是 volclava 控制的一个进程。它负责调用 volclava 并控制作业的执行环境。

4. **实例**

下面是一些作业启动器的例子:

- 在 linux 中，一个定义为/bin/ksh –c 的作业启动器促使作业被运行在一个 Korn shell 环境下
- 在 lsb.queues 中设置 JOB_STARTER 参数为$USER_STARTER 使得用户可以通过定义 USER_STARTER 环境变量来定义他们自己的作业启动器。
- 设置一个作业启动器为 make clean 促使这个 make clean 命令在用户作业前被运行。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

4.4.2 命令级作业启动器

一个命令级作业启动器允许你指定一个可执行文件，这个可执行文件为作业做任意必要的设置并在设置完成时运行这个作业。你能选择一个已存在的命令作为一个作业启动器，或者你能创建一个包含了所需命令集的脚本作为一个作业启动器。

这部分描述如何设置并使用一个命令级作业启动器以运行交互式作业。 命令级作业启动器对批处理作业没有任何影响（包括交换式批处理作业）。

一个作业启动器也能被定义在队列级（使用 JOB_STARTER 参数）。只有 volclava 管理员能配置队列级作业启动器。

1. **LSF_JOB_STARTER 环境变量**

使用 LSF_JOB_STARTER 环境变量指定一个命令或脚本，这个命令或脚本就是这个交互式作业的作业启动器。当这个环境变量 LSF_JOB_STARTER 被定义时，RES 调用作业启动器而不是运行这个作业本身，并把作业作为命令行参数传递给作业启动器。

2. **使用命令级作业启动器**

作业启动器在 bourne shell 中被调用。使得命令行等价于：

```
/bin/sh -c "$LSF_JOB_STARTER command [argument ...]"
```

其中 command 和 argument 是你在 lsrun、lsgrun 或者 ch 中指定的命令行参数。

3. **实例**

如果你使用下面的 C-shell 命令定义 LSF_JOB_STARTER 环境变量：

```
% setenv LSF_JOB_STARTER "/bin/sh -c"
```

那么你运行一个简单的 C-shell 作业：

```
% lsrun "’a.out; hostname’"
```

这个命令的实际运行是：

```
/bin/sh -c "/bin/sh -c ’a.out hostname’"
```

这个作业启动器能是一个 shell 脚本。在下面的例子中，这个 LSF_JOB_STARTER 环境变量被设置为名为 JOB_starter 的 bourne shell 脚本：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
$ LSF_JOB_STARTER=/usr/LocaL/JOB_starter
```

这个 JOB_starter 脚本包含下面内容:

```
#!/bin/sh
set term = xterm
 evaL "$*"
```

4.4.3 队列级作业启动器

volclava 管理员能为每个单独的队列定义一个作业启动器，以便为在队列中运行的作业创建一个指定的环境。一个队列级作业启动器指定一个可执行体，这个执行体能执行任意的必要设置并在设置完成时运行作业。lsb.queues 中的 JOB_STARTER 参数指定的这个命令或者脚本就是队列的作业启动器。

队列级作业启动器对交互式作业没有任何影响，除非这个交互式作业被作为一个交互式批处理作业提交至该队列。volclava 用户也能使用 LSF_JOB_STARTER 环境变量选择一个已存在的命令或脚本作为他们的交互式作业的作业启动器。

1. **配置一个队列级作业启动器**

使用 lsb.queues 的队列定义中的 JOB_STARTER 参数指定一个队列级作业启动器。所有提交至这个队列的作业被使用这个作业启动器运行。这些作业被这个指定的作业启动器进程调用而不是被批处理守护进程发起调用。

```
Begin  Queue
JOB_STARTER = xterm -e
End Queue
```

所有提交至这个队列的作业在 xterm 终端竞争者下被运行

2. **JOB_STARTER 参数（lsb.queues）**

队列定义（lsb.queues）中的 JOB_STARTER 参数的格式如下:

```
JOB_STARTER=starter [starter] ["%USRCMD"] [starter]
```

这个 starter 字符串是被用于启动作业的命令或脚本。其可以是任意能接受一个作业作为输入参数的执行体。可选的、额外的字符串能被指定。

当启动一个作业时，volclava 运行这个 JOB_STARTER 命令并把这个包含了作业命令的 shell 脚本作为参数传递给作业启动器。这个作业启动器被期望做一些处理然后运行包含了作业命令的 shell 脚本。这个命令被运行在/bin/sh –c 下，并能包含任意合法的 bourne shell 语法。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

3. **%USRCMD 字符串**

这个特殊的字符串%USRCMD 指示作业启动器命令在作业命令行的位置。缺省地，这个用户命令在作业启动器之后运行，所以这个%USRCMD 字符串通常不是必须的。例如，这两个作业启动器都得出相同的结果：

```
JOB_STARTER = /bin/csh -c
JOB_STARTER = /bin/csh -c "%USRCMD"
你必须用引号包含%USRCMD。这个%USRCMD 字符串后面可以跟额外的命令
例如： JOB_STARTER = /bin/csh -c "%USRCMD; sLeep 10"
如果一个用户提交下面的作业至含有这个作业启动器的队列：
bsub myJOB arguments
这个命令的实际运行是：
/bin/csh -c "myJOB arguments; sLeep 10"
```

4.4.4 使用作业启动器控制运行环境

在一些情形下，使用 busb –l 不能在执行机上正确地构建运行环境。为此 volclava 提供下面两种作业启动器： ¨

- Preservestarter——保存执行机器的缺省环境。不包含作业提交机上的任何设置。
- Augmentstarter——对比作业提交机，在执行机上添加所缺少的设置， 从而加强执行机的缺省用户环境。

1. **作业启动器执行体在哪**

缺省情况下，作业启动器执行体被安装在 LSF_BINDIR。如果你选择把他们保存在其他地方，确保他们所在的目录被包含在执行机器上的缺省的 PATH 环境变量中。

例如：

- 在 linux 上，放置作业启动器在$HOME/bin 下

2. **作业启动器的源代码**

作业启动器的源代码被安装在 LSF_MISC/exampLes 下

3. **添加至初始的登录环境**

缺省情况，preservestarter 作业启动器保存 RES 在执行机器上建立的环境，并为用户建立一个初始的登录环境，包含下面从执行机器上的用户登录环境的变量：

- HOME

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

- USER
- shell
- LOGNAME

任何额外的存在在作业提交机器上的用户登录环境的环境变量必须被添加至作业启动器的源代码

4. **实例**

作业提交机器上的一个用户的登录脚本包含下面的设置:

```
if ($TERM != "xterm") then
  set TERM=`tset - -Q -m 'switch:?vt100' ....
eLse
  stty -tabs
endif
```

为了在执行机器上登录成功，这个 TERM 环境变量也必须被包含在执行机器上的环境中。如果在作业启动器中没有设置，登录将失败，作业启动器也可能失败。如果在这个初始环境设置中作业启动器还能继续，那么作业也就可能正确地执行，但这不太可能发生。

4.5 外部作业提交和执行控制

4.5.1 理解外部执行体

1. **关于 esub**

通过使用 esub 执行体，volclava 提供验证、修改、或拒绝作业提交、修改执行环境、从作业提交机器直接传递数据至执行机器的能力。两者都是特殊的用户所写的并且必须放置在 LSF_SERVERDIR 目录下。

2. **验证、修改或拒绝作业**

为了验证、修改、或拒绝作业，需要写一个 esub。可参考本章的“使用 esub” 部分。

3. **传递数据**

为了直接传递数据至执行机器，需要写 esub

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

4.5.2 使用 esub

1. **关于 esub**

一个 esub，external submission 缩写，是用户所写的执行体（二进制文件或脚本）被用于验证、修改、拒绝作业。不带任何后缀名字的 esub 是默认 esub，这个 esub 被放置在 LSF_SERVERDIR 目录下（在 lsf.conf 中定义）。当有作业被提交、重启动、修改时，volclava 检查 esub 的存在。如果 volclava 找到了一个 esub，volclava 运行它。作业是否被提交、修改、拒绝依赖于 esub 中的逻辑。

用户可以编写一个或者多个带有后缀名字的 esub 放置在 LSF_SERVERDIR 目录下，例如 esub.app1、esub.app2、esub.app3，并可以通过在 lsf.conf 文件中定义 LSB_ESUB_METHOD="app1 app2" 来配置全局 esub 程序。用户提交或者修改作业时，也可以通过 bsub -a "app3"/ bmod -a "app3" 来指定作业级别的 esub 程序；当默认 esub 、全局配置和作业级别 esub 同时存在时，执行顺序按 LSF_SERVERDIR/esub → LSB_ESUB_METHOD → command -a app_list 依次生效。

注意：在 esub 脚本中，任何需要被提供给用户的消息应该被定向至标准错误（stderr）流，而不是标准输出（stdout）流

2. **连接 esub 和 volclava 的环境变量**

在 esub 执行环境中，volclava 提供下面的环境变量。

- **LSB_SUB_PARM_FILE**

这个变量指向一个包含了作业参数的零时文件， 当作业被提交时 esub 读取它。这些提交参数是名－值对集，每行指定一个， 格式为“option_name=value”。

提交参数文件实例：

```
bsub -q normaL -x -P my_project -R “r1m rusage[dummy=1]”-n 90
sLeep 10
LSB_SUB_PARM_FILE 的内容将是：
LSB_SUB_QUEUE="normaL"      #提交队列名
LSB_SUB_EXCLUSIVE=Y         #“Y”指定独占地执行
LSB_SUB_RES_REQ="r1m rusage[dummy=1]"    #资源需求串
LSB_SUB_PROJECT_NAME="my_project"     #工程名
LSB_SUB_COMMAND_LINE="sLeep 10"     #bsub 作业命令参数
LSB_SUB_NUM_PROCESSORS=90       #需要的最小处理器数
LSB_SUB_MAX_NUM_PROCESSORS=90    #需要的最大处理器数
```

- **LSB_SUB_ABORT_VALUE**

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

这个变量指示如果 volclava 拒绝这个作业提交，esub 应该的退出值。

- **LSB_SUB_MODIFY_ENVFILE**

esub 把任何作业环境变量的改变写入这个文件。esub 把需要被修改的变量写入这个文件，格式与 LSB_SUB_PARM_FILE 一样。变量的顺序不被影响。

在 esub 运行后，volclava 检查 LSB_SUB_MODIFY_ENVFILE 是否被改变，并且如果被改变了，volclava 将把他们应用至作业环境变量。

- **LSB_SUB_MODIFY_FILE**

esub 把任何提交参数的改变写入这个文件。esub 把需要被修改的作业选项写入这个文件，格式与 LSB_SUB_PARM_FILE 一样。选项的顺序不被影响。

在 esub 运行后，volclava 检查 LSB_SUB_MODIFY_FILE 是否被改变，并且如果被改变了，volclava 将把他们应用至作业。

提示：LSB_SUB_ADDITIONAL 不能在 LSB_SUB_MODIFY_FILE 中被改变或被添加 LSB_SUB_MODIFY_FILE。 LSF_INVOKE_CMD ， 指示最后一个 volclava 命令的名字（bsub、bmod、lsrun）。这个变量包含最后一个调用执行体的 volclava 命令的名字。

3. **通用 esub 逻辑**

在 esub 运行后，volclava 检查：

1） esub 的退出值是否是 LSB_SUB_ABORT_value?

a） 是，跳至第 2 步

b） 否，跳至第 4 步

2） 拒绝这个作业

3） 跳至第 5 步

4） LSB_SUB_MODIFY_FILE 或者 LSB_SUB_MODIFY_ENVFILE 是否存在。

a） 应用改变

5） 结束

4. **拒绝作业**

依赖于你的策略，你可能选择拒绝一个作业。为了做到这些，让 esub 的退出值是 LSB_SUB_ABORT_VALUE。如果 esub 拒绝这个作业，它不应该写至 LSB_SUB_MODIFY_FILE 或 LSB_SUB_MODIFY_ENVFILE。

例如：

下面的 bourne shell esub 通过 LSB_SUB_ABORT_VALUE 退出值拒绝所有的作

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

业提交：

```
#!/bin/sh
# Redirect stderr to stdout so echo can be used for
# error messages
exec 1>&2
# Reject the submission
echo "volclava is Rejecting your JOB submission..."
exit $LSB_SUB_ABORT_VALUE
```

5. **验证作业提交参数**

验证的一个用处是支持基于 project 的计数。用户能要求被一个作业使用的资源被某个特殊的 project 收费。在作业提交时，project 被连接至作业。所以 volclava 将接受任意字符串作为 project 名字。为了保证只有有效的 project 被进入并且用户是符合这个 project 的收费条件的，可以通过撰写 esub 来实现这一功能。

例如：下面的 bourne shell esub 验证作业提交参数：

```
#!/bin/sh
. $LSB_SUB_PARM_FILE
# Redirect stdout to stderr so echo can be used for error
messages
exec 1>&2
# Check vaLid projects
if [ $LSB_SUB_PROJECT_NAME != "proj1" -o
$LSB_SUB_PROJECT_NAME != "proj2" ]; then
echo "Incorrect project name specified"
exit $LSB_SUB_ABORT_value
fi
USER=whoami
if [ $LSB_SUB_PROJECT_NAME = "proj1" ];
then # OnLy user1 and user2 can charge to proj1
if [$USER != "user1" -a $USER != "user2" ]; then
echo "You are not allowed to charge to this project"
exit $LSB_SUB_ABORT_VALUE
fi
fi
```

6. **修改作业提交参数**

esub 能被用于在作业被实际提交前修改提交参数和作业环境。下面的例子把下面参数的修改写进 LSB_SUB_MODIFY_FILE：

- LSB_SUB_QUEUE

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

- USER
- shell

在这个例子，用户 userA 能仅仅提交作业至队列 queueA。用户 userb 必须使用 bourne shell（/bin/sh）。并且用户 userC 应该永远不能提交任何作业。

```
#!/bin/sh
. $LSB_SUB_PARM_FILE
# Redirect stderr to stdout so echo can be used for error
messages
exec 1>&2
USER=`whoami`
# Ensure userA is using the right queue queueA
if [ $USER="userA" -a $LSB_SUB_QUEUE != "queueA" ]; then
echo "userA has submitted a JOB to an incorrect queue"
echo "...submitting to queueA"
echo 'LSB_SUB_QUEUE="queueA"' > $LSB_SUB_MODIFY_FILE
fi
# Ensure userb is using the right shell (/bin/sh)
if [ $USER="userb" -a $shell != "/bin/sh" ]; then
echo "userb has submitted a JOB using $shell"
echo "...using /bin/sh instead"
echo 'shell="/bin/sh"' > $LSB_SUB_MODIFY_ENVFILE
fi
# Deny userC the abiLity to submit a JOB
if [ $USER="userC" ]; then
echo "You are not permitted to submit a JOB."
exit $LSB_SUB_ABORT_VALUE
fi
```

4.5.3 缺省的作业控制动作

在一个作业被启动后，这个作业能被系统杀死、挂起、或者恢复（volclava 用户或 volclava 管理员执行操作）。volclava 作业控制动作能改变作业的状态。volclava 支持下面的作业控制缺省动作：

- SUSPEND
- RESUME
- TERMINATE

一旦成功完成作业控制动作，volclava 作业控制命令导致作业的状态改变。 当一

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

个作业控制动作被初始化时，环境变量 LS_EXEC_T 被设置为 JOB_CONTROLS

1. **SUSPEND 动作**

   改变一个运行中作业的状态从 RUN 状态到下面状态中的一个：

   - USUSP 或 PSUSP 状态（响应 bstop）
   - SSUSP 状态（当 volclava 系统挂起作业时）

   缺省的动作是发送下面的信号量给作业： SIGTSTP（并行作业或交互式作业）。SIGTSTP 被 master 进程捕获并传递给所有运行在其他机器上的 sLave 进程。

   - SIGSTOP（串行作业）。SIGSTOP 不能被用户程序捕获。SIGSTOP 信号量能在 lsf.conf 中使用 LSB_SIGSTOP 参数配置。

2. **volclava 调用 SUSPEND 动作**，当：

   - 用户或 volclava 管理员向作业发出一个 bstop 或 bkill 命令
   - 执行机器上的负载条件满足任一：

     Ø 队列的挂起条件，在 lsb.queues 中 STOP_COND 参数指定的

     Ø 队列或执行机器的调度阀值

   - 队列的运行窗口关闭

3. **RESUME 动作**

   改变一个挂起作业的状态从 SSUSP、USUSP 或 PSUSP 状态到 RUN 状态。缺省的动作是发送信号量 SIGCONT。

   volclava 调用 RESUME 动作，当：

   - 用户或 volclava 管理员向作业发出一个 bresume 命令
   - 执行机器上的负载条件满足所有：

     Ø 队列的恢复条件，在 lsb.queues 中 RESUME_COND 参数指定的

     Ø 队列或执行机器的调度阀值

   - 队列的关闭的运行窗口被重新打开

4. **TERMINATE 动作**

   终止一个作业。这实际上导致作业变成 EXIT 状态。缺省的动作是先发送信号量 SIGINT，然后 10 秒钟后发送信号量 SIGTERM，再 10 秒后发送信号量 SIGKILL。信号量之间的延时允许用户程序捕获信号量并在作业终止前做一些清理。

   为了覆盖这 10 秒间隔，使用 lsb.params 文件中的

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

JOB_TERMINATE_INTERVAL 参数

volclava 调用 TERMINATE 动作，当：

- 用户或 volclava 管理员向作业发出一个 bkill 或 brequeue 命令
- 队列定义（lsb.queues）中的 TERMINATE_WHEN 参数导致一个 SUSPEND 动作以重定向至 TERMINATE
- 作业到达了它的 CPULIMIT、MEMLIMIT、RUNLIMIT 或 PROCESSLIMIT。

如果一个动作的执行正在进行，没有新的动作会被初始化，除非这个新动作是 TERMINATE 动作。为所有状态的作业发出一个 TERMINATE 动作，除了 PEND 状态。

4.6 自定义作业的工作目录

4.6.1 CWD 功能说明

CWD （Current Working Directory）是 volclava 对作业工作目录管理机制的增强实现，支持为每个作业动态指定独立的执行工作目录，并在作业生命周期结束后按 TTL 策略自动清理残留目录。

关键名词解释：

名词	含义
cwd (Current Working Directory)	作业执行时的当前工作目录，作业的所有相对路径操作都基于该目录，目前在命令 bsub、bmod 和配置文件 lsb.params 中已经支持-cwd 参数，用以配置工作目录。 cwd 未定义时则使用默认的/tmp 路径来执行任务。
submitCwd	提交目录（`bsub` 运行时所在目录），用于相对 `DEFAULT_JOB_CWD` 解析与 `bmod -cwdn` 回退
DEFAULT_JOB_CWD	`lsb.params` 中的集群级默认 CWD 模板（可含动态模板）
JOB_CWD_TTL	`lsb.params` 中动态 CWD 目录的存活时间

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

	（小时）
SUB2_JOB_CWD_PATTERN	标志位（`0x8000`）：cwd 路径含动态路径
动态路径	`%J`(jobId)、`%I`(数组下标)、`%U`(用户名)、`%P`(项目名)、`%H`(首个执行节点)

4.6.2 lsb.params 参数配置说明

在 volclava 配置文件 lsb.params 中支持如下参数设置：

配置项	语法	说明	默认值
DEFAULT_JOB_CWD	DEFAULT_JOB_CWD=<路径>	集群全局默认作业工作目录，支持动态占位符和跨平台路径配置	未定义，使用执行 bsub 的提交目录
JOB_CWD_TTL	JOB_CWD_TTL=<小时数>	作业工作目录自动清理的生存时间 系统每 300 秒扫描一次待清理目录列表，仅删除路径的最后一级目录，避免多个共享父目录的作业互相影响；volclava 服务启动时会自动扫描清理过期的工作目录。	未定义，默认数值 2147483647（永不删除）

其中 JOB_CWD_TTL 对于工作目录清理限制条件：

- 工作目录含动态路径
- 作业自己创建的路径
- 只删路径最后一级目录

以上参数的详细使用说明请参见 lsb.params 用户手册（例如：在集群环境中 man lsb.params）。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

4.6.3 cwd 维护机制说明

所有 CWD 记录持久化在 /tmp/.volclava.sbd/cwdlist，格式为：

<路径长度> <路径> <作业 ID> <完成时间戳> <TTL 秒> <设备号> <Inode 号>

- 首次创建时权限固定为 0644，不受 umask 影响，确保后续所有 sbatchd 进程可读写。
- 每行一条记录，字段间以空格分隔，路径中不含空格（空格在作业提交阶段已校验拒绝）。
- sbatchd 周期性（300 秒）读取 cwdlist 文件内容，检查并清理过期的 cwd。

清理每个过期 CWD 前，执行三重校验：

a. 路径类型检查：确认目标仍为目录（非符号链接，使用 rmdirNoFollow，不跟随符号链接递归删除）。

b. Dev/Ino 身份验证：将当前目录的 (dev, ino) 与 cwdlist 记录的 (dev, ino) 比对，只有完全一致才允许删除。防止以下误删场景：

- 原 CWD 被 mv 到其他位置、同名新目录替换。
- 挂载点变化导致路径指向不同文件系统。

c. 一次性尽力删除：rmdirNoFollow 递归删除目录树。成功后从 cwdlist 中移除记录；失败则记录 WARNING、丢弃该记录（不再跟踪），不重试、不阻塞后续记录的清理。

5. 交互式作业

5.1 用 bsub 来提交交互式作业

5.1.1 交互式作业

1. **调度策略**

对于资源密集型作业，交互式作业可让你利用批处理调度策略和机器选择功能。在一个队列中，交互式批处理作业和其它作业一样，使用相同的策略来调度。这意味着，交互式作业在被派发之前，可能需要等待很长的时间。如果需要快速反应时间，交互式作业应该被提交到调度限制比较松散，但优先级很高的队列中。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

2. **交互式队列**

你可以通过 lsb.queues 文件中的 INTERACTIVE 参数，来配置队列，使其只支持交互式作业，只支持批处理作业，或者是两者都支持。

5.1.2 提交交互式作业

使用 bsub –I 选项来提交批处理交互式作业，使用 bsub –Is 和-Ip 选项在伪终端提交批处理交互式作业。

1. **找出哪些队列接受交互式作业**

在你提交交互式作业之前，需要使用 bqueues –l 命令来找出哪些队列接受交互式作业。如果该命令的输出中包含下面的信息，说明该队列只支持批处理作业，不支持交互式作业：

SCHEDULING POLICIES: NO_INTERACTIVE .

如果该命令的输出中包含下面的信息，说明该队列只支持交互式作业：SCHEDULING POLICIES: ONLY_INTERACTIVE 。

如果以上两种信息都没有，或者 SCHEDULING POLICIES 没有在 bqueeues -l 命令的输出中，那么该队列既支持交互式作业，也支持批处理作业。 您可以在 lsb.queues 文件中配置交互式队列。

2. **提交一个交互式作业**

使用 bsub –I 选项来提交一个交互式批处理作业

例如： bsub –I ls

提交一个交互式批处理作业：在用户的终端显示 ls 的输出结果。

```
% bsub -I -q interactive -n 4,10 lsmake
<Waiting for dispatch ...>
```

该命令在 4 至 10 个处理器上启动 PLatform Make，并在终端显示输出结果。 直到交互式作业完成或终止，才能提交新的作业。

当交互式作业被提交，作业在等待调度时，会有信息显示。bsub 命令将停止显示从 shell 的输出，直到该作业完成，默认没有邮件发送给用户。用户可以使用 ctrl –c 在任何时候来终止该作业。

交互式作业不能设置检查点。

交互式作业不能重运行（bsub -r）。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

您可以将交互式批处理作业提交到可重运行的队列（在 lsb.queues 文件中 RERUNNABLE=y）中

3. **bsub –Ip**

使用 bsub –Ip 选项，用伪终端来提交一个批处理交互式作业

例如： % bsub -Ip vi myfile

提交一个编辑 myfile 的交互式作业。

当你指定-Ip 选项时，bsub 命令提交一个批处理交互式作业，并且在该作业开始时建立一个伪终端。一些应用，例如 vi，需要一个伪终端，以便正常运行。

4. **bsub –Is**

使用 bsub –Is 选项，提交批处理交互式作业时，能创建一个特定 shell 模式的伪终端。

例如： % bsub -Is csh

提交一个批处理交互式作业，该作业启动 csh，将其作为交互式 shell。

当你指定-Is 选项时，bsub 命令提交一个批处理交互式作业，并且在该作业开始时建立一个特定 shell 模式的伪终端。该选项应指定提交时的交互式 shell， 或者用 CTRL –C 和 CTRL –Z 重新界定的应用（例如 jove）。

5. **bsub –i, -o, -e**

你能使用-I 选项和-i， -o， -e 选项一起，用 bsub 来选择重定向流文件。

当标准输入和标准输出来自终端时，为了保存标准错误流到 JOB.err 文件:

% bsub -I -q interactive -e JOB.err Lsmake

6. **划分 stdout 和 stderr**

如果你的环境中有一个围绕 bsub 和 volclava 命令的封装，使最终用户不知道 volclava 和 volclava 的具体选项，就可以在批处理交互式作业中用>操作符，将标准输出和标注错误信息重定向。

默认情况下，在批处理交互式作业中，标准错误和标准输出信息都写入提交主机的 stdout 内。

将 stderr 和 stdout 信息写入 mystdout:

bsub -I myJOB 2>mystderr 1>mystdout

在 lsf.conf 文件中设置 LSF_INTERACTIVE_STDERR=y，或者作为环境变量，以便将 stdout 和 stderr 重定向到不同的文件

bsub -I myJOB 2>mystderr 1>mystdout

stderr 重定向到 mystderr，stdout 重定向到 mystdout。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

5.1.3 交互式批处理作业信息

1. **作业等待状态**

   当一个作业在等待状态时，显示的信息如下：

```
bsub -Is -R "ls < 2" csh
JOB <2812> is submitted to defauLt queue .
<<Waiting for dispatch ...>>
<< JOB's resource requirements not satisfied: 2 hosts; >>
 << Load information unavaiLabLe: 1 host; >>
 << Just started a JOB recentLy: 1 host; >>
 << Load information unavaiLabLe: 1 host; >>
 << JOB's resource requirements not satisfied: 1 host; >>
```

2. **作业被用户终止**

   当一个在等待状态的作业被用户终止时，显示的信息如下：

```
bsub -m hostA -b 13:00 -Is sh
JOB <2015> is submitted to defauLt queue <normaL>
JOB wiLL be scheduLed after Fri Nov 19 13:00:00 1999
<<Waiting for dispatch ...>>
<< New JOB is waiting for scheduLing >>
 << The JOB has a specified start time >>
 bkill 2015
 << JOB <2015> has been terminated by user or administrator >>
<<Terminated whiLe pending> >
```

3. **作业挂起，然后重启**

   当一个作业被分发，挂起，然后重启时，显示的信息如下：

```
bsub -m hostA -Is sh
JOB <2020> is submitted to defauLt queue <normaL>
 <<Waiting for dispatch ...>>
<< New JOB is waiting for scheduLing >>
<<Starting on hostA> >
bstop 2020
<< The JOB was suspended by user >>
bresume 2020
<< Waiting for re-scheduLing after being resumed by user >>
```

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

5.1.4 用 bsub 运行 X 应用

你可以启动一个 X 会话，将其提交作为批处理作业，提交到负载最轻的节点：bsub xterm 一个 xterm 进程将在该集群内负载最轻的机器上启动。 当你使用 lsrun 或者 bsub 运行 X 应用时，DISPLAY 环境变量将为你妥善处理。 它就像你在本地机器上运行 X 应用一样。

5.1.5 写作业脚本

例如：

```
% bsub -q simuLation
bsub> cd /work/data/myhomedir
bsub> myJOB arg1 arg2 ......
bsub> rm myJOB.Log
bsub> ^D
JOB <1234> submitted to queue
```

在上面的例子中，三行命令作为一个 bourne shell（/bin/sh）脚本来运行。在这种情况下，只接受有效的 bourne shell 命令行。

1. **在文件中指定作业选项**

   在下面的例子中，运行作业的选项在 ptions_fiLe 文件中指定。

   % bsub -q simuLation < options_fiLe

   JOB <1234> submitted to queue <simuLation>

   options_file 文件必须是一个包含 bourne shell 命令行的文本文件。 它不能是一个二进制可执行文件。

2. **缓冲作业命令文件**

   使用 bsub –Zs，将作业命令文件缓冲到 lsb.params 文件中 JOB_SPOOL_DIR 参数指定的目录，并使用该缓冲文件作为作业的命令文件。在作业提交后，可以使用 bmod –Zsn 命令来修改或者删除命令文件。删除或者修改原始输入文件，不影响提交的作业。

3. **重定向脚本为 bsub 标准输入**

   你可以重定向脚本为 bsub 命令的标准输入：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
% bsub < myscript
Job <1234> submitted to queue <test> .
```

在这个例子中，myscript 文件既包含作业提交的选项，也包含要执行的命令行。

当 bsub 命令从它的标准输入读取一个脚本时，它能为下一个作业的提交而修改 busb 的返回。

当脚本中指定 bsub 命令行，并且脚本没有被缓冲：

```
% bsub myscript
JOB <1234> submitted to defauLt queue .
```

在这种情况下，myscript 文件的命令行被缓冲，代替了原来 myscript 文件的内容。稍后对 myscript 文件的修改，将影响作业的行为。

4. **指定嵌入式提交选项**

你可以用脚本从标准输入读取，以#BSUB 开头的 bsub 命令行，从而指定提交作业的选项：

```
% bsub -q simulation
bsub> #BSUB -q test
bsub> #BSUB -o outfile -R "mem>10"
bsub> myjob arg1 arg2
bsub> #bsub -J simjob
bsub> ^D
JOB <1234> submitted to queuen <simuLation>
```

注意：

- 命令行选项会覆盖嵌入式选项。在这个例子中，作业被提交到 simulation 队列而不是 test 队列。
- 提交选项可以标准输入的任何地方被指定。在上面的例子中，bsub 的-J 选项在 ¨ 命令执行之后被指定。
- 像上一个例子所示，一行命令可以指定多个选项。

5. **在特定的 shell 下运行作业**

默认情况下，volclava 使用 bourne（/bin/sh）shell 来运行批处理作业。你可以在运行作业时指定特定的 shell。可以在脚本的第一行指定注释来做到这一点。

例如：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

```
% bsub
bsub> #!/bin/csh -f
bsub> set coredump=‘Ls |grep core‘
bsub> if ( "$coredump" != "") then
bsub> mv core core.‘date | cut -d" " -f1‘
bsub> endif
bsub> myJOB
bsub> ^D
JOB <1234> is submitted to defauLt queue <normaL>
```

bsub 命令必须从标注输入读取作业脚本来指定执行的 shell。如果没有在脚本中指定 shell，脚本将使用/bin/sh。如果脚本第一行以#开始，但是紧跟的不是感叹号（!），那么会使用/bin/csh 来运行该作业。

例如：

```
% bsub
bsub> # This is a comment Line. This teLLs the system to use
/bin/csh to
bsub> # interpret the script.
bsub>
bsub> setenv DAY ‘date | cut -d" " -f1‘
bsub> myJOB
bsub> ^D
JOB <1234> is submitted to defauLt queue <normaL>
```

如果经常有运行作业时要指定 shell 的需求，你可以使用命令行作业启动器指定轮流 shell，也可以交互式的运行你的作业。

6. 集群的监控

6.1 错误与事件日志

6.1.1 系统目录和日志文件

volclava 为临时工作，日志文件，执行文件和文件池，指定目录。volclava 将通过在作业子树下维护一个执行日志来跟踪系统中的所有作业。这些 volclava 日志文件可以在 VOLCLAVA_TOP/work/logdir 目录下找到。 下面的文件包含了 volclava 系统的状态。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

1. **lsb.events**

volclava 使用 lsb.events 文件来保持跟踪所有作业的状态。每一个作业都经过从作业提交到作业完成的过程。volclava 系统记录每一件与作业相关的日志，并存储在 lsb.events 文件中。

2. **lsb.events.n**

事件文件会被自动地调整，旧的作业事件会被存储在 lsb.event.n 文件中。当 mbatchd 启动后，将只提交到 lsb.events 这个文件，而不会提交到 lsb.events.n 文件中。bhist 命令可参考这些文件。

3. **在信息目录下的作业脚本文件**

当一个用户在 shell 提示符下运行 bsub，volclava 会收集运行在 bsub 行以及存储这些数据到 mbatchd 命令，这会存储 bsub 命令脚本在信息目录（或者是 lsb.params 文件中的 MAX_INFO_DIRS 参数指定的目录）为在分发时间使用或者如果作业重新运行。信息目录由 volclava 管理，并且不能被任何人所修改。

4. **日志目录权限与所有权**

确保 **LSF_LOGDIR** 目录的 owner 是 volclava 管理员，并且 root 用户可以写入此目录

日志级别与说明。

编号	层次	说明
0	LOG_EMERG	仅记录这些系统不可用信息。
1	LOG_ALERT	仅记录操作必须马上被执行这些信息。
2	LOG_CRIT	仅记录关键的一些信息。
3	LOG_ERR	仅记录指定错误条件的这些信息。
4	LOG_WARNING	仅记录一些警告或者非常严重的信息。这是调试信息默认的级别。
5	LOG_NOTICE	记录一些正常但又重要的条件,或警告和非常严重的信息。
6	LOG_INFO	记录所有通知信息以及非常严重的信息。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

7	LOG_DEBUG	记录调试信息。
8	LOG_DEBUG1	LOG_DEBUG1 , LOG_DEBUG2, LOG_DEBUG3 记录非常细节的调试信息，一般用于开发者调试程序或者有目的性的问题诊断。

6.1.2 错误日志管理

错误日志在 volclava 操作中是很重要的信息。当你在 volclava 中遇到任何不正常行为时，你首先应该检查相关的错误日志，然后找到问题的原因。 volclava 日志随着时间在增多。这些文件应该间或地被手动或自动清除。

1. **守护进程错误日志**

volclava 日志文件当每一个信息被记录时，每次都被打开。如果你重命名或者移动了一个守护进程日志文件，那么这个守护进程会自动创建一个新的日志文件。

当 volclava 守护进程检测到问题或者不正常的状态时就会记录有关信息。

可以配置守护进程输出这些信息到文件。

volclava 系统守护进程的错误日志文件名如下:

- res.log.host_name
- sbatchd.log.host_name
- mbatchd.log.host_name

2. **设置日志文件所用者**

必要条件:

你必须为群集的管理员。性能检测度量（perfmon）必须可使用或者你必须设置 LC_PERFM 为调试。 你可以为 volclava 守护进程设置日志文件所有者。默认的所有者为 volclava 管理员。

限制:

仅适用于 linux 主机

限制:

这种变化仅影响作为 root 用户运行的守护进程。

1 编辑 lsf.conf 并且增加 LSF_LOGFILE_OWNER 参数 。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

2 指定用户帐户的名字来设定日志文件的所有者。

3 关闭 volclava 守护进程或者想设置日志文件所有权的进程在主机运行 volclava shutdown

4 删除或移除任何存在的日志文件。

注意：如果你没有清除已经存在的日志文件，文件的所有者关系不会变更。

5 重启你关闭的 volclava 守护进程在主机运行 volclava startup 。

3. **查看文件操作符剩余数**

必要条件:

性能检测度量（perfmon）必须可使用或者你必须设置 LC_PERFM 为调试。

当提交一个大量作业给 volclava 时，mbatchd 守护进程能在一小段时间内记录大量的文件。你可以在任意时间查看剩余文件操作符。

限制:

仅适用于 linux 主机

运行 badmin perfmon view 空的，使用过的，以及所有文件操作符数都显示出来。

4. **错误日志**

如果选项 LSF_LOGDIR 的参数被在 lsf.conf 中定义，volclava 服务器错误信息被记录在这个目录下的文件中。

如果 LSF_LOGD 被定义但是守护进程没有写入到这儿的文件中，那么错误日志被创建在/tmp 下。

如果 LSF_LOGDIR 没有被定义，错误被使用 LOG_DAEMON 功能记录到系统错误日志（syslog）。系统日志信息可配置性很灵活，默认的配置根据系统的不同而不同。从搜索/etc/syslog.conf 文件开始，然后阅读 syslog（3）到 syslogd（1） 主页。

如果错误日志被系统错误日志所管理，那么可能会被自动地清除。 如果守护进程不能找到 lsf.conf，当他们启动时，他们将找不到 LSF_LOGDIR 的定义。在这种情况下，错误日志将写到系统错误日志。如果你在日志文件中找不到任何错误信息，他们就可能在系统错误日志中。

6.1.3 系统事件日志

volclava 守护进程保持一个事件日志在 lsb.events 文件中。mbatchd 守护进程使用这些信息来恢复服务器失败，主机重启，mbatchd 重新运行。lsb.events 文件同时被 bhist 命令用来显示关于 batch 作业可执行历史的详细信息，以及 badmin 命令来显

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

示主机，队列，守护进程的可操作历史记录。

在默认情况下，当 1000 个批作业完成后，mbatchd 会自动备份和重写 lsb.events 文件。这个值可以通过在 lsb.params 文件中 MAX_JOB_NUM 参数来设定。旧的 lsb.events 文件被移到 lsb.events.1。每一个旧的 lsb.events.n 都会被移到 lsb.events.n+1。而 volclava 不会删除这些文件。如果考虑到硬盘空间的问题，volclava 管理员应该周期性地压缩或者移出 lsb.events.n。

注意事项：不能移除或修改当前的 lsb.events 文件。移除或修改 lsb.events 可能会引起 batch 作业丢失。

6.1.4 复制事件日志记录

当恢复服器失败，主机重启，mbatchd 重新运行，volclava 可以使用存储在 lsb.events 中的信息。 如果包含主要自制日志的主机失败，volclava 将会继续操作使用复制日志。当主机恢复后，volclava 使用这些复制日志来更新主拷贝。

1. **如何复制日志**

默认情况下，事件日志位于 LSB_SHAREDIR。通常情况下 LSB_SHAREDIR 位于一个可信赖的文件服务器，同时为了运行作业的必要性也包含了其它关键应用程序，一但如果主机不可用，那么 volclava 接下来的失败是一个次要的问题。 当你配置复制日志后，复制将会在文件服务器不断产生，主事件日志被存储在第一个主主机上。换句话说，LSB_LOCALDIR 被用来存储批状态信息的主拷贝，LSB_LOCALDIR 的内容被复制一个拷贝到 LSB_SHAREDIR，这个位于中央文件服务器。这会带来以下影响：

- 建立 lsb.events 备份副本
- 减小中央文件服务负载
- 增加 volclava 主机负载

2. **文件服务器失效**

如果包含 LSB_SHAREDIR 的文件服务器失效，volclava 会继续处理作业。客户直接从 LSB_SHAREDIR 读的命令例如 bhist 命令 bhist 将不会工作。 当文件服务器恢复以且，当前的日志文件会被复制到 LSB_SHAREDIR 中。

3. **第一个主结点失效**

如果第一个主结点失败，那么在 LSB_LOCALDIR 中的文件主拷贝变成不可用。接下来一个新的主机会被选中。这个新主机使用在 LSB_SHAREDIR 中的复制文件来恢复它的状态并且记录以后的事件。不会有第二次或更下次的 volclava 主机复制发生。

当第一个主机在失败后恢复，它将会更新来自位于 LSB_LOCALDIR 的复制文件

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

主拷贝，然后继续如同以前一样工作。

如果第一个主机还没有恢复，volclava 会继续使用在 LSB_SHAREDIR 中的文件，但是没有更多的日志文件副本。

4. **两个主结点同时失效**

如果包括 LSB_LOCALDIR 的主结点和包括 LSB_SHAREDIR 的文件服务器同时失效，那么 volclava 将会不可使用。

5. **网络分区**

我们认为网络分区不会引起群集分成两个独立的群集，而两个同时地运行 mbatchd。

鉴于网络拓扑与失效模型，这个是可能会发生。例如，第一主机 M1 与文件服务器和次主机 M2 的连通性可能会丢失。M1 和 M2 都会运行 mbatchd 服务，M1 记录日志事件到 LSB_LOCALDIR，M2 记录日志到 LSB_SHAREDIR。当连通性恢复时，M1 从来自 LSB_LOCALDIR 中它的拷贝更新 LSB_SHAREDIR 时， M2 到 LSB_SHAREDIR 的变化可能将被丢失。

压缩事件文件只能在 LSB_LOCALDIR 可用，所以在网络分区情况下，例如 commands 命令不能访问这些文件。做为预防，你应该定时地从 LSB_LOCALDIRLSB_SHAREDIR 中复制出压缩文件。

6. **设置事件更新间隔**

如果 NFS 流量很高，你也想降低网络流量，你可以使用在 lsb.params 中的 EVENT_UPDATE_INTERVAL 参数来指定多长时间来备份数据以及同步 LSB_SHAREDIR 和 LSB_LOCALDIR 目录。目录通常在数据写入记录日志文件和 mbatchd 被在第一个 volclava 主机启动时被同步。

7. **事件日志**

压缩事件日志，lsb.events.n 不会被复制到 LSB_SHAREDIR 中。，如果 volclava 启动一个新事件日志，当包括 LSB_SHAREDIR 文件服务器当机，可能会注意到在 LSB_SHAREDIR 中历史数据的缺口。

8. **配置复制日志**

使得可以复制日志，设置 lsf.conf 中的 LSB_LOCALDIR 为一个在第一个主机的目录（第一个主机在 lsf.cluster.cluster_name 配置），这将被用来存储 lsb.events 的主拷贝。这个目录必须在主第一个主机上唯一存在。

- 编辑 lsf.conf，设置 LSB_LOCALDIR 为唯一存在在第一主机上的当前目录。
- 使用 lsadmin 重新配置，badmin mbdrestart 使得配置起作用。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

6.1.5 volclava 作业退出码

volclava 检测作业运行的过程，并且返回作业本身的退出代码。volclava 通过 linux 平台的 wait3（）来收集退出代码。退出代码是系统结束的值。使用 bhist 来查看作业的退出代码。 例如，如果你的应用程序有一个详尽的退出代码 129，这时你可以在输出中查看退出代码 129。当你发出一个信号，即终止作业， bhist 报告这个信号或者信号值 +128 。如果返回状态大于 128，作业就被信号终止，因此 return_status - 128 = 信号。

示例:

返回 133 状态，作业以信号 5 结束（在大多数系统中的 SIGTRAP，133-128=5）。

返回 139 状态，作业以信号 11 结束（在大多数系统中的 SIGTRAP，139-128=11）， 这意味着应用程序有一个核心的 dump

注意，终止信号是与系统有关的，所以信号 5 不会在 SIGTRAP，而信号 11 可能不会 SIGSEGV 的所有 UNIX 系统。

6.2. 故障排除和错误信息

6.2.1 访问共享文件

volclava 一个常见的问题是不可访问文件，这是由于不统一的文件空间造成。如果一个任务运行在被请求的文件不能以相同的名字被访问的远程主机上，这时错误将会产生。如果用户当前的工作目录无法在远端主机上找到，volclava 几乎所有的交互式命令将会失败。

1. **linux 上的共享文件**

如果你正在运行的是 NFS ，重新排列装载 NFS 可能会解决这一问题。如果你的系统运行的是 automount 服务器，volclava 试图映射文件名，在大多数情况下会成功。如果使用共享 mount，这些文件的映射可能中断。在这种情况下，必须采取具体措施而避开它。Automount 映射必须通过 NIS 管理。当 volclava 试图映射文件名时，它认为 automounted 文件系统已经在/ tmp_mnt 目录下装载。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

6.2.2 volclava 上的常见问题

1. **LIM 自动停止**

   运行以下命令以检查是否有错误的 LIM 配置文件。

   lsadmin ckconfig -v 这显示大多数配置错误。如果这样没有报告任何错误，请检查 LIM 错误日志。

2. **LIM 不可用**

   有时 LIM 正常工作，但执行 lsload 命令时，输出如下的错误信息：Communication time out。

   如果 LIM 刚刚开始启动，这是正常的，因为 LIM 需要时间来通过读取配置文件来初始化和其他 LIM 联系。如果 LIM 在一两分钟内不可用，请检查你所使用主机上的 LIM 错误日志。

   当新启动或重新启动本地的 LIM 时，为了防止超时， 定义 lsf.conf 文件中的参数 LSF_SERVER_HOSTS 参数。客户端将联系在 LSF_SERVER_HOSTS 之一的 LIM，并执行命令， 它所提供的列表中至少有一个有 LIM 的主机可以正常工作或正在运行。

   当本地 LIM 正在运行，但在群集中没有主 LIM，volclava 应用程序将显示以下消息：

   Cannot locate master LIM now, try later.

3. **主 LIM 关闭**

   有时主 LIM 在正常运行，但执行 lsload 或 lshosts 命令时会出现下面的错误信息：Master LIM is down； try later

   如果在主 LIM 正在运行的主机上的/etc/hosts 文件被以回路 IP 地址（127.0.0.1 主机名）配置，volclava 客户 LIMs 不能连接到主 LIM。当主 LIM 启动时，它设定正式的主机名称和 IP 地址到回路地址。任何客户请求将得到以 127.0.0.1 的主 LIM 地址，并尝试连接到它，事实上在尝试访问本身。

   检查在/etc/hosts 上的主 LIM 的 IP 配置 。下面的例子错误地设置了主 LIM 的 IP 地址到回路地址：

   127.0.0.1 localhost myhostname

   下面的例子正确设置主 LIM 的 IP 地址：

   127.0.0.1 localhost

   192.168.123.123 myhostname

4. **RES 没有启动**

   请检查 RES 错误日志。

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

5. **用户权限被拒绝**

如果远程执行失败，出现下面的错误信息，远程主机将无法安全地决定请求远程执行的用户的用户 ID 。

User permission denied.

a. 检查远程主机上的 RES 错误日志，这通常包含一个更详细的错误信息。

b. 如果你使用的不是一个确定守护进程（ LSF_AUTH 中没有在 lsf.conf 文件中定义），那么所有的应用程序做远程执行必须被带有 setuid 位集合的 ROOT 用户所承认。这可以按照以下方式来做:

chmod 4755 filename

c. 如果二进制文件在 NFS-mounted 文件系统中，请确保该文件系统没有被以 nosuid 标示来装载。

d. 如果你使用的是识别守护进程（被 LSF_AUTH 定义在 lsf.conf 文件中）上，inetde 必须被配置来运行守护进程。确定的守护进程不能直接被运行。

e. 如果 LSF_USE_HOSTEQUIV 被定义在 lsf.conf 文件，检查目的主机上的 /etc/hosts.equiv 或 Home / 。rhosts 是否有客户主机名在里面。 在 /etc/hosts 和 /etc/hosts.equiv 的命名服务器上的不一致的主机名也可能引起该问题。

f. 在 SGI 主机上运行一个命名服务器，你可以尝试下面的命令来让主机名在调用命名服务器之前来查询代码来搜索/etc/hosts 这个文件。 setenv HOSTRESORDER "local,nis,bind".

6. **不一致的命名空间**

由于不一致的命名空间，命令可能会失败，会出现下面的错误消息:

chdir(...) failed: no such file or directorychdir

你正试图远程执行命令，然而你的当前的工作目录不在远程主机，或者你当前工作目录是映射到一个不同的名称的远端主机上。 如果你的当前工作目录在远程主机上不存在，你不应该在该主机上执行远程命令。

7. **在 linux 上**

如果目录存在，但被映射到一个不同名称的远端主机上，你需要创建符号链接，使它们一致。

volclava 可以使用 automount 解决大多数问题，但不是所有问题。该 automount 映射必须通过 NIS 管理。如果你正在运行 automount 而 volclava 无法找到目录远程主机 。

8. **守护进程自动停止**

首先，检查 sbatchd 和 mbatchd 错误日志。尝试运行以下命令检查配置。

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

badmin ckconfig

这个报告了最重要的错误。如果 mbatchd 正在运行，但一些主机上的 sbatchd 已死亡，这可能是因为 mbatchd 在这些主机上尚未配置使用。

9. **sbatchd 开始但 mbatchd 并没开始**

检查 LIM 是否正在运行。你可以通过运行 lsid 命令进行检测。如果 LIM 运行不正常，首先请按照这章的建议修复 LIM。有可能是 mbatchd 暂时不可用，因为主 LIM 暂时未知，造成下面的错误信息。 sbatchd: unknown service

检查是否正确注册服务。

10. **主机没有被 volclava 使用**

如果你在 lsb.hosts 文件 Host 部分配置了服务主机列表。mbatchd 允许 sbatchd 只在列表中的主机上运行。如果你尝试在 lsb.hosts 文件的 HostGroup 或 HostPartition 上配置未知的主机， 或作为主机定义一个队列中的 lsb.queues 文件，mbatchd 记录下面的消息：

```
Host hostname is not used by lsbatch; ignored
```

如果你在 mbatchd 无法识别的主机上启动 sbatchd， mbatchd 将拒绝 sbatchd，记录下面的消息：

```
sbatchd This host is not used by lsbatch system.
```

这两个错误，往往都是因没有运行以下命令，因此，需要增加主机到配置中。

lsadmin reconfig

badmin reconfig

你必须在新主机上开始运行这守护进程之前运行这两个命令。

11. **查看 UNKNOWN 主机类型或模式**

运行 lshosts 。一个的模式或类型 UNKNOWN 说明主机已关闭或主机上的 LIM 已关闭。你必须立即采取行动。

例如： lshosts

```
HOST_NAME type        model    cpuf  ncpus  maxmem  maxswp
server RESOURCES
hostA     UNKNOWN     Ultra2   20.2   2     256M    710M
Yes      ()
```

修复 UNKNOWN 匹配的主机类型或匹配的模式：

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

a. 启动主机。

b. 运行 lsadmin limstartup 命令在主机上启动 LIM。

例如:

```
lsadmin limstartup hostA
Starting up LIM on  .... done
```

你可以在多个主机上指定一个以上的主机名称来启动 LIM。如果你没有指定主机名，LIM 将会从命令提交的主机开始。

linux 平台上，为了远程启动 LIM，你必须是在根用户并且能够所有主机运行 rsh 命令而不需要输入密码。

c. 等待几秒钟，然后运行 lshosts 一次。你现在应该可以看到一个具体的模式，类型的主机或者默认的方式（DEFAULT）。如果你看到 DEFAULT，这意味着自动检测主机类型或模式已经失败，在 lsf.shared 中的主机类型配置无法找到。volclava 将在主机上运行，但 DEFAUL 模式可能是低效的，这是因为不正确的 CPU 因素。默认类型也可能会导致二进制不兼容，因为 DEFAULT 主机类型迁移到另一个预设主机类型上。

12. **查看 DEFAULT 主机类型或模式**

如果你使用 lim- t 来查看 DEFAULT 值，这意味着自动检测主机类型或模式已经失败，并且在 lsf.shared 中主机类型配置无法找到。volclava 将在该主机上工作， 但默认模式可能是无效的，这是由于不正确的 CPU 因素。默认类型也可能会导致二进制不兼容，因为 DEFAULT 主机类型迁移到另一个预设主机类型上。

运行 lshosts 。当你使用 lshosts 并且自动主机模式和类型检测启用，那么模式和类型将显示为 DEFAULT 。你可以不用管它是或改变它。例如:

```
lshosts
  HOST_NAME    type     model     cpuf ncpus maxmem maxswp
server RESOURCES
  hostA        DEFAULT  DEFAULT  1       2       256M
710M     Yes     ()
```

如果模式是 DEFAULT，volclava 将正常工作，但主机将有一个因子为 1 的 CPU，这可能无法有效地利用本地模型。

如果模式是 DEFAULT，有可能是二进制不兼容的。例如，有 2 个主机，一个是 Solaris，另一种是 HP。如果两个主机类型设置为 DEFAULT，它意味着在 Solaris 主机上运行的工作可以迁移到 HP 主机上， 反之亦然。

13. **修复 DEFAULT 匹配的主机类型或匹配的模式**

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

a. 在主机类型为 DEFAULT 上， 运行 lim -t:

```
lim -t
Host Type : LINUX86
Host Architecture : SUNWUltra2_200_sparcv9
Physical Processors : 2
Cores per Processor : 4
Threads per Core: : 2
Matched Type: DEFAULT
Matched Architecture: DEFAULT
Matched Model: DEFAULT
Note the value of Host Type and Host Architecture
```

b. 编辑 lsf.shared。在 HostType 节中，输入一个新的主机类型。使用 lim -t 检测来的主机。

例如:

```
Begin HostType
TYPENAME
DEFAULT
CRAYJ
LINUX86
 ...
End HostType
```

c. 在 HostModel 节，输入新主机架构的模型以及 CPU 的因素。使用检测 lim-t 的架构。在主机模型列表中的最后添加主机类型。主机类型的输入限制是 127，用 # 行注释掉的不计算在 127 线的限制内。

例如:

```
Begin HostModel
MODELNAME CPUFACTOR ARCHITECTURE # keyword
Ultra2 20 SUNWUltra2_200_sparcv9
End HostModel
```

d. 保存更改到 lsf.shared 文件

e. 运行 lsadmin reconfig 重新配置 LIM

f. 等待几秒钟，并运行 LIM- T，再次检查类型和主机型号

14. bjobs 查询不到刚刚提交的作业

管理员应该检查

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

- LSB_QMBD_PORT 是否已配置；
- LSB_QMBD_JOB_SYNC_MODE 是否配置

如果正确设置了 LSB_QMBD_PORT，而 LSB_QMBD_JOB_SYNC_MODE 设置为 off，那么新的作业会在 query mbatchd 重建后才会同步，需要将 LSB_QMBD_JOB_SYNC_MODE 设置为 socket（默认） 或者 shm 才能实时同步新提交的作业。

附录：

附一、变更历史

日期	**版本**	**变更描述**
2024-11-26	volclava 1.0.0	volclava 1.0.0 派生自 Openlava 2.0。 添加了以下功能和问题修复： • 多项功能支持：bjobs -UF；bjobs -o/-j-json; bsub -pack; bsub -Ep; 等等。 • 多处错误修复：设置 “！” 时 MXJ 与 maxCpus 不相等；lshosts -l 出现段错误； • sbatchd 因超过 1000 个任务而受阻；在 RPM 安装中前缀无法用于自定义目录； • 修复主机空闲时却达到作业槽位限制的问题。 • 在相关文件中将新项目名称定义为 volclava
2025-06-16	volclava 2.0.0	1. 《volclava 管理员手册》文档更新了一下内容： ◦ 第 1.6.3 节 取消 default 对用户组的影响 ◦ 第 3.2.3 节 定义资源预留方式 ◦ 第 3.3.2 节 公平共享调度策略 ◦ 第 3.9 节 定义最大排队任务数

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

		◦ 第 3.10 节 自定义资源限制的单位 2. volclava2.0.0 新特性和 bug 修复的明细: https://github.com/bytedance/volclava/blob/master/ChangeLog
2025-09-12	volclava 2.1.0	更新文档中 volclava “2.0”描述为“2.1”
2025-11-16	volclava 2.1.1	volclava2.1.1 新特性和 bug 修复的明细: https://github.com/bytedance/volclava/blob/master/ChangeLog
2026-6-30	volclava 2.2	1. 《volclava 管理员手册》文档更新了一下内容: ◦ 1.1.5 章节添加作业级别和队列级别资源请求合并规则 ◦ 1.2.5 章节增加了 query mbd 的功能介绍 ◦ 4.5.2 章节丰富了 esub 的定义方式 ◦ 完成图片清洗，使之更加适合知识库检索 2. volclava2.2 新特性和 bug 修复的明细: https://github.com/bytedance/volclava/releases#release-2.2.0
2026-10-30	volclava 2.2.1	1. 《volclava 管理员手册》文档更新了一下内容: ◦ 更新版本信息为 2.2.1 ◦ 1.2.1 章节添加 badmin showconf mbd/sbd, lsadmin showconf lim 的示例 ◦ 1.5.2 章节添加如何查看作业 slots/cpu 使用率 ◦ 新增 1.5.3 章节，介绍如何检查历史作业信息，介绍 bhist 加速服务的安装方式以及注意事项 ◦ 调整文章结构，把原来 3.1/3.2 章节的资源相关话题调整到 2 章节“资源的操作”下。 ◦ 增加 2.3.3 章节，介绍通用资源配额限制功能和使

ByteDance

Copyright (C) 2021-2026 ByteDance Ltd. and/or its affiliates

		用方法 ◦ 增加 4.6 章节 自定义作业的工作目录 2. volclava2.2.1 新特性和 bug 修复的明细: https://github.com/bytedance/volclava/releases#release-2.2.1

|（注：部分内容由豆包工作 AI 生成）

ByteDance